SHA-256 as a Mathematical Object:
A Field Guide and a Program of Small Questions

[Author name here]*

Last updated July 17, 2026, 4:03 PM Pacific

Abstract

The function SHA-256 maps arbitrary bit-strings of bounded length to 256-bit strings. It is one of the most heavily used mathematical objects on Earth, and one of the least understood: a quarter-century of study has produced neither a proof of any of its conjectured properties nor any observable deviation from the behavior of a random function. This note introduces SHA-256 historically and empirically—what it is for, why it has lasted—and then frames it geometrically, as a self-map of the finite space $(\mathbb{Z}/2^{32}\mathbb{Z})^8$ built by deliberately alternating two incompatible group structures on the same 2^{256}-point set—a clash the 2-adic metric brings into sharp focus, sorting the machine operations by whether they respect it and opening an unexpected line to the nonarchimedean dynamics of p-adic disks. We describe the frameworks that exist for analyzing such objects, why a large gap separates "passes every test we have" from "provably pseudorandom," and how the complex-analytic tradition of Flajolet and collaborators, brought to bear on the statistics of random mappings, offers concrete, measurable coordinates for locating SHA-256 in the space of pseudorandom generators. We then open the adversary's own toolbox—differential cryptanalysis, the method that felled this function's ancestors, with its remarkable secret history—and close with the view from computational complexity: SAT solvers, Impagliazzo's five worlds, and what theorem provers did and did not prove. Throughout, we flag self-contained subprojects suitable for undergraduates in short research windows.

Contents

*Working draft for mathematician colleagues. Comments very welcome.

A word on how to read what follows. This is a deliberately leisurely introduction: we approach SHA-256 the long way around, with excursions into history, epistemology, finite geometry, and analytic combinatorics, because the object of interest is not the algorithm so much as where it sits—its context, its significance, and the peculiar shape of what is and is not known about it. Readers impatient for the algorithm itself should know that it is short and entirely public: the Wikipedia page on SHA-2 contains a complete and perfectly accessible, if dense, definition in pseudocode, and the official standard (NIST's FIPS 180-4) is only a few pages longer; both are cited where the round function is anatomized (section 3.3). Nothing about the definition is hidden. Everything about its behavior is.

1 A pattern where none was promised

Concrete Mathematics opens with a small massacre (Graham et al., 1994, §1.1). Number n people $1, 2, \ldots, n$ standing in a circle, and going around the circle eliminate every second person until one survives. Which position $J(n)$ survives? For $n = 10$ the eliminations run $2, 4, 6, 8, 10, 3, 7, 1, 9$, and the survivor is $J(10) = 5$ (fig. 1). Computing small cases and staring at them reveals that if $n = 2^m + \ell$ with $0 \le \ell < 2^m$, then

$$J(2^m + \ell) \;=\; 2\ell + 1, \tag{1}$$

which says something better in binary: $J(n)$ *is the one-bit cyclic left shift of* n. For $n = 10 = (1010)_2$ we get $J(n) = (0101)_2 = 5$. An iterative algorithmic process ends up having a solution involving a simple bit manipulation of the binary representation of the input.

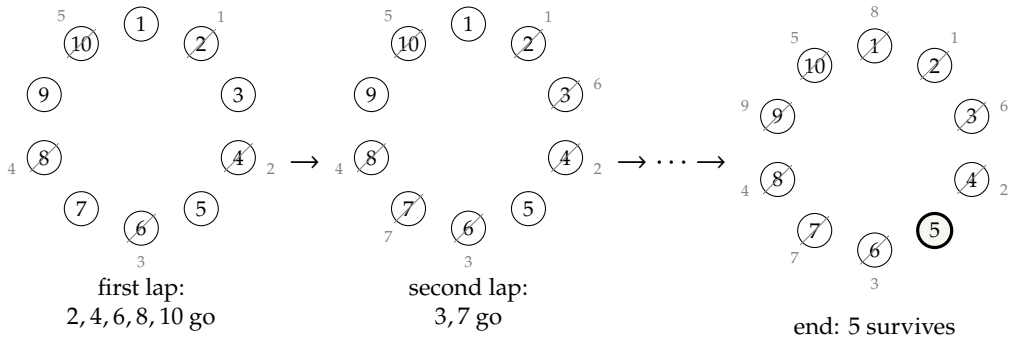


Figure 1: Circular decimation for $n = 10$, read left to right; the ellipsis elides the intermediate eliminations of the final lap. Small gray indices give the global elimination order; position 5 survives, and $J(10) = 5 = (0101)_2$ is the one-bit cyclic shift of $n = 10 = (1010)_2$.

Dwell for a moment on what *kind* of fact this is. The problem is defined *iteratively*: the survivor is specified as the outcome of $n - 1$ eliminations performed in order, and the obvious way to compute $J(n)$ is to simulate them, one by one, around the circle. The closed form abolishes the simulation. The answer is obtained by merely *rearranging the bits of the input*—one cyclic shift, no iteration at all: a computation that appeared to require $n - 1$ sequential steps

collapses to a single stroke. Nothing in the problem statement promised that such a collapse existed; it was simply there, waiting to be noticed.

It will pay immediately to name this operation the way the SHA-256 standard does (National Institute of Standards and Technology, 2015). For a w-bit word x and $0 \le r < w$, write

$$\mathrm{ROTL}^r(x), \qquad \mathrm{ROTR}^r(x), \qquad \mathrm{SHR}^r(x)$$

for the cyclic left rotation, the cyclic right rotation, and the ordinary (bit-discarding) right shift of x by r positions. In this notation the Josephus law eq. (1) is simply

$$J(n) \;=\; \mathrm{ROTL}^1(n),$$

the rotation taken in the $(\lfloor \lg n \rfloor + 1)$-bit window that n occupies. The rotation operators are *literal components of SHA-256*. Its round function stirs the state with fixed maps $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$, each an exclusive-or of two or three rotations of a 32-bit word (section 3.3)—the survivor map of a circle of 32 bits, composed with itself in threes. The same innocent operation that *creates* the visible structure in the Josephus problem is deployed in SHA-256 to *destroy* visible structure; and one of the very few theorems known about SHA-256's internals is a statement about exactly these xor-of-rotation maps (section 3.5).

Innocent discrete processes secretly respect structure, and the structure is usually found empirically—by computing small cases, arranging them suggestively, and noticing. The recurrence came first; the closed form eq. (1) was extracted from data.

The Josephus problem is not a lone curiosity; a small gallery of famous iterative processes collapse the same way, each to a one-line bit operation. The Tower of Hanoi—defined recursively, $2^n - 1$ moves to transfer n disks—has an entirely non-recursive solution in binary: at step t the disk to move is the position of the lowest set bit of t (the ruler sequence, the 2-adic valuation $v_2(t)$), and the disk configuration after step t is read directly off the reflected binary (Gray) code of t (Graham et al., 1994, §1.1). Gray codes themselves are the fixed point of this idea: the n-th codeword is simply $n \oplus (n \gg 1)$, and successive integers thereby differ in exactly one bit—an $\oplus$ and a shift where a naive account sees an elaborate recursion. The game of Nim is the most striking: an entire two-player combinatorial game, seemingly demanding lookahead, has optimal play given by a single bitwise exclusive-or—a position is lost exactly when the $\oplus$ of the pile sizes is zero (Bouton, 1901), the result that launched combinatorial game theory. In each case an apparently sequential, lookahead-hungry computation turns out to be a transparent function of the binary digits of its input. The moral compounds: bit operations are not just where these collapses *land*, they are a recurring *language* in which discrete iteration, when it is secretly simple, tends to become simple—and (section 3.2) they are precisely the alphabet from which SHA-256 is built.

This paper is about the mirror image of that phenomenon. The function SHA-256, which we will introduce slowly, is a discrete process on binary strings that was *engineered so that no analyst would ever have a Josephus moment with it*: no change of representation, no clever coordinate system, no generating function is supposed to reveal any respected structure whatsoever—even though the function is built from a few dozen lines of elementary operations, all of them individually as transparent as the decimation rule. SHA-256 too is defined iteratively—a fixed round transformation applied sixty-four times, itself chained block after block—and the working conjecture is that, unlike the decimation, *its iteration admits no collapse*: no bit-shift pattern, no closed form, no shortcut of any kind computes the result materially faster than

performing the rounds, and no property of the output is predictable without doing so. Whether the engineers succeeded is, remarkably, not a settled question but an empirical one, now running as an uncontrolled worldwide experiment for a quarter of a century. Making parts of that experiment controlled, small, and measurable is the research program of this note.

2 The object, observed from outside

Before any internals, here is the object as its users see it. SHA-256 is a single, fixed, publicly specified function

$$\mathsf{SHA\text{-}256} : \bigcup_{0 \le \ell < 2^{64}} \{0,1\}^{\ell} \longrightarrow \{0,1\}^{256},$$

published by the U.S. National Institute of Standards and Technology in 2002 and unchanged since (National Institute of Standards and Technology, 2015). It takes a bit string of any length below 2^{64}—the empty string, a single bit, the concatenated works of a library—and returns 256 bits. (We stress *bit* string: the standard defines the function on arbitrary bit lengths, not merely on whole bytes; software interfaces usually round to bytes, but the mathematical object does not. This matters in section 3.)

Two set-theoretic observations frame everything that follows. The domain is astronomically larger than the codomain, so the function is nowhere near injective: by pigeonhole, colliding pairs exist in unimaginable abundance—the average fiber over a 256-bit output is itself a set of size around $2^{2^{64}-256}$. Whether the function is *surjective*, on the other hand, is not known: nobody has proved that every 256-bit string is attained, and more generally nothing nontrivial is proved about the image size or about how evenly the domain is spread across the range—the property Bellare and Kohno isolate as hash-function *balance*, showing that deviations from it would measurably accelerate collision search (Bellare and Kohno, 2004); for SHA-256 the balance is simply unknown. Heuristically, of course, a random function with this domain misses a given output with probability too small to name, and section 4 is about taking such heuristics seriously as measurements.

The function is deterministic and involves no secrets: everyone can evaluate it, and everyone computes the same values.[1] It is also fast, in a way worth pausing on: the definition is in effect a small fixed circuit of bitwise operations and 32-bit additions, so it is directly expressible in hardware. A laptop core computes it at gigabytes per second; dedicated silicon does far better—purpose-built SHA-256 ASIC designs were an academic subject within two years of standardization (Dadda et al., 2004), and the economics of proof-of-work mining (section 2.1) later industrialized them, the arc from CPU software to modern mining ASICs gaining roughly six orders of magnitude in throughput per device and five in energy per hash (Taylor, 2017).

[1] "No secrets" means no secret *inputs*; it emphatically does not mean no design intelligence. Every constant and structural parameter was chosen with visible care, by a designer—the NSA—demonstrably in possession of nonpublic cryptanalytic knowledge. The clearest episode: NIST published the family's ancestor, now called SHA-0, in 1993; within two years the NSA had it withdrawn, citing an undisclosed weakness, and reissued with exactly one change, a single one-bit rotation added to the message schedule (National Institute of Standards and Technology, 1995). In 1998 academic cryptanalysis independently discovered a differential collision attack on SHA-0 that this tweak happens to block (Chabaud and Joux, 1998), and by 2005 explicit SHA-0 collisions had been computed (Biham et al., 2005), while the amended SHA-1 stood for another decade (section 2.3). Whatever process selects these functions, it is not simple random choice; section 3.6 discusses how the SHA-2 generation made its constants auditable in response to exactly this kind of asymmetry.

What is it *for*? The honest answer is: for behaving, in public, like a random function—a function sampled uniformly at random from the set of all functions into $\{0,1\}^{256}$—while actually being a short, fixed program. All of its uses are corollaries of facets of that one pretension.

2.1 A catalogue of uses

Fingerprinting. If H behaves like a random function, two different documents have the same 256-bit hash with probability 2^{-256}; a hash is a practically unforgeable fingerprint. The version-control system Git addresses every object it stores by its hash: file contents, directory trees, commits. Equality of fingerprints is treated as equality of content—mathematically unjustified and, at planetary scale, empirically flawless so far.

Commitment. Publish $H(x)$ today, reveal x next month; anyone can verify you did not change your mind, yet the hash revealed nothing about x. A random-looking function gives sealed envelopes without paper. (This is how one archives a prediction, a bid, or a claim of priority.)

Merkle trees. Hash the leaves; hash the concatenations of sibling hashes; repeat. The root is a 256-bit commitment to an arbitrarily large collection, and membership of any single leaf is certified by a logarithmic-length path (Merkle, 1988, 1990a). Certificate Transparency (the public audit log of all TLS certificates on the web) and every blockchain are Merkle trees at heart. Notice the epistemic shape of what the tree provides: the inclusion of one transaction among billions is *proved* by a chain of a few dozen hashes—a certificate anyone can verify in microseconds, revealing essentially nothing about the other leaves. Pushed further, this seed grows into the theory of zero-knowledge and succinct proof systems (Goldwasser et al., 1989): modern hash-based protocols such as STARKs commit to the execution trace of an entire computation with a Merkle tree, then prove its correctness by opening a few hash chains (Ben-Sasson et al., 2018). The pattern recurs everywhere in mathematical cryptography and deserves explicit statement: a task—here, inverting or colliding SHA-256—is *presumed* hard, and the presumed hardness then serves as a primitive from which richer structures (commitments, proofs, signatures, digital money) are built by explicit reduction. Hardness conjectures function the way axioms do elsewhere in mathematics: unproved, load-bearing, and generative.

Message authentication. With a shared secret key k, the construction HMAC(k, m), built from two nested SHA-256 calls, authenticates messages; its analysis (Bellare et al., 1996) is a rare case of a *theorem* in this landscape—assuming the compression function behaves as a pseudorandom family, a notion we take up in section 6.

Proof of work. Bitcoin asks miners to find a nonce making SHA-256(SHA-256(block header)) land below a target threshold (Nakamoto, 2008). If the function behaves randomly, there is no better strategy than exhaustive guessing, so a solution *is* verifiable evidence of expended computation. As of this writing the Bitcoin network evaluates the function roughly 10^{21} times per second—surely the most intensively exercised nontrivial function in the history of mathematics—and this entire expenditure is, among other things, a brute-force search for regularities that has found none.

Key derivation and pseudorandom generation. Operating systems stretch and mix entropy pools through SHA-256; standardized deterministic random-bit generators iterate it. Here

the function is used precisely *as* a pseudorandom number generator, which frames the question of section 6: where does it sit in the space of PRNGs?

Note how different the demands of these uses are: fingerprinting needs *collision resistance* (no collision $x \neq y$ with $H(x) = H(y)$ has ever been *published*—though note the epistemic fine print: were one discovered, its discoverer might have excellent reasons to keep it private), commitments need *hiding* and *binding*, proof-of-work needs *progress-freeness* (no shortcut to partial preimages), key derivation needs *pseudorandomness of outputs*. A taxonomy of these properties and their logical relations exists (Rogaway and Shrimpton, 2004), and the standard reference surveying the whole subject at textbook depth is the Handbook chapter of Menezes et al. (1996, Ch. 9); the striking fact is that one fixed function is trusted, simultaneously, with all of them.

2.2 The original hash functions

The word "hash" is decades older than its cryptographic sense, and the two meanings are worth separating before the cryptographic one takes over the paper. Hashing as a storage technique is nearly as old as stored-program computing: scatter-storage schemes circulated inside IBM from 1953 (in an internal memorandum of H. P. Luhn), reached the open literature with Peterson's study of random-access addressing (Peterson, 1957), and were codified, history and all, in Knuth's chapter on searching (Knuth, 1998, §6.4). In that tradition a hash function need only *scatter*; collisions are a fact of life to be chained through, probed around, and amortized away, and nobody is trying to *manufacture* one.

Here is a complete, honest member of that tradition, as a working one-liner,

```
def h(k):  return k % 997
```

—division hashing, filing the integer key k into one of 997 buckets (Knuth, 1998, §6.4). Mathematically it is the reduction map $\mathbb{Z} \to \mathbb{Z}/997\mathbb{Z}$, and because that map is a ring homomorphism, every anatomical question one could ask of it has a one-line proof. Its collisions are not merely abundant but *listable*: the keys colliding with k are exactly the arithmetic progression $k + 997\mathbb{Z}$. It is provably injective on a natural piece of its domain: any 997 consecutive keys land in 997 distinct buckets. It is provably surjective, with exact equidistribution: among any $997m$ consecutive keys, every bucket is hit exactly m times. And it respects algebraic structure outright: $h(j + k) \equiv h(j) + h(k)$ and $h(jk) \equiv h(j)\,h(k) \pmod{997}$.

How far does this transparency stretch toward SHA-256's actual machinery? Further than one might guess. Trade the prime modulus for a power of two and make the map affine,

$$h(k) = (a\,k + b) \bmod 2^n, \qquad a \text{ odd},$$

and *carries* enter the story: the multiplication and addition now ripple information from low-order bits into high ones—the 2-adically continuous operations we meet again in section 3.2. Yet nothing is lost to analysis. With a odd the map is a *bijection* of the n-bit words (no longer a compressing bucket hash but a reversible mixing map), invertible in closed form as $k = a^{-1}(h - b) \bmod 2^n$, where a^{-1} is the inverse of a modulo 2^n; iterated, it is precisely a linear congruential generator, whose period is a classical theorem (section 6.6). Now throw an exclusive-or on top,

$$h(k) = \big((a\,k + b) \bmod 2^n\big) \oplus c,$$

and—the pleasant surprise—it stays analyzable: an $\oplus$ with a constant is itself a bijection, living in the *rival* $\mathbb{F}_2$-geometry of section 3.2, and a composition of bijections is a bijection, so the whole map is a reversible permutation one can still invert by hand. What one now holds is a one-round baby SHA-256: two of its three ARX primitives (carry-bearing arithmetic and $\oplus$) already in place, and analyzable only because nothing yet forces the two geometries to *interfere*. SHA-256 adds the third primitive—bitwise rotation, the operation that couples the geometries—and iterates the mixture 64 times; it is that composition, not any single ingredient, that carries the design out of reach of every one-line argument above.

2.3 Cryptographic hash functions

SHA-256 is the survivor of an explicit evolutionary lineage, and the extinctions in that lineage are the best available evidence about what kind of object it is. The cryptographic branch split from the storage tradition of section 2.2 the moment someone set out to *manufacture* collisions rather than merely tolerate them—the moment, that is, an adversary entered the room. One-way functions appeared first as a systems trick for protecting stored login passwords (in use by the late 1960s; first carefully engineered constructions in Purdy, 1974; the classic Unix case history in Morris and Thompson, 1979); one-wayness was then named as a foundational primitive of the new public-key cryptography (Diffie and Hellman, 1976); and the modern object—a compressing, collision-resistant, *one-way hash function*—was isolated in Merkle's 1979 dissertation (Merkle, 1979). Same word as before, escalating demand: from "collisions are a performance nuisance" to "collisions exist in abundance, but no adversary on Earth can exhibit one." The escalation is a change of adversary model, not of machinery—and it took thirty years to complete.

Put the four-question anatomy exam of section 2.2 in front of SHA-256 now, and every clean answer becomes an open problem. Collisions: they exist in astronomical abundance (pigeonhole), yet none has ever been exhibited, and no known method materially beats generic birthday search at 2^{128} evaluations (section 4.3). Injectivity on a restricted domain: is SHA-256 injective on the 2^{128} inputs of exactly 128 bits? For a random function this would be very nearly a fair coin flip—the expected number of colliding pairs is about $\frac{1}{2}$—and for SHA-256 nobody can improve on that shrug in either direction. Surjectivity: unknown; a random function of these dimensions would miss at most a fantastically small fraction of its range, but nothing of the kind is proved. Respected structure: the design *intent* is the wholesale negation of the homomorphism property, and "respects nothing" is precisely the conjunction of unproved statements this paper keeps circling. Same species as the one-liner, same questions; but every answer has moved from a line of algebra to an open problem. The rest of the paper lives in that gap.

The modern era begins with Merkle's and Damgård's 1989 work putting hash functions on a design principle. Rather than design a function on arbitrary-length inputs directly, one fixes a single *compression function* f that maps a fixed-size state and one message block to a new state, and extends it to messages of any length by iterating: split the (padded) message into blocks $m_1, \ldots, m_k$, start from a fixed initial value, and fold the blocks through f one at a time,

$$h_0 = \mathrm{IV}, \qquad h_i = f(h_{i-1}, m_i), \qquad H(m) = h_k,$$

reading off the final chaining value h_k as the digest. The virtue of this construction is a *theorem*: any collision in the full hash H can be unwound into a collision in the single map f, so

collision resistance of the whole reduces to collision resistance of f alone (the Merkle–Damgård theorem, theorem 3.1). This is exactly the skeleton SHA-256 is built on, compression function and all—the finite map f and the two theorems that wrap it are anatomized in sections 3.1 and 3.4. Rivest's concrete proposals MD4 (1990) and MD5 (1992) (Rivest, 1992) were the first mass-market instances—128-bit hashes built as fast iterated compression—and MD5 was, for a decade, everywhere. NIST's SHA-1 (1995) enlarged the state to 160 bits and adjusted the internals; it inherited MD5's ubiquity. The SHA-2 family—of which SHA-256 is the 256-bit member—was standardized in 2002 with longer digests and a reworked, more conservative round function.

Then the towers fell, in order of age. In 2005 Wang and Yu exhibited explicit collisions in MD5 by hand-crafted differential analysis (Wang and Yu, 2005); within a few years refinements produced colliding documents in seconds on a laptop, and forged certificates in the wild. The same summer, Wang, Yin, and Yu broke SHA-1's collision resistance on paper, at cost 2^{69} evaluations versus the 2^{80} birthday bound (Wang et al., 2005); twelve years later the attack was industrialized: the SHAttered project published two distinct PDF files with equal SHA-1 hashes, at a cost of roughly 2^{63} evaluations—about 6,500 CPU-years plus 100 GPU-years, real but affordable for a well-resourced organization (Stevens et al., 2017). Git, which had bet its object model on SHA-1, began a still-ongoing migration to SHA-256.

Two lessons, both load-bearing for this paper. First, *the choice of algorithm matters enormously*: MD5, SHA-1, and SHA-256 look like siblings—the same iterated-compression skeleton, the same diet of 32-bit additions, rotations, and Boolean operations—yet differential cryptanalysis (anatomized in section 7) shattered the first two while SHA-256, attacked by the same schools with the same tools for twenty years, has not visibly moved (current state of the art in sections 6 and 7.4). Whatever separates broken from unbroken here is not the general species of design; it lives in the fine structure of round counts, rotation offsets, and message scheduling—quantitative differences producing a qualitative divide, and nobody can currently compute, from the specification alone, which side of the divide a design falls on. Second, breaks are not black swans; they are the *normal* fate of these objects, which makes SHA-256's quarter century of stability a phenomenon in want of explanation rather than a default to be assumed.

2.4 The epistemological situation

How do we know what we know about SHA-256? Not the way we know things about the Josephus function. There is no theorem asserting any of the properties in section 2.1; as we will see in section 6, for a fixed unkeyed function it is delicate even to *state* them. What exists instead is a corpus of observations: a published specification, a fossil record of broken relatives (section 2.3), a large literature of failed and partially successful attacks on weakened versions, and an industrial base exercising the function at cosmological rates without surfacing an anomaly.

This is the epistemic posture of a *naturalist*, not an axiomatist: we know SHA-256 the way nineteenth-century biology knew a species—by morphology, habitat, behavior under stress, and comparison with extinct relatives. The cryptographic literature has even formalized the ignorance: Rogaway's "human ignorance" framework (Rogaway, 2006) proves theorems of the form "any explicit method for breaking protocol P yields an explicit collision in SHA-256"—transferring trust not from an axiom but from the observed, decades-long failure of a very motivated species to produce such a collision. It is Popperian science embedded inside pure

mathematics: the conjecture "SHA-256 has no exploitable structure" is falsifiable, massively tested, and unproven.

The rest of this paper takes the naturalist's program seriously and asks what a *quantitative* natural history of this object would look like: first the anatomy (section 3), then the field measurements one can actually take, where work of Flajolet supplies exactly the right instrument (section 4), then the existing theoretical frameworks and the genuine gap between them (section 6).

3 Anatomy: a self-map of a finite space

We now open the organism. The dissection has two stages, and only the second is specific to SHA-256: first a completely general—and genuinely theoretical—reduction of "hash anything" to "iterate one map on a finite space"; then the finite space itself and the map that SHA-256 iterates on it.

3.1 From arbitrary bit strings to one finite map

A function on $\bigcup_{\ell<2^{64}}\{0,1\}^\ell$ looks unwieldy—its domain is (for practical purposes) infinite and ragged. The first structural fact about SHA-256 is that all of this unwieldiness is handled by a single, provably safe reduction, the *Merkle–Damgård construction*, and that everything interesting then happens on a finite set.

Padding. A message m of bit length ℓ (any ℓ, including 0; bits, not bytes) is padded by appending a single 1 bit, then the minimum number of 0 bits, then the number ℓ written as a 64-bit integer, so that the total length is a multiple of 512. The padded message splits into 512-bit blocks $m_1, \dots, m_k$ (fig. 2). Padding is where "arbitrary bit length" lives: the map

$$\mathrm{pad} : \bigcup_{\ell<2^{64}} \{0,1\}^\ell \longrightarrow \left(\{0,1\}^{512}\right)^+$$

is injective (the appended 1 and the length field between them guarantee it), and encoding the *length* in the final block—so-called Merkle–Damgård strengthening—is not bookkeeping but the hypothesis that makes theorem 3.1 below true.

Chaining. All real work is delegated to one *compression function*

$$f : \mathcal{S} \times \{0,1\}^{512} \longrightarrow \mathcal{S}, \qquad \mathcal{S} = (\mathbb{Z}/2^{32}\mathbb{Z})^8,$$

which eats a 512-bit block and updates a point of the finite space $\mathcal{S}$ (about which everything in section 3.2). Fix a public starting point $h_0 = \mathrm{IV} \in \mathcal{S}$ and set

$$h_i = f(h_{i-1}, m_i), \qquad \text{SHA-256}(m) = h_k,$$

reading the final state out as 256 bits (fig. 3).

This is not an ad hoc recipe; it is a construction with a theorem, proved independently in 1989 by Damgård—stated and proved as Theorem 3.1 of Damgård (1990, pp. 419–420)—and by Merkle, under the name "meta-method," in Merkle (1990b). Both original papers are short, readable, and archived alongside this note; the theorem is exactly why the design has been reused by MD5, SHA-1, and SHA-2 alike:

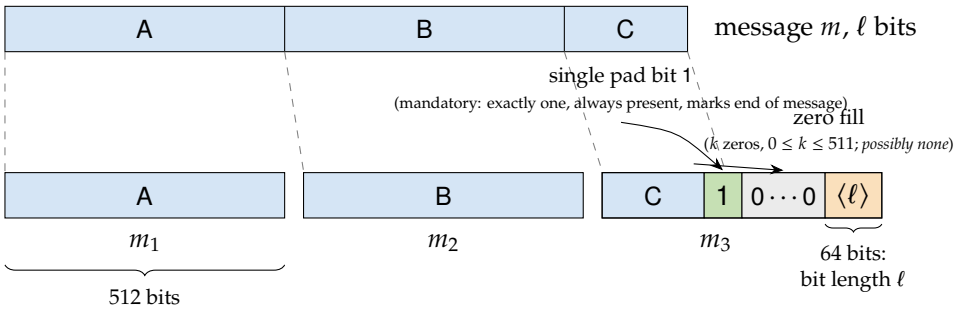


Figure 2: Padding and blocking, for a message needing $k = 3$ blocks. The linear message (blue; portions A, B, C) is consumed left to right by 512-bit blocks. The final block is completed by a single 1 bit (green), a run of 0s, and the 64-bit binary encoding of the message length ℓ (orange)—the Merkle–Damgård strengthening. The rule is exact and admits no options: to a message of ℓ bits one *always* appends one 1, then the fewest 0s (possibly none) so that the total reaches a multiple of 512 once the 64-bit length is affixed—i.e. k is the least ≥ 0 solution of $\ell + 1 + k + 64 \equiv 0 \pmod{512}$. The 1 bit and the length field are never omitted; only the zero-count k depends on ℓ, and if the tail C leaves no room for the 65 closing bits, k simply runs on into an extra all-padding block.

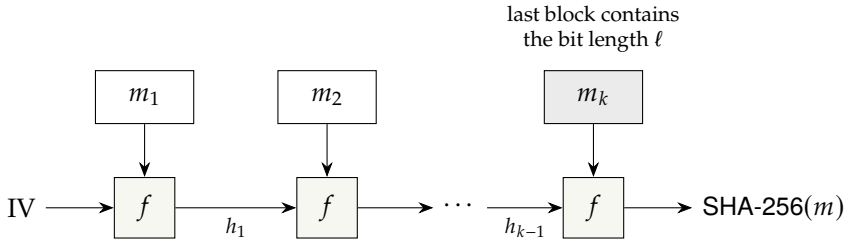


Figure 3: The Merkle–Damgård construction: an arbitrary-length message is folded through one compression function $f : \mathcal{S} \times \{0,1\}^{512} \to \mathcal{S}$ on the finite state space $\mathcal{S} = (\mathbb{Z}/2^{32}\mathbb{Z})^8$.

Theorem 3.1 (Merkle, Damgård)**.** *Any collision of the iterated hash yields, constructively, a collision of the compression function: from $m \neq m'$ with SHA-256(m) = SHA-256(m') one can extract $(h, b) \neq (h', b')$ with $f(h, b) = f(h', b')$.*

Proof idea. Walk the two chains backward from the equal outputs. If the messages have different lengths, the final blocks already differ (they encode the lengths—this is where strengthening is used) yet map to the same output: a collision in f. If the lengths agree, compare positions from the end; at the last position where the inputs to f differ, the outputs agree, again a collision in f. □

The moral: *the arbitrary-length-input problem reduces, with proof, to the study of one map on one finite space.* (The domain is itself finite—every allowed input is shorter than 2^{64} bits—but its inputs run through unboundedly many *lengths*, and it is that length-variability, not any actual infinity, that the construction tames.) The reduction is airtight only for collision resistance—the construction leaks other structure, such as the famous length-extension property (from the digest $H(m)$ alone, without knowing m, one can compute $H(m\|\text{pad}\|m')$; defined and dissected

in section 6.2), whose repair spawned its own theory (indifferentiability) and whose avoidance shaped the post-2000 designs, including the sponge construction underlying SHA-3 (Bertoni et al., 2007). But it means the rest of this paper may honestly focus on the finite object f.

3.2 The state space and its two rival geometries

The state space is

$$\mathcal{S} \;=\; (\mathbb{Z}/2^{32}\mathbb{Z})^8, \qquad |\mathcal{S}| = 2^{256}\,:$$

an 8-dimensional module-like array of 32-bit words—the "8×32" geometry in which the entire life of SHA-256 unfolds. As a plain set, $\mathcal{S}$ is also $\mathbb{F}_2^{256}$, a 256-dimensional vector space over the two-element field. Everything hinges on the fact that *the same underlying set carries two incompatible abelian group structures*, both used incessantly:

- bitwise addition $\oplus$ (exclusive or): the $\mathbb{F}_2$-vector space structure of $\mathbb{F}_2^{256}$;
- wordwise addition $\boxplus$ modulo 2^{32}: the product group structure of $(\mathbb{Z}/2^{32}\mathbb{Z})^8$, where carries propagate within each word.

These two structures share the zero element and almost nothing else. A map that is linear for $\oplus$ (a matrix over $\mathbb{F}_2$) is, generically, maximally nonlinear from the point of view of $\boxplus$, and vice versa; the carry chains of integer addition are precisely the terms that $\mathbb{F}_2$-linear algebra cannot see.

It helps to picture the additive ($\boxplus$) structure literally, as a clock (fig. 4). Read the state as an odometer counting modulo 2^{256}: every modular addition advances the needle by some amount, and the crucial event is whether it *wraps* past 0—an overflow, a carry out of the top, thrown away by the reduction. That one bit of "did it go around?" is exactly what $\oplus$-algebra is blind to and what the 2-adic metric, next, is built to record.

There is a third, more analytic way to say this, and it is the one we expect resonates with a p-adic sensibility. Identify $\mathbb{Z}/2^{32}\mathbb{Z}$ with the quotient $\mathbb{Z}_2/2^{32}\mathbb{Z}_2$ of the 2-adic integers. Then the basic machine operations sort themselves by how they treat the 2-adic metric (in which two integers are close when they agree in many low-order bits):

operation	$\mathbb{F}_2$-linear?	2-adically 1-Lipschitz?	where it is "tame"
$x \oplus c$, bitwise $\wedge, \vee, \neg$	yes ($\oplus$) / no	yes	both lenses see it
$x \boxplus c$, $x \boxplus y$	no	yes	only the 2-adic lens
rotation $x \lll r$, shift $x \gg s$	yes	no	only the $\mathbb{F}_2$ lens

Bitwise logic and modular addition are 1-*Lipschitz* (2-adically continuous in the strongest sense: output bit i depends only on input bits $0, \ldots, i$, the triangular or "causal" maps); compositions of these are the *T-functions* of Klimov and Shamir (Klimov and Shamir, 2003), and there is a genuine ergodic theory, due to Anashin, characterizing when such maps act measure-preservingly or ergodically on $\mathbb{Z}_2$ (Anashin, 2006). Rotations, by contrast, are $\mathbb{F}_2$-linear but violently discontinuous 2-adically: they let high-order bits influence low-order ones.

The 2-adic lens is not an ad hoc bookkeeping device; it opens onto a genuine subject. Nonarchimedean dynamics—the iteration theory of self-maps of p-adic disks, opened up by Lubin (1994)—asks exactly this paper's kind of question one level of structure up: when must the iteration of a p-adic map secretly respect algebraic structure? Lubin's paper records

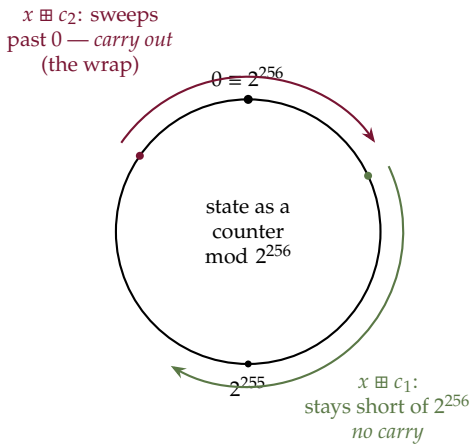


Figure 4: Wordwise addition $\boxplus$ as motion on a clock. As an odometer counting mod 2^{256}, the state advances by each addition; it *wraps* past 0 exactly when the sum overflows—a carry out of the top bit, discarded by the reduction. Whether a given addition wraps is invisible to $\mathbb{F}_2$-linear ($\oplus$) algebra and is precisely the information the 2-adic metric records. (The true state is not one circle of 2^{256} positions but eight *coupled* clocks of 2^{32} each, $(\mathbb{Z}/2^{32}\mathbb{Z})^8$: the single odometer is the intuition, the product of eight is the reality.) A round is a great many such additions, and iterating SHA-256 (section 5) sends the counter wandering—sometimes around the circle, sometimes not.

a suspicion of exactly the Josephus flavor—that a noninvertible series commuting with an invertible nontorsion one must have a formal group hidden behind it—and its main result makes a sharp instance of that suspicion into a theorem (Lubin, 1994, Main Theorem 6.3):

Theorem 3.2 (Lubin)**.** *Let o be the ring of integers in a finite extension of $\mathbb{Q}_p$, with maximal ideal $\mathfrak{m}$, and let $u, f \in o[[x]]$ be power series fixing 0, with u* invertible *(its linear coefficient $u'(0)$ a unit) and f* noninvertible *($f'(0) \in \mathfrak{m}$) of finite Weierstrass degree δ—the number of zeros f has in the open disk, counted with multiplicity. If u and f commute, $u \circ f = f \circ u$, then* either *some iterate of u is the identity series (u is a root of unity in the group of invertible series)* or *$\delta = p^d$ for some $d \geq 1$.*

The content is rigidity. The only Weierstrass degrees a noninvertible map may carry while sharing its orbits with a genuine (nontorsion) invertible symmetry are the powers of p—and those are precisely the degrees of the endomorphisms of a one-dimensional formal group. So bare commutation, a purely dynamical hypothesis with no algebra in it, already forces the numerical signature of a formal-group skeleton; whether the skeleton can ever be truly absent is Lubin's own still-open question.

The phenomenon is visible in miniature over our own ring $\mathbb{Z}_2$ ($p = 2$). The noninvertible $f(x) = 4x + x^2$—linear coefficient $4 \in 2\mathbb{Z}_2$, Weierstrass degree $2 = 2^1$, a p-power on the nose—commutes with the invertible $u(x) = 9x + 6x^2 + x^3$ (unit linear coefficient 9), as one checks by expanding $u \circ f = f \circ u$ directly. The companion Corollary 3.2.1 of the same paper is just as concrete: every zero of f' inside the disk must be a zero of some iterate of f. Here $f'(x) = 2x + 4$ vanishes at $x = -2$, and indeed $f(f(-2)) = f(-4) = 0$. Two lines of arithmetic thus exhibit a symmetry (u), a forced degree (2^1), and a forced coincidence among the iterates—structure precipitating out of commutation alone, the Josephus moral in p-adic dress.

How near does this reach SHA-256? Adjacent, honestly, not overhead. Theorem 3.2 governs *analytic* series on the p-adic disk, whereas SHA-256's ARX pieces are merely 1-Lipschitz T-functions—the coarse, non-analytic relatives tabulated above—so the theorem does not bind the function on the nose, and Anashin's ergodic theory is what survives at that lower regularity (Anashin, 2006; Klimov and Shamir, 2003). What *does* transfer is the question and its shape: does any nontrivial map commute with a (reduced, word-narrowed) SHA-256 component, and if one did, would it drag in hidden algebraic structure the way it must for analytic series? At toy scale that is a finite, machine-checkable hunt—search for a commuting symmetry; find none, and the absence is itself a measurement of structurelessness. In this language SHA-256's designers are in the business of manufacturing 1-Lipschitz dynamics that commute with nothing and hide no formal group, and theorem 3.2 is the evidence that, one regularity class up, such structurelessness is genuinely hard to achieve by accident.

SHA-256's round function is an *ARX* design—Addition, Rotation, Xor—and the table explains the acronym's cunning: each primitive is transparent in one geometry and wild in the other, and the design alternates them so that the composition is wild in both. An analyst who adopts $\mathbb{F}_2$-linear algebra kills the X's and R's but is left helpless against the carries; an analyst who adopts the 2-adic/T-function lens domesticates additions and logic but is destroyed by the rotations. This is not an accident of SHA-256; it is the design principle of the whole ARX family. What *is* specific to SHA-256 is the exact schedule of alternation, and section 2.3 showed how much such specifics matter.

Project 1: Single operations under the 2-adic lens

Anashin's theory gives clean criteria for when a T-function like $x \mapsto x \boxplus c$ or $x \mapsto x \oplus c \boxplus (x \wedge d)$ is a bijection of $\mathbb{Z}/2^{32}\mathbb{Z}[k]$ for all k, and when its iteration visits every residue (ergodicity) (Anashin, 2006; Klimov and Shamir, 2003). Verify the criteria experimentally on 8- and 16-bit words, visualize orbits, then break the hypotheses with a single rotation and measure what dies. A perfect first encounter with p-adic ideas producing visible, computable phenomena.

Prerequisites: elementary number theory; curiosity about p-adic numbers *Window:* 3–4 weeks

3.3 The compression function, structurally

We describe f at the level of shape; the full specification fits on two pages of National Institute of Standards and Technology (2015) (with an accessible pseudocode rendering in Wikipedia contributors, 2026) and is worth reading only after the shape is clear. The state is eight words (a, b, c, d, e, f, g, h). The 512-bit message block is first *expanded*: its sixteen words $W_0, \ldots, W_{15}$ are stretched to sixty-four by the recurrence

$$W_t \;=\; \sigma_1(W_{t-2}) \boxplus W_{t-7} \boxplus \sigma_0(W_{t-15}) \boxplus W_{t-16}, \qquad 16 \le t < 64,$$

where σ_0, σ_1 are fixed $\oplus$-combinations of rotations and shifts of a single word ($\mathbb{F}_2$-linear), while the additions are modular (2-adic). Then sixty-four rounds are applied; round t computes

$$T_1 = h \boxplus \Sigma_1(e) \boxplus \mathrm{Ch}(e, f, g) \boxplus K_t \boxplus W_t, \qquad T_2 = \Sigma_0(a) \boxplus \mathrm{Maj}(a, b, c),$$

and shifts the state pipeline: $(a, \ldots, h) \leftarrow (T_1 \boxplus T_2,\ a,\ b,\ c,\ d \boxplus T_1,\ e,\ f,\ g)$. Here Σ_0, Σ_1 are again xors of three rotations, and

$$\mathrm{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z), \qquad \mathrm{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z)$$

are bitwise Boolean functions of degree two ("choose" and "majority"). Finally—a crucial asymmetry called the Davies–Meyer feedforward—the input state is added back: the block's net effect is $h \mapsto h \boxplus \tilde{f}(h, m)$, which prevents running the rounds backward even though each individual round is invertible. This feedforward is not a detail but the hinge on which one-wayness turns, and it is one of the two places where SHA-256's architecture carries a genuine theorem; we return to it in section 3.4.

Figure 5 traces how a single block's data reaches every one of the sixty-four rounds. The block supplies only the first sixteen schedule words; the recurrence then *stretches* those seeds forward, each new word reaching back to four earlier ones, until all sixty-four exist—so a bit set in the incoming block propagates, through the schedule alone, into arbitrarily late rounds. Each round t then draws exactly two external inputs, its schedule word W_t and its constant K_t, and folds them into the running eight-word state; everything else in the round is internal feedback.

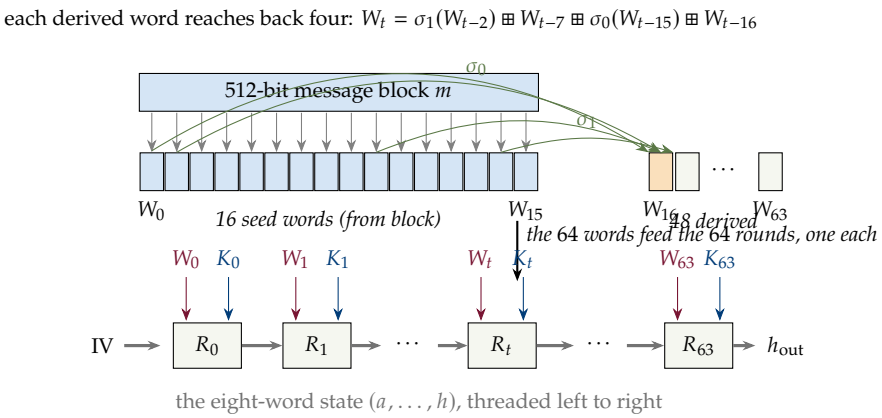


Figure 5: How one block's data reaches every round. The block fills only the sixteen seed words $W_0, \ldots, W_{15}$ (blue); the message-schedule recurrence then stretches them into $W_{16}, \ldots, W_{63}$ (each derived word, shown for W_{16}, built from four earlier ones via the $\mathbb{F}_2$-linear maps σ_0, σ_1 and modular addition). Each of the 64 rounds R_t consumes exactly two external inputs—its schedule word W_t (wine) and its constant K_t (navy)—and threads the eight-word state through from the incoming chaining value to h_{out}. Arrows show influence: a bit of the block, via the schedule's reach-back, can steer arbitrarily late rounds.

The reach of that schedule is worth making quantitative, because it is a clean computed pattern rather than a hand-drawn cartoon. Trace, for each word W_t, the set of *seed* words $W_0, \ldots, W_{15}$—the only ones carrying message data—that it transitively depends on, following the recurrence's four back-pointers ($t-2$, $t-7$, $t-15$, $t-16$) down to the data layer. The dependency graph (fig. 6) then answers "how fast does a block's data spread through the schedule?" exactly: the count of seeds reached climbs $4, 4, 7, 7, 10, 10, 13, 14, 15, 15, \ldots$ and *saturates at* W_{26}—from that word on, every schedule word depends on all sixteen data words.

Ten steps of the recurrence suffice to fully entangle the message, before a single one of the sixty-four rounds has done its diffusion work. The four coloured arc families in the figure are the four terms of the recurrence, and their regular weave is the schedule's linear-feedback skeleton laid bare; whether that visible regularity is a foothold or a red herring is one more question small experiments can chew on.

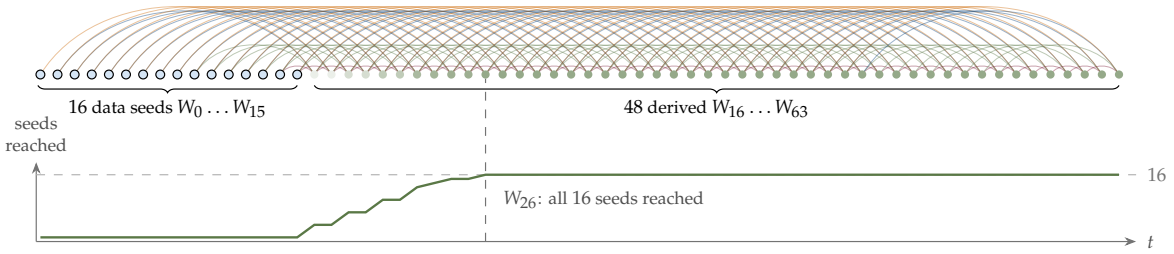


Figure 6: The message schedule as a computed dependency graph (generated by `tools/wschedule_graph.py`, which also emits a Graphviz `.dot` version). Nodes are the 64 schedule words in index order; the sixteen data-carrying seeds $W_0, \dots, W_{15}$ are blue, and derived words darken with the number of seeds they transitively depend on. Each of the four arc colours is one term of $W_t = \sigma_1(W_{t-2}) \boxplus W_{t-7} \boxplus \sigma_0(W_{t-15}) \boxplus W_{t-16}$. The lower trace counts seeds reached versus t; it reaches all sixteen at W_{26} (dashed line), after which the message is fully diffused through the schedule alone.

One further structural reduction deserves emphasis, because it shrinks the apparent dimensionality of the mystery. Only two of the eight registers ever receive fresh values: each round computes one new word entering the a-position and one entering the e-position, while the other six merely shift along. The eight-word state is a *delay line*—a memoization of recent history, not eight independent degrees of freedom. Writing A_t and E_t for the words produced at round t (so that the state at that moment reads $(A_t, A_{t-1}, A_{t-2}, A_{t-3}, E_t, E_{t-1}, E_{t-2}, E_{t-3})$), the sixty-four rounds are exactly the coupled two-track recurrence

$$T_t = E_{t-4} \boxplus \Sigma_1(E_{t-1}) \boxplus \mathrm{Ch}(E_{t-1}, E_{t-2}, E_{t-3}) \boxplus K_t \boxplus W_t,$$
$$A_t = T_t \boxplus \Sigma_0(A_{t-1}) \boxplus \mathrm{Maj}(A_{t-1}, A_{t-2}, A_{t-3}), \qquad E_t = A_{t-4} \boxplus T_t,$$

with the eight initial values $A_{-1}, \dots, A_{-4}, E_{-1}, \dots, E_{-4}$ read off the incoming chaining state. So the "8-dimensional" iteration is really a pair of scalar recurrences over $\mathbb{Z}/2^{32}\mathbb{Z}$ with lookback four—formally a nonlinear cousin of the lagged-Fibonacci and linear-feedback recurrences whose theory is complete (section 6.6)—and the cryptanalytic literature does in fact work in exactly this A/E form. Nothing is lost in the reduction; the honest shape of a round is two new words under an eight-word window of memory.

Notice what is *absent*: no multiplications, no S-box tables, no number-theoretic structure—only the three ARX primitives and two quadratic Boolean functions, iterated 64 times. The function is degree-two nonlinearity compounded through 64 rounds of geometric whiplash between $\mathbb{F}_2^{256}$ and $(\mathbb{Z}/2^{32}\mathbb{Z})^8$. That so spare a recipe apparently produces a completely structureless map is the central mystery this paper wants to sell.

3.4 Two constructions that come with proofs

It is worth pausing to locate the mystery precisely, because SHA-256 is built in three nested layers and—remarkably—the two *outer* layers each carry a solid theorem. Only the innermost object is theorem-free. Naming the layers from the outside in: the *mode* (Merkle–Damgård iteration, section 3.1) wraps the *compression function* f; the compression function is itself built by the *Davies–Meyer* construction around a block cipher E; and E—the cipher SHACAL-2, which is just the round machinery above run as a keyed permutation—is the finite map about which nothing is proved. The provable security of SHA-256 is entirely a statement about the two wrappers, conditional on the core behaving well.

The mode: Merkle–Damgård. The outer layer's theorem is already in hand: theorem 3.1 says that a collision in the full hash constructively yields a collision in f, so *collision resistance of f is inherited by the whole function*—unconditionally, in the standard model, needing only that the final block encode the message length (the "strengthening" the proof consumes). This is as solid as theorems in this subject get: no idealization, no unproven assumption, a one-page reduction. What it does *not* give is equally important and sharply delimited—it transfers collision resistance and nothing else, which is exactly why length extension and the multicollision family of section 6.3 slip through. The mode is a theorem about one property, and an honest one.

The compression function: Davies–Meyer. The middle layer is the feedforward met in section 3.3. Take a block cipher $E_m(h)$—plaintext the chaining value $h \in \mathcal{S}$, key the message block m—and set

$$f(h, m) \;=\; E_m(h) \boxplus h.$$

The feedforward "$\boxplus h$" is the whole point. Without it, $f = E_m$ would be a *permutation* of $\mathcal{S}$ for each fixed m, hence trivially invertible (decrypt with key m) and useless as a one-way compression—the same observation that, read dynamically, section 5.3 shows is what makes iterated SHA-256 lose entropy at all. Adding the input back breaks invertibility, and does so in a way one can prove is optimal, provided the cipher is idealized:

Theorem 3.3 (Preneel–Govaerts–Vandewalle; Black–Rogaway–Shrimpton)**.** *Model E as an* ideal cipher *(for each key an independent, uniformly random permutation, accessible only as a black box in both directions). Then the Davies–Meyer compression $f(h, m) = E_m(h) \boxplus h$ is optimally collision- and preimage-resistant: any adversary asking q cipher queries finds a collision with probability at most $q(q+1)/2^n$, and a preimage with probability of order $q/2^n$—matching the birthday bound $2^{n/2}$ for collisions and the brute-force bound 2^n for preimages, and both are tight.*

Preneel et al. (1994) isolated this by classifying all 64 natural ways to build a rate-one compression function from a block cipher and singling out the 12 sound ones (Davies–Meyer among them); Black et al. (2002) then supplied the rigorous black-box proofs and the exact bounds quoted above. So the middle layer, too, has a theorem—but note the price on the ticket. It holds in the *ideal cipher model*, the block-cipher cousin of the random-oracle model of section 6.2, and is therefore a heuristic about SHA-256, not a proof about it: SHACAL-2 is a fixed, public, emphatically non-ideal cipher whose "key schedule" is literally the message expansion of section 3.3, so the assumption that distinct message blocks act like independent

random permutations is exactly the leap of faith the model licenses and cannot justify. A concrete reminder that the idealization is lossy: Davies–Meyer has *trivially computable fixed points*—solving $E_m(h) \boxplus h = h$ means $E_m(h) = 0$, i.e. $h = E_m^{-1}(0)$, one short computation per block—a harmless-looking feature that became a lever for the long-message second-preimage attacks of section 6.3.

The moral, localized. Two layers, two theorems: the mode proved unconditionally, the compression proved in an idealized model, and between them they reduce the security of the whole edifice to a single demand—that the core map f (equivalently the cipher E) have no exploitable structure. That demand is section 6's gap, and it is where every remaining page of this paper lives. The architecture is solved mathematics; the mystery has been squeezed into one finite object, and the constructions of this section are precisely the machinery that did the squeezing.

3.5 One honest theorem: invertible diffusion

Amid so much that is unproved, it is worth exhibiting in full one of the few genuine theorems about SHA-256's internals—partly because it is lovely, partly because its proof lives in exactly the finite geometry of section 3.2, and partly because it shows the "middle territory" of section 6.7 is not empty by necessity.

Ask a basic anatomical question about the stirring maps of section 3.3: are $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$ *bijections* of the word space $\mathbb{F}_2^{32}$? (They are $\mathbb{F}_2$-linear, hence "nonlinearities" only through the other lens of section 3.2—as maps of $\mathbb{Z}/2^{32}\mathbb{Z}$ they are wildly nonlinear. The question is whether they discard information.) Rivest answered this for the pure-rotation case in a four-page note (Rivest, 2011):

Theorem 3.4 (Rivest)**.** *Let w be a power of two. The exclusive-or of t distinct rotations of a w-bit word, $x \mapsto \mathrm{ROTR}^{r_1}(x) \oplus \cdots \oplus \mathrm{ROTR}^{r_t}(x)$, is invertible if and only if t is odd.*

Proof idea. Identify $\mathbb{F}_2^{32}$ with the quotient ring $R = \mathbb{F}_2[x]/(x^{32} - 1)$, a bit vector becoming a polynomial and rotation becoming multiplication by x^r. The map in question is multiplication by $p(x) = x^{r_1} + \cdots + x^{r_t}$, invertible in R exactly when $\gcd\big(p(x),\, x^{32} - 1\big) = 1$. Over $\mathbb{F}_2$ the modulus collapses—$x^{32} - 1 = (x - 1)^{32}$, because 32 is a power of two and squaring is a ring homomorphism in characteristic 2—so the gcd is 1 precisely when $(x - 1) \nmid p$, i.e. when $p(1) = t \bmod 2$ is 1. □

So $\Sigma_0 = \mathrm{ROTR}^2 \oplus \mathrm{ROTR}^{13} \oplus \mathrm{ROTR}^{22}$ and Σ_1, with their three rotations each, are bijections: they stir without spilling. The theorem repays contemplation. It is two lines of algebra, yet it turns on the arithmetic accident that the word length is a power of two, which makes the cyclic algebra R *local* (one maximal ideal, generated by $x - 1$): the only way an xor-of-rotations can fail to be invertible is by having an even number of terms. Had the designers used 33-bit words, $x^{33} - 1$ would factor richly and the parity rule would be false. Small finite geometries have loud opinions.

Because the proof is constructive, invertibility need not be left as an existence statement: the inverses can be exhibited outright. Carrying out the division in R—equivalently, inverting the corresponding 32×32 circulant matrices over $\mathbb{F}_2$; section A gives a complete reference

implementation, which also confirms the results on thousands of random words—produces closed forms that are again xors of rotations:

$$\begin{aligned} \Sigma_0^{-1} &= \bigoplus_{r\in S_0} \mathrm{ROTR}^r, \quad S_0 = \{0,1,2,5,7,8,9,10,12,16,17,19,22,23,25,29,30\}, \\ \Sigma_1^{-1} &= \bigoplus_{r\in S_1} \mathrm{ROTR}^r, \quad S_1 = \{2,3,5,7,9,11,12,13,18,21,22,23,24,26,27,29,30\}, \end{aligned} \tag{2}$$

each an xor of *seventeen* rotations (S_0 includes $r = 0$, the identity rotation). Seventeen is odd, as theorem 3.4 demands of any invertible xor-of-rotations—and as it must be on general grounds: evaluating $p\,q \equiv 1$ at $x = 1$ forces $|p| \cdot |q| \equiv 1 \pmod 2$, so odd-weight elements have odd-weight inverses. The asymmetry of sparseness is the striking part: three rotations to stir, seventeen to unstir—more than half of the thirty-two possible offsets. Invertible is not the same as symmetrically cheap, even inside the linear layer.

The ring picture yields more than the inverses; it computes the whole multiplicative life of these maps, essentially by hand. Composition of rotations adds offsets modulo 32, so write an xor-of-rotations as a polynomial with the *offsets as exponents*: Σ_0 becomes $p_0 = x^2 + x^{13} + x^{22}$. Squaring is the Frobenius map of characteristic 2—it simply doubles every offset—so squaring p_0 three times gives, after reducing exponents modulo 32, $\Sigma_0^8 = \mathrm{ROTR}^8$: *eight applications of the three-rotation stir collapse to one bare rotation*. Hence $\Sigma_0^{32} = \mathrm{id}$, and the order of Σ_0 in $\mathrm{GL}_{32}(\mathbb{F}_2)$ is exactly 32; the same computation gives $\Sigma_1^{16} = \mathrm{id}$ with order exactly 16. In particular $\Sigma_0^{-1} = \Sigma_0^{31}$ and $\Sigma_1^{-1} = \Sigma_1^{15}$, so eq. (2) could have been generated, offsets and all, by repeated squaring. One more turn of the crank and the observation becomes a theorem: for *any* odd xor-of-rotations u of 32-bit words, $u^{32} = u(x)^{32} = u(x^{32}) = u(1) = 1$, so every invertible xor-of-rotations has order dividing 32. The stirring maps of SHA-256, for all their visual violence, are elements of small 2-power order in a 2-group—one more exact structural fact living quietly next door to the conjectured structurelessness (section 3.7).

Note what the theorem does *not* cover: $\sigma_0 = \mathrm{ROTR}^7 \oplus \mathrm{ROTR}^{18} \oplus \mathrm{SHR}^3$ and σ_1 each replace one rotation by a bit-discarding shift, leaving the polynomial ring for plain linear algebra; and the genuinely non-invertible ingredients of SHA-256 are elsewhere entirely—the bitwise-quadratic Ch and Maj and the Davies–Meyer feedforward, where all the designed information loss is concentrated. The picture that emerges is architectural: *lossless* stirring layers (provably so, by theorem 3.4), *lossy* mixing concentrated in three auditable places. That so clean a structural statement was worth a research note in 2011—twenty years into the design lineage—says something about how sparsely theorized this territory is.

Project 2: The linear algebra of diffusion

Write each of $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$ as an explicit 32×32 matrix over $\mathbb{F}_2$ and decide invertibility of the σ's (settling what theorem 3.4 leaves open). Compute the order of each invertible map in $\mathrm{GL}_{32}(\mathbb{F}_2)$, the cycle structure of its action on $\mathbb{F}_2^{32}$, and the subgroup the four maps generate. Then reprove theorem 3.4 for general word length w: the answer is a pretty gcd condition (worked out in Rivest's note), and comparing $w = 32$ with $w = 33$ makes the power-of-two accident vivid.

Prerequisites: linear algebra; programming *Window:* 3–4 weeks

Project 3: Deriving the exact inverses of Σ_0 and Σ_1

Theorem 3.4 guarantees that Σ_0 and Σ_1 are bijections, and eq. (2) displays their inverses; this project derives that display from scratch, by two independent methods, and decides what eq. (2) deliberately leaves out. The steps are fully determined:

1. **Move to the ring.** Identify a word with a polynomial in $R = \mathbb{F}_2[x]/(x^{32} - 1)$, bit i becoming the coefficient of x^i, so that a rotation is multiplication by a power of x. Write Σ_0 as multiplication by $p_0(x) = x^2 + x^{13} + x^{22}$ and Σ_1 by $p_1(x) = x^6 + x^{11} + x^{25}$ (the SHA-256 rotation offsets).

2. **Invert by linear algebra.** Multiplication by p is an $\mathbb{F}_2$-linear circulant map; build its 32×32 matrix (columns are $p, xp, x^2p, \dots$) and invert it over $\mathbb{F}_2$ by Gaussian elimination. The last column of the inverse, read as a polynomial q, satisfies $p\,q \equiv 1 \pmod{x^{32} - 1}$.

3. **Read off the answer.** You should recover exactly the seventeen-term rotation sets of eq. (2). Verify $p\,q = 1$ in R directly, and check your derivation against the reference implementation of section A. Then reproduce eq. (2) a third way, with no linear algebra at all: by repeated squaring, using $\Sigma_0^{-1} = \Sigma_0^{31}$ and $\Sigma_1^{-1} = \Sigma_1^{15}$.

4. **Cross-check by Euclid.** Recompute q by the extended Euclidean algorithm on $p(x)$ and $x^{32} - 1 = (x - 1)^{32}$ in $\mathbb{F}_2[x]$; the two methods must agree.

5. **Contrast with the σ's.** Repeat for $\sigma_0 = \mathrm{ROTR}^7 \oplus \mathrm{ROTR}^{18} \oplus \mathrm{SHR}^3$: the shift SHR^3 leaves the cyclic ring (it is not a unit multiple of x), so the polynomial trick fails and only the matrix method applies. Decide invertibility of σ_0, σ_1 this way—settling what theorem 3.4 does not cover.

Deliverable: the two explicit inverse maps, a proof they are correct, and a one-paragraph account of why the σ's need the heavier tool.

Prerequisites: polynomial arithmetic over $\mathbb{F}_2$; programming *Window:* 2–3 weeks

3.6 Nothing up my sleeve

One question a mathematician should ask of section 3.3: where do the constants come from? The initial state IV and the sixty-four round constants K_t could, in principle, hide a trapdoor—constants engineered so their designer knows structure nobody else can see. The history is not hypothetical: the unexplained S-box constants of the 1970s DES cipher fed decades of suspicion (later resolved in the designers' favor), and post-Snowden, an actual back-doored NIST standard (the Dual_EC random generator) was withdrawn. The SHA family itself supplies the sharpest demonstration that its designer's choices are informed rather than arbitrary: the NSA's unexplained one-bit revision of SHA-0 into SHA-1 (see the footnote in section 2) anticipated by four years the differential attack later found in the open literature (Chabaud and Joux, 1998). The SHA-2 designers therefore committed to constants they visibly could not have chosen:

$$\mathrm{IV}_j = \text{first 32 fractional bits of } \sqrt{p_j}, \qquad K_t = \text{first 32 fractional bits of } \sqrt[3]{p_t},$$

where $p_1 < p_2 < \cdots$ are the primes $2, 3, 5, \ldots$ (National Institute of Standards and Technology, 2015). (So IV_1 = `6a09e667` encodes $\sqrt{2} - 1$ in binary.) Such choices are called *nothing-up-my-sleeve* constants: points of the state space selected by a rule so canonical that no freedom remains in which to hide intent.

There is a pleasing irony here that deserves a mathematician's smile: the constants are trusted *because* they are digits of irrational numbers, yet essentially nothing is proved about those digits either—the normality of $\sqrt{2}$ is an open problem. The construction does not replace an unproved assumption with a proved one; it replaces an *unauditable* choice with an *auditable* one. Epistemology, not theorems, is doing the work—a theme by now familiar (section 2.4).

3.7 Reasons for hope: exact solutions next door

Almost everything after this section will be statistical: random-mapping silhouettes, test batteries, mixing arguments—the approximate machinery one reaches for when exact structure is unavailable. Before conceding that this is the only possible register, it is worth assembling the evidence that exact structure keeps turning up *next door*, because the history of discrete iteration is littered with problems that looked hopelessly procedural until someone found the right coordinates.

The Josephus process (section 1) is this paper's opening specimen: an $(n-1)$-step elimination dynamics collapsed to a single bit rotation. Theorem 3.4 is a second, living inside SHA-256 itself: a question about its diffusion maps settled by two lines of algebra in a local ring. The genre is much larger. Additive cellular automata—iterative, visually turbulent, every bit as "chaotic" in appearance as a hash round—were solved outright by Martin et al. (1984): represent configurations as polynomials over $\mathbb{F}_2$ and time evolution becomes multiplication, with global transient and cycle structure falling out as exact divisibility statements. Sutner (1989) dispatched the Lights-Out puzzle the same way (the σ-game as $\mathbb{F}_2$-linear algebra). The pattern is older than computing: the logistic map at parameter 4, the textbook "chaotic" system, has the closed form $x_t = \sin^2(2^t \arcsin\sqrt{x_0}\,)$—an observation going back to Ulam and von Neumann, and the simplest instance of Schröder's program of solving iteration by conjugation (Schröder, 1871): find ψ making $\psi \circ f \circ \psi^{-1}$ linear, and the t-th iterate is computed in one stroke. That program grew into modern complex dynamics (Milnor, 2006). "Chaotic" and "exactly solvable" are not antonyms; they are frequently the same object described before and after the right change of variables.

Within the hash family the lesson recurs. SHA-1's message schedule is $\mathbb{F}_2$-linear, hence *exactly* analyzable—its difference propagation is linear algebra—and that handle is precisely what the collision attacks of section 2.3 gripped. The professional response to such objects has been the cryptanalyst's panoply, differential attacks in the tradition of Biham and Shamir (1991) and their descendants: a *statistical* theory of near-structure, probability-weighted trails threaded through the rounds (section 7 is devoted to it). The examples above argue for keeping a different question alive: whether some change of coordinates—a direct finite-geometry path, in the spirit of the polynomial representations that solved cellular automata, with no mixing theory, no ergodicity, and nothing approximate—renders some nontrivial slice of the iteration exactly computable. The designers' explicit goal was to foreclose this (section 3.2), and the smart money says they succeeded. But the foreclosure is a conjecture, not a theorem; the components keep falling one at a time to exact methods (theorem 3.4, the T-function theory (Klimov and Shamir, 2003; Anashin, 2006), the two-track reduction of section 3.3); and the hunting grounds

are tiny finite objects, where undergraduate computation is a genuine research instrument (section 9).

4 Random mappings: Flajolet's telescope

If the working conjecture is "SHA-256 behaves like a random function," then the right response is the scientist's: *a conjecture that mentions randomness is a bundle of quantitative predictions*. What does a random function actually look like? Remarkably, this question has an exact, beautiful answer, developed by Harris in the probabilistic era (Harris, 1960) and perfected by Flajolet and Odlyzko with generating-function methods (Flajolet and Odlyzko, 1990). The answer turns the vague conjecture into a table of measurable constants.

4.1 The functional graph of a random map

Let φ be a function from a finite set of size N to itself—for us, eventually, φ = (some restriction of) SHA-256, with $N = 2^n$. Draw the *functional graph*: a node for each point, an arrow $x \to \varphi(x)$. Because every node has out-degree exactly one, the shape is forced: each connected component consists of exactly one directed cycle, with trees hanging off the cycle nodes (fig. 7). Iterating φ from any seed x_0 walks a ρ-shaped path: a *tail* of λ new points, then a *cycle* of length μ repeated forever.

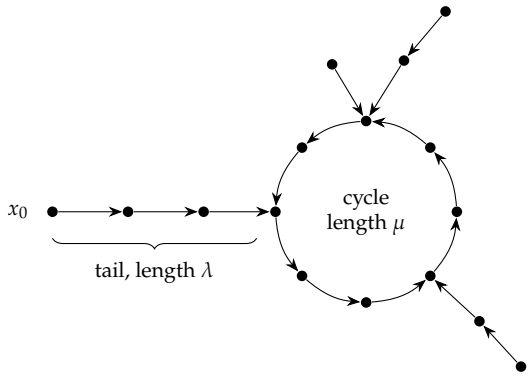


Figure 7: One component of a functional graph. Iteration from any seed traces a ρ: a tail of length λ into a cycle of length μ. Trees hang off the cycle; components with their unique cycles tile the whole space.

Now let φ be chosen *uniformly at random* among all N^N self-maps. Flajolet and Odlyzko computed the expected value of essentially every natural parameter of this picture; a sample, asymptotically as $N \to \infty$ (Flajolet and Odlyzko, 1990):

parameter	expectation	at $N = 2^{256}$
tail length λ (random seed)	$\sqrt{\pi N/8}$	$\approx 2^{127.3}$
cycle length μ (random seed)	$\sqrt{\pi N/8}$	$\approx 2^{127.3}$
rho length $\lambda + \mu$	$\sqrt{\pi N/2}$	$\approx 2^{128.3}$
number of cyclic points	$\sqrt{\pi N/2}$	$\approx 2^{128.3}$
number of components	$\frac{1}{2} \ln N$	≈ 89
image size $\lvert\varphi(\mathcal{S})\rvert$	$(1 - e^{-1})\, N$	$\approx 0.632\, N$
points with no preimage (leaves)	$e^{-1}\, N$	$\approx 0.368\, N$
largest component	$\approx 0.7582\, N$	—
largest tree	$\approx 0.48\, N$	—

Pause on the flavor of these results. A "typical" random map is nothing like a random permutation: it is lossy (a third of the space is never hit at all), it has one giant component swallowing three quarters of everything, its cycles are tiny compared to N (order $\sqrt{N}$), and the number of components grows only logarithmically. This is a sharp, peculiar, recognizable statistical silhouette—a fingerprint of structurelessness itself.

4.2 Where the constants come from

The route to that table is pure Flajolet, and worth a paragraph even in an overview, because it connects our discrete object to complex analysis. A tree rooted at a node whose arrow enters a cycle is a labelled rooted tree; its exponential generating function $T(z)$ satisfies the functional equation

$$T(z) = z\, e^{T(z)}$$

(a root, plus a set of subtrees). Components are cycles of trees, and mappings are sets of components, which by the symbolic calculus gives the mapping EGF

$$M(z) \;=\; \frac{1}{1 - T(z)}.$$

All the parameters in the table are obtained by marking features in these equations and then reading off coefficient asymptotics. The engine for the last step: T has a square-root singularity at $z = e^{-1}$, where $T(e^{-1}) = 1$—so M blows up there—and *singularity analysis* transfers the local behavior of a generating function at its dominant singularity directly into asymptotics of its coefficients. The half-integer exponents in the singular expansions are what produce all those $\sqrt{N}$'s; the method is developed at book length in Flajolet and Sedgewick (2009), with random mappings as a recurring worked example. For a mathematician, this is the pleasant surprise of the whole subject: questions about iterating a hash function route through the local behavior of an analytic function at a branch point.

4.3 The telescope, pointed at SHA-256

Here is the epistemological payoff. The random-function conjecture of section 2 predicts that if we build from SHA-256 a self-map of an N-point space—most simply

$$\varphi_n : \{0,1\}^n \to \{0,1\}^n, \qquad \varphi_n(x) = \text{first } n \text{ bits of } \mathsf{SHA\text{-}256}(x),$$

for moderate n—then the functional graph of φ_n, an object SHA-256's designers never contemplated and certainly never tuned, should match every line of the Flajolet–Odlyzko table, not merely in expectation but in distribution (the limiting distributions are also known). Each parameter is a falsifiable prediction; jointly they are a *coordinate system* in which the position of SHA-256 among pseudorandom objects can be measured rather than argued about. For contrast, structured maps land visibly elsewhere in these coordinates: $x \mapsto x \boxplus c$ decomposes into cycles only (no trees: it is a bijection), $x \mapsto x^2 \bmod p$ has its graph dictated by quadratic reciprocity, and a one-round toy SHA-256 should sit measurably off the random silhouette. Where exactly it rejoins the silhouette as rounds are added is, as far as we know, unmeasured—and eminently measurable (section 9).

At full scale ($n = 256$, $N = 2^{256}$) nobody will ever traverse a rho: the expected rho length is about 2^{128} steps. The table is then the *only* available description of the global geometry of the object the whole world is using—a telescope pointed at a structure we can predict in detail but never visit. And the predictions are not idle: they are the engineering basis of real algorithms. Pollard's rho method (Pollard, 1978) finds collisions and discrete logarithms precisely by betting that its iteration behaves like a random map (cycle detection in $O(\sqrt{N})$ steps with $O(1)$ memory); van Oorschot and Wiener's parallel collision search (van Oorschot and Wiener, 1999) turned that bet into an industrial discipline (distinguished points, linear parallel speedup), and the SHAttered attack's budget of 2^{63} (section 2.3) was costed with exactly this calculus. Every time such an algorithm's observed running time matches its predicted constant, a Flajolet statistic of a real hash function has been confirmed to one more decimal place.

So far the telescope has resolved a *single* application of the function. Because SHA-256 is so often composed with itself—in password stretching, hash chains, and Bitcoin's double hash—the dynamics of *iterating* φ form their own coordinate system, and section 5 takes it up next: how the image shrinks, where orbits settle, and the precise sense in which iteration almost, but not quite, induces a random permutation.

Project 4: A functional-graph atlas of truncated SHA-256

For $n = 16, 18, \ldots, 30$: exhaustively build the functional graph of φ_n, measure every parameter in the table above (and the distributions, not just means), and compare with the Flajolet–Odlyzko asymptotics *and* with genuinely random maps of the same size as a control. Deliverable: an "atlas" of plots—theory curve, random-map control cloud, SHA-256 data points. Then repeat with reduced-round SHA-256 ($1, 2, 4, \ldots, 64$ rounds) and watch the data walk onto the random silhouette as rounds accumulate. To our knowledge no published systematic atlas of this kind exists; undergraduates can own it.

Prerequisites: programming; a first probability course *Window:* 6–8 weeks (flagship)

Project 5: Collision hunting, the van Oorschot–Wiener way

Implement Pollard rho with Brent or Floyd cycle detection on φ_n for n up to ~48, then the van Oorschot–Wiener distinguished-points parallel method on a lab's worth of machines. Measure the distribution of time-to-collision against the theory of section 4.1. Students reproduce, at toy scale, the exact methodology of the SHAttered attack—and learn why

256 bits is chosen so that $\sqrt{N}$ is out of humanity's reach.
Prerequisites: programming; probability *Window:* 4–6 weeks

5 Iterating the function

Section 4 pointed Flajolet's telescope at a single application of SHA-256. But the function is very often applied to its own output—

$$\varphi(x) \;=\; \mathsf{SHA\text{-}256}(x), \qquad \varphi^{k} \;=\; \underbrace{\mathsf{SHA\text{-}256} \circ \mathsf{SHA\text{-}256} \circ \cdots \circ \mathsf{SHA\text{-}256}}_{k},$$

read as a self-map of the 256-bit strings (feed the digest back as a one-block message). This is no idle variation. Password stretching runs φ^k for k in the hundreds of thousands; hash chains for one-time passwords and hash-based signatures are iteration made into a protocol; Bitcoin's core primitive is literally φ^2, the double hash $\mathsf{SHA\text{-}256}(\mathsf{SHA\text{-}256}(\cdot))$. And iteration is its own coordinate system for the random-function conjecture: the *dynamics* of φ—how its image shrinks, where its orbits settle, whether it hides fixed points—are a fresh battery of falsifiable predictions, sharper in some ways than the static silhouette of section 4 because they probe the function's interaction with *itself*. This section reads those predictions off the random-map model and asks, in each case, what an anomaly would mean. The user's instinct is the right place to start: one might imagine iteration eventually shuffling the state space like a random permutation—and seeing *exactly why that almost happens, and exactly what stops it*, turns out to expose the single design decision that makes SHA-256 one-way.

5.1 Iteration leaks: the shrinking image

A permutation loses nothing: applied repeatedly, it merely relabels. A random-looking map is different, and the difference compounds. Recall from section 4.1 that one application of a random map covers only $(1 - e^{-1})N \approx 0.63\,N$ of the space; the leaves—the 37% of points with no preimage—are gone after one step and never return. Apply φ again and the survivors thin again, by the same logic applied to the image. Writing β_k for the expected fraction of the space still reachable after k steps, the recurrence is exactly

$$\beta_0 = 1, \qquad \beta_{k+1} = 1 - e^{-\beta_k},$$

("a point survives to step $k+1$ iff at least one of its φ-preimages survived to step k"), and its solution decays with a clean asymptotic law (Flajolet and Odlyzko, 1990):

$$\beta_k \;\sim\; \frac{2}{k} \qquad (k \to \infty).$$

Iteration is a funnel (fig. 8). After k rounds the reachable image has shrunk to about $2N/k$ points, so the process has quietly discarded roughly $\log_2(k/2)$ bits of range. The number is small but not zero and the moral is exact: *composing a hash with itself can only lose entropy, never create it*. For a password-stretching count of $k = 10^6$, the naive iterate φ^k confines its outputs to

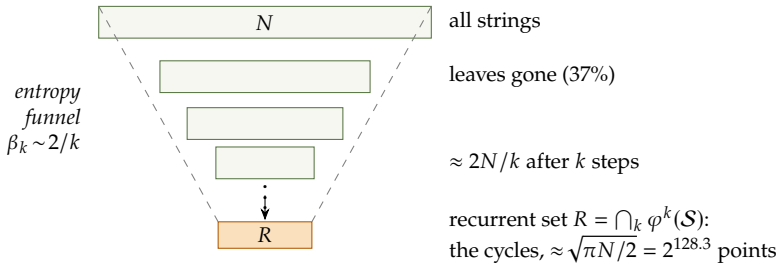


Figure 8: Iterating a random-looking map is a one-way funnel. Each application discards the current leaves, so the reachable image shrinks as $\beta_k \sim 2/k$ toward the *recurrent set* R—the union of all cycles, about $\sqrt{\pi N/2} \approx 2^{128.3}$ points. On R, and only there, φ acts as a permutation (section 5.2). The funnel is why iterated hashing loses (a little) entropy, and why real key-stretching functions are *not* pure iteration.

a 2^{-19} fraction of the digest space—nineteen bits of the advertised 256 silently surrendered to the funnel.

Two consequences deserve flagging. First, the prediction is *measurable*: the shrinkage law $\beta_k \sim 2/k$ is a specific curve that truncated SHA-256 must trace if it is random-like, and a reduced-round or structured map need not (section 9). Second, the practical fix is illuminating precisely because it *breaks* the composition. Standardized password-based key derivation does not compute φ^k; it re-injects the password and a salt at every step, iterating a map like $x \mapsto \text{SHA-256}(x \oplus \text{password})$ that changes each round (Turan et al., 2010). That is no longer a single function composed with itself—it is a *driven* iteration, formally the random walk of section 6.6 rather than a functional graph—and the salt-dependent driving is exactly what defeats the funnel (and the precomputation attack of section 5.4). The theory tells the engineer why the naive design leaks and the standard one does not.

5.2 The permutation at the bottom of the funnel

Where does the funnel end? Not at nothing: the image cannot shrink below the points that lie on cycles, since a cyclic point is its own distant iterate and survives forever. These *recurrent* points—the set $R = \bigcap_{k \geq 0} \varphi^k(\mathcal{S})$, the union of all the cycles of fig. 7—are the funnel's floor, and here the user's intuition lands exactly. *On R, iterated SHA-256 is a genuine permutation.* This is not a probabilistic near-miss; it is a theorem about every finite self-map: restricted to its recurrent set a function is a bijection, because each cyclic point has exactly one cyclic preimage (the point before it on its cycle). SHA-256 permutes its cycles, and φ carried to the limit is a permutation of the roughly $\sqrt{\pi N/2} \approx 2^{128.3}$ recurrent points—conjecturally a *uniformly random* permutation of a random such subset, which is the strongest form of the random-map hypothesis.

So the imagined "random permutation induced by iteration" is real—but confined to an unreachable floor. The recurrent set is a random-looking subset of size $2^{128.3}$, with no short description and no way to enumerate it short of traversing orbits of length 2^{128}; one can prove it hosts a permutation without ever exhibiting a single one of its elements. This is the same epistemic shape as the rest of the paper (section 4.3): a structure we can characterize in full and visit never.

Now the sharp contrast that answers "why astronomically unlikely?" Being a permutation

on the *recurrent set* is automatic; being a permutation on the *whole* space $\mathcal{S}$ is the rarest thing a map can be. The fraction of self-maps that are permutations is

$$\frac{N!}{N^N} \;\approx\; \sqrt{2\pi N}\, e^{-N},$$

so a uniformly random map is a permutation with probability of order e^{-N}: not astronomically but *doubly*-exponentially small, about $2^{-1.44\cdot 2^{256}}$. Equivalently, injectivity anywhere is expensive—a random map already has about $0.37N$ points sharing their images with others. Hence if SHA-256 were ever found to be injective on its full domain, or even to cover meaningfully more than the predicted $0.63N$ on the first step, that would not be a lucky coincidence: it would be a gross structural signal, a deviation of the exact kind section 6.7 is built to detect. The permutation lives at the bottom of the funnel by necessity; a permutation at the *top* would be a five-alarm anomaly.

5.3 "Unless...": the one addition that breaks the bijection

Why is SHA-256 not that anomaly—why does it funnel at all, rather than permute? The answer is a single line of its construction, and it is worth seeing because it identifies one-wayness with lossiness as the *same design act*. Recall from section 3.3 that the compression function is a block cipher run in Davies–Meyer mode:

$$f(h, m) \;=\; E_m(h) \boxplus h,$$

where E_m—the core of SHA-256, essentially the cipher SHACAL-2—is, for each fixed message block m, a *permutation* of the state (every round is invertible, so the whole cipher is a bijection one could run backward). If f were just E_m, iteration would be the iteration of a permutation: reversible, lossless, its functional graph all cycles and no trees, injective on the whole space—precisely the doubly-exponentially unlikely object of section 5.2. The feedforward "$\boxplus h$" is what destroys this (fig. 9). Adding the input back turns a bijection into a generically many-to-one map—the sum of a permutation and the identity need not be injective—and it is this one modular addition that makes f one-way, non-invertible, and lossy all at once. The tree-and-funnel geometry of sections 4.1 and 5.1, the 37% of leaves, the $2/k$ shrinkage, the very existence of preimage-resistance: all of it is manufactured by the single "$\boxplus h$" that the designers added exactly so the rounds could not be run backward.

This is the resolution of the opening puzzle. Iterated SHA-256 does induce a random permutation—but on its recurrent floor, by necessity, not on the whole space, which the feedforward forbids. The "unless" is literal: *unless one deletes the "*$\boxplus h$*,"* in which case SHA-256 degenerates into its underlying block cipher, iteration becomes reversible, and the funnel disappears along with the security. The lossiness the user intuited as a near-impossibility is, seen correctly, the deliberate and load-bearing content of the design.

5.4 Vulnerabilities specific to iteration

Iteration is not merely a lens; it is an attack surface and a building block, and three concrete phenomena round out the picture.

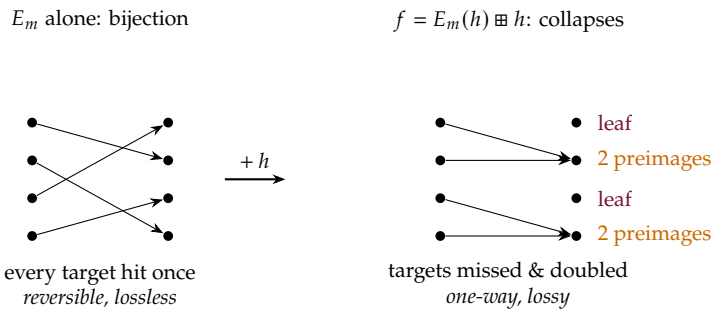


Figure 9: The "unless." The cipher E_m at the heart of SHA-256 is a permutation (left): iterate it and nothing is lost. The Davies–Meyer feedforward $f = E_m(h) \boxplus h$ (right) adds the input back, turning the bijection into a many-to-one map with leaves and collisions—the source of every tree, every lost bit, and all one-wayness. Iterated SHA-256 would be a random permutation on its whole domain (section 5.2) were it not for this one modular addition.

Hash chains, and why they are only as strong as one step. Applying φ repeatedly and revealing the chain *backward* is a classic authentication primitive: Lamport's one-time passwords (Lamport, 1981) store $\varphi^n(s)$ on the server and release $\varphi^{n-1}(s), \varphi^{n-2}(s), \ldots$ one login at a time, each value verified by one forward application and unforgeable ahead of time by one-wayness. The same skeleton—iterate a one-way step, reveal in reverse—underlies hash-based signatures (section 2.1). What the functional-graph view contributes is a caution: a chain is exactly as strong as the one-wayness of a *single* step, and if a short cycle or a cheap partial inversion existed for the toy scales of section 9, the whole chain would unravel at once. Iteration concentrates risk onto the per-step preimage problem.

Time–memory trade-offs: iteration as the attacker's weapon. The most striking vulnerability is that iterated hashing is the data structure an adversary uses to *invert* a one-way function by precomputation. Hellman's time–memory trade-off (Hellman, 1980) covers the domain with long chains $x, \varphi(x), \varphi^2(x), \ldots$, stores only the endpoints, and thereby inverts φ on a fresh target in online time T and memory M obeying

$$T\,M^2 \;\approx\; N^2,$$

so that a one-time precomputation buys balanced online cost $T = M = N^{2/3}$—far below the N of blind brute force. Oechslin's *rainbow tables* (Oechslin, 2003) sharpen the constants with per-column reduction functions and made the technique an everyday password-cracking tool. At full SHA-256 scale $N^{2/3} = 2^{170.7}$ is safely unreachable, but against low-entropy inputs (short passwords) the trade-off is devastating—and its defeat is once again the *driving* of section 5.1: a per-user salt makes every target's chain independent, so no shared precomputation applies (Turan et al., 2010). The same iteration that stretches a key for the defender builds the table for the attacker; salt is what breaks the symmetry.

Fixed points and short cycles: cheap, sharp predictions. Finally, iteration poses questions with crisp random-model answers that are eminently testable at toy scale. How many fixed points—strings with SHA-256$(x) = x$—should exist? For a random map the count is Poisson(1):

expected value exactly 1, with probability $e^{-1} \approx 0.37$ of *none*. So SHA-256 very probably has a handful (likely one) genuine 256-bit fixed point that no one will ever exhibit, a pleasant cousin of the "does SHA-256$(x) = x$ have a solution?" puzzle. More generally the expected number of cycles of length ℓ is $1/\ell$, a harmonic profile summing to the $\sqrt{\pi N/2}$ recurrent points of section 5.2. Each of these is a falsifiable prediction about a structureless map; a truncated SHA-256 that showed too many fixed points, or cycles of anomalous length, would be exhibiting structure. That nobody has looked, at model-organism scale, is the recurring opportunity of this prospectus.

Project 6: The dynamics of iteration

Extend the functional-graph atlas of section 4.3 from static statistics to *dynamics*. For $\varphi_n(x) =$ first n bits of SHA-256(x), $n = 16, \ldots, 30$: (i) measure the image-shrinkage curve $\beta_k = |\varphi_n^k(\mathcal{S})|/N$ and fit it against the $\beta_{k+1} = 1 - e^{-\beta_k}$ law and its $2/k$ tail (Flajolet and Odlyzko, 1990); (ii) count fixed points and short cycles and compare with the Poisson(1) and $1/\ell$ predictions; (iii) locate the recurrent set and confirm φ permutes it. Then repeat with reduced-round SHA-256 and with the feedforward *removed* (pure E_m, a permutation) as a control, watching the funnel switch off. As a capstone, implement a small Hellman table (Hellman, 1980) on φ_n and measure the $TM^2 = N^2$ trade-off directly, with and without salt. Every quantity is exactly computable for $n \leq 30$, and the "remove the feedforward" control makes the mechanism of section 5.3 visible as data.

Prerequisites: programming; a first probability course *Window:* 6–8 weeks (flagship)

6 Frameworks, and the size of the gap

We now assemble the existing theoretical apparatus around SHA-256 and take honest stock of how far it reaches. The summary in advance: there are mature frameworks on *both* sides of the object—idealized models above it, empirical and cryptanalytic measurements below it—and a canyon in the middle where the function itself lives.

6.1 What "pseudorandom" officially means

The foundational definitions of pseudorandomness are about *families*, not single objects. A pseudorandom function family in the sense of Goldreich, Goldwasser, and Micali (Goldreich et al., 1986) is a keyed collection $\{F_k\}$ such that no efficient algorithm, allowed to query a black box, can tell a randomly keyed F_k from a truly random function with nonnegligible advantage. This definition is a triumph—it supports composition theorems, equivalences, a whole complexity theory of randomness—but notice that it *cannot even be stated* for the fixed, keyless, public function SHA-256. For any fixed function, a trivial *distinguisher* exists. A distinguisher is the formal adversary in these definitions: an efficient algorithm that queries a black box containing either the function under test or a truly random function, emits a one-bit verdict, and is scored by its *advantage*, the gap between its acceptance probabilities in the two worlds. The notion descends from Yao's theory of computational indistinguishability (Yao, 1982) and receives its textbook treatment in Chapter 3 of Goldreich (2001). In our context the trivial distinguisher simply hard-codes one known value: query the box at 0 and accept if the answer is the published SHA-256(0). Against SHA-256 it accepts always; against a random

function, with probability 2^{-256}; its advantage is essentially 1, and no keyless public function escapes it. The same foundational awkwardness afflicts collision resistance: since collisions exist by pigeonhole, "no efficient algorithm finds one" is technically false—some two-line program prints a collision; humanity just cannot exhibit it. Rogaway's "human ignorance" formalism (Rogaway, 2006) repairs the semantics by making theorems constructive (explicit reductions transforming any breaker of a protocol into an explicit collision-finder), and it is the honest logical home of every security claim about SHA-256. But note what has happened: *the definition of success has been moved from mathematics into history*—"no one has written down a collision, so far."

6.2 Idealized models: reasoning above the object

Protocol designers escape upward. In the random-oracle model (Bellare and Rogaway, 1993) one simply *postulates* a public uniformly random function and proves protocols secure relative to it; the leap back to SHA-256 is heuristic, frankly acknowledged, and empirically successful. The model even has an internal quality-control theory: *indifferentiability* (Maurer et al., 2004) asks whether a construction built from an ideal component is safely interchangeable with an ideal whole, and by this standard plain Merkle–Damgård *fails*—the length-extension property (from SHA-256(m) anyone computes SHA-256($m\|\text{pad}\|m'$) without knowing m) is a genuine, exploitable deviation from random-oracle behavior, identified precisely and repaired by variants with security proofs (Coron et al., 2005). This is the theory at its best: it cannot certify f itself, but given an idealized f it classifies *constructions* completely. The architecture above the compression function is, in a real sense, solved mathematics—the two-layer picture of section 3.4, where the Merkle–Damgård mode (theorem 3.1) and the Davies–Meyer compression (theorem 3.3) each carry a theorem—and all residual mystery is compressed into the finite map f.

There is a sharper way to say that the model is a heuristic, and it is a theorem rather than a caveat. Canetti, Goldreich, and Halevi constructed signature and encryption schemes that are *provably secure* in the random-oracle model and *provably insecure* the moment the oracle is replaced by *any* concrete, publicly specified function whatsoever, SHA-256 or otherwise (Canetti et al., 2004). The construction is diagonal—the scheme tests whether the code it is given is a short program and betrays its secret if so, which a true random oracle never is but every real hash is—so it is contrived, and no natural protocol is known to suffer. But it settles the logic: a random-oracle proof is not a proof about SHA-256 with an admitted gap; it is a proof about an idealized object that *no* function can fully become. The heuristic is not merely unproven, it is in general false, and its continued empirical success (section 6.4) is therefore itself part of the natural history—a regularity of the world, not a lemma.

6.3 The construction is not the oracle: provable structure above f

Here is a genuine surprise, and it cuts against the drumbeat of this paper. We keep asserting that SHA-256 exhibits *no* observable deviation from a random function. That is true of the compression function f; it is *false* of the Merkle–Damgård wrapper around it. The iteration mode has provable, nameable, exploitable structure—deviations from random-oracle behavior that hold *even if f is a perfect random oracle*—and they are among the most elegant results in the subject. They locate the paper's mystery more precisely than we have so far: the residue of the

unknown lives in f, because the part outside f is not unknown at all. Its non-randomness is a set of theorems.

The keystone is Joux's multicollision attack (Joux, 2004) (fig. 10). For a true 256-bit random function, finding 2^k messages that all share one digest should cost about $2^{256(2^k-1)/2^k}$ work—for eight-way agreement, near 2^{224}, astronomically beyond a single collision's 2^{128}. In an iterated hash it costs only k times a single collision. The trick is pure construction-level reasoning: birthday-find a colliding *block pair* at the first chaining value, then *from the merged state* find another colliding block pair, and so on for k stages; freely mixing the two choices at each stage yields 2^k full messages all landing on the same final digest, at total cost $k \cdot 2^{128}$ instead of $2^{128\cdot(2^k-1)/2^k}$. Cascading is the immediate casualty: hashing with $H_1 \| H_2$, the "belt and braces" habit of concatenating two hashes for safety, is no stronger than the better single one—build a $2^{n_1/2}$-multicollision in H_1, then by pigeonhole two of its exponentially many members collide in H_2 as well. A folklore engineering practice was refuted by one structural idea, no cryptanalysis of any f required.

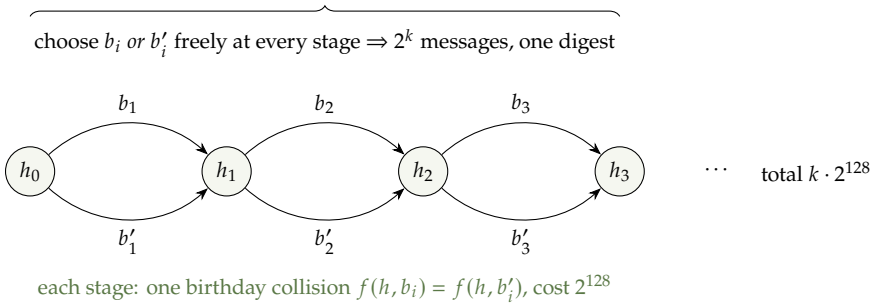


Figure 10: Joux multicollisions (Joux, 2004). Because the state is a narrow *delay line* passed from block to block (section 3.3), a chain of k independent single collisions multiplies into 2^k colliding messages at only k times the cost of one—a provable, non-statistical deviation from random-oracle behavior that lives entirely in the iteration, not in f. The narrow internal state is the culprit; a wide-pipe or sponge design (the SHA-3 sponge of section 3.1) closes exactly this door.

Two further theorems mine the same seam, and both convert the *narrowness of the internal state*—the eight-word delay line of section 3.3, no wider than the output—into an attack. Kelsey and Schneier showed that a *second preimage* for a long message is far cheaper than the ideal 2^{256}: given a target message of 2^ℓ blocks, "expandable messages" (a multicollision between message lengths) let one splice in a forged prefix that rejoins the victim's chain at some interior state, at cost about $2^{256-\ell}$ (Kelsey and Schneier, 2005). Long messages are *provably* weaker than short ones—a sentence with no meaning at all for a random oracle, where every input is independent. Kelsey and Kohno pushed the genre from collision to *commitment fraud*: their "herding" attack precomputes a branching "diamond" structure of chaining values, publishes a single digest as a soothsayer's sealed prophecy, and then, given any later-revealed prefix, herds it into the committed digest by appending a short suffix (Kelsey and Kohno, 2006). The hash's advertised binding property—the entire basis of the commitment use of section 2.1—is quantitatively weakened by construction-level structure alone.

The moral reframes the whole paper, so we state it flatly. Every one of these attacks treats f as an ideal random oracle and still breaks the advertised behavior of SHA-256-the-construction;

each is a *theorem*, not a measurement, and each traces to one visible design fact—the internal state is no wider than the digest, so the chaining value is a bottleneck that a random function would not have (section 3.3). This is why the SHA-3 competition selected a sponge with a *wide* internal state (Bertoni et al., 2007) that structurally forecloses all three attacks, and why the honest slogan is not "SHA-256 behaves like a random oracle" but the sharper "*f behaves like a random function, and the wrapper's deviations from a random oracle are completely known.*" The mystery of this paper is now pinned exactly where it belongs: not in the architecture, which is theorems all the way down, but in the single finite map f, about which there are none.

6.4 Empirical and adversarial measurement: attacking from below

From below, two very different measurement cultures apply.

Statistical testing treats the function as a black-box PRNG and runs batteries of hypothesis tests: NIST's SP 800-22 suite (Rukhin et al., 2010) and the far more demanding TestU01 "BigCrush" (L'Ecuyer and Simard, 2007). SHA-256-based generators pass everything, which is informative but bounded: these batteries famously demolish the classical linear generators (a Mersenne Twister fails BigCrush's linear-complexity tests; lattice structure betrays linear congruential generators—the pathologies catalogued since Knuth (Knuth, 1997)), so passing places SHA-256 strictly above the traditional PRNG canon; but a battery is a fixed, finite set of null hypotheses, and structure invisible to all of them may be gross. Knuth's own moral stands: statistical testing can refute, never confirm. For SHA-256 the concrete record is easily stated: the counter sequence SHA-256(1), SHA-256(2), SHA-256(3), ... is not merely testable but *standardized*—NIST's Hash_DRBG is essentially this generator (Barker and Kelsey, 2015)—and the same construction over SHA-1 passed every one of BigCrush's tests in the paper that introduced the battery. Passes of this kind are so thoroughly expected that the literature scarcely bothers to record them. To see what that expectation is calibrated against—and what a *finished* theory of pseudorandomness looks like—we pause (section 6.6).

The record deserves tabulating battery by battery, because the honest picture distinguishes three things the phrase "passes every test" usually blurs: what has actually been *run* and published, what is merely *expected* and therefore never written down, and what is *provably out of reach* of any feasible test at all (table 1).

Battery (size)	What it would catch	Status on the SHA family	Source
NIST SP 800-22 (15 tests)	monobit & block frequency, runs, matrix rank, spectral, template matching, Maurer universal, linear complexity, serial, approximate entropy, cusums, excursions	The standard instrument for cryptographic RNGs. The SHA-256 counter generator is standardized as Hash_DRBG and passes; a failure would be reportable and none is reported. Passing is *expected*, so rarely published as such.	Rukhin et al. (2010); Barker and Kelsey (2015)
TestU01 Crush / BigCrush (~96/106 tests, 160 stats)	birthday spacings, collisions, gaps, poker, rank, linear complexity, Hamming-weight, random-walk statistics	**Documented run.** SHA-1 (OFB and CTR modes) passes *all*; AES likewise, bar one p-value that cleared on replication. SHA-256 itself was not in that study, but is expected to behave identically.	L'Ecuyer and Simard (2007)
DIEHARD / Dieharder (~30 tests)	birthday spacings, overlapping permutations, binary rank, "monkey" (bitstream) tests	Routinely applied to cryptographic generators and passed; we know of no SHA-256-specific published study—passing is folklore, exactly as the text describes.	Marsaglia (software)
PractRand (streams to $\gtrsim$ TB)	bit-pattern and transition anomalies visible only at very large sample volume	Applied to SHA-256-based counter generators and passed to the volumes tested. A software tool, with no formal paper to cite.	— (software)
Global 256-bit uniformity / closeness	*any* constant-ε deviation of the *full* output distribution from uniform	**Not feasible.** Requires $\sim 2^{128}$ samples—the birthday wall as an information-theoretic lower bound; provably undecidable at any affordable cost (section 6.5).	Canonne (2020)

Table 1: Standard statistical batteries and their verdict on the SHA-256 family. Only the TestU01 row is a *documented, published* run—and it tested SHA-1, not SHA-256. The SP 800-22 row rests on standardization and the absence of reported failure; the DIEHARD and PractRand rows on routine practice that the literature, expecting success, does not record; and the last row is not a gap in effort but a theorem (section 6.5) that no feasible sample can close. Every battery that *has* been run reports uniform; the one that would see the whole distribution cannot be run.

It is worth being concrete about *which* regularities are absent and which are present, since "passes the batteries" is too coarse to convey the texture of the thing. A ledger.

*Regularities SHA-256 does **not** exhibit* (all empirical, none proven absent):

- **No lattice structure.** Successive output tuples show no Marsaglia effect: where the d-tuples of a linear congruential generator lie on at most $(d!\,m)^{1/d}$ hyperplanes (section 6.6; Marsaglia, 1968), the points $(\text{SHA-256}(n), \text{SHA-256}(n{+}1), \dots)$ display no detectable hyperplane concentration, no lattice, no low-dimensional alignment, in any projection anyone has computed.
- **No observable period.** The counter stream $\text{SHA-256}(1), \text{SHA-256}(2), \dots$ has no detected cycle; viewed as iteration (section 4) the expected recurrence time is $\sim 2^{128}$, operationally

unreachable. Contrast the classical generators, whose periods are exact theorems ($2^{19937}-1$ for the Mersenne Twister; Matsumoto and Nishimura, 1998).

- **No linear or low-degree structure.** The $\mathbb{F}_2$-linear complexity of the output stream is empirically near-maximal—no low linear complexity, the exact defect that sinks the Twister on BigCrush—and no correlation of any output bit with a low-degree Boolean function of the input has been found. Each output bit, as a function of the input, appears to have a flat Walsh–Hadamard spectrum: no good linear approximation, the property linear cryptanalysis would exploit.

- **No first-order statistical bias.** Output bit frequencies, runs, gaps, block frequencies, and autocorrelations at every measured lag sit within sampling noise of the uniform expectation—the content of "passes SP 800-22 and BigCrush."

Regularities ***SHA-256 does*** *exhibit* (these are the true structure, and they are exactly the harmless kind):

- **Determinism.** It is a fixed function: SHA-256(x) is always the same, the one perfectly reliable "pattern," and the sole basis of every use in section 2.1.

- **The avalanche property.** Flipping a single input bit flips each of the 256 output bits with probability indistinguishable from $\frac{1}{2}$, and near-independently, so the number of changed output bits is essentially Binomial$(256, \frac{1}{2})$, sharply peaked at 128. This *strict avalanche criterion*, a deliberate design target since Webster and Tavares (1986), is a regularity in the sense that it holds tightly and measurably—a property of *structurelessness itself*, and the subject of a project below.

- **Completeness/diffusion.** After the full round count every output bit depends on every input bit; there is no input coordinate any output ignores.

The avalanche property is the one to dwell on, because it dictates the self-correlation structure—or rather its total absence. Consecutive counter values n and $n+1$ differ, as bit strings, in just a few low-order bits; avalanche then forces SHA-256(n) and SHA-256$(n+1)$ to differ in about half of *their* bits, with no residual correlation between them. So the output stream has essentially zero autocorrelation at every lag and no continuity in any metric on the inputs: nearness of arguments carries no information about nearness of values. This is precisely the opposite of the individual ARX primitives, each of which is 2-adically 1-Lipschitz—continuous, correlation-preserving (section 3.2)—and it is the composition of 64 rounds that annihilates that continuity. The designed regularity (avalanche) is the mechanism that manufactures the absence of every other regularity a statistical test could name.

Cryptanalysis is the adversarial culture: bespoke, design-specific, and quantitative in a different way—its currency is the number of rounds that can be broken and at what cost. Section 7 anatomizes the method behind these numbers; the headline state of the art on SHA-256: practical collisions for the compression function reduced to 28 of its 64 rounds and theoretical ones at 31 (Mendel et al., 2013), "semi-free-start" collisions (the attacker also chooses the chaining value) at 39 (Li et al., 2024), and preimage attacks via bicliques reaching about 45 rounds at cost barely below brute force (Khovratovich et al., 2012). Two readings of these numbers coexist. Comforting: after twenty years of world-class effort, half the rounds stand untouched, and nothing approaches the full function. Discomfiting: the same community's

tools went, for SHA-1, from academic paper to industrial collision in one decade (section 2.3), and there is no theory predicting which trajectory SHA-256 is on. The round margin is a measurement of *our* progress, not of *its* strength.

6.5 How much can statistics even see? The sublinear ceiling

Knuth's moral—testing refutes, never confirms—has a quantitative refinement worth stating, because it says precisely *which* questions about SHA-256's output distribution are answerable with a feasible number of samples and which are not. The relevant theory is *distribution testing*, the sublinear-algorithms study of how many independent samples from an unknown distribution over a domain of size N are needed to decide a property of it; the modern survey is Canonne (2020).

The foundational result is the sample complexity of *uniformity testing*: to distinguish "the distribution is uniform on N points" from "it is ε-far from uniform in total-variation distance," one needs, and suffices with, $\Theta(\sqrt{N}/\varepsilon^2)$ samples. The upper bound is achieved by a startlingly simple statistic—*count collisions*: draw m samples, count coinciding pairs, and compare against the $\binom{m}{2}/N$ expected under uniformity, a test going back to Goldreich and Ron and made sample-optimal by Paninski (both in Canonne, 2020). The reader will recognize the birthday calculus of section 4: collisions among $\sqrt{N}$ samples are exactly the signal, and $\sqrt{N}$ is exactly the threshold.

Point this at SHA-256's 256-bit output distribution ($N = 2^{256}$) and the ceiling is stark: detecting *any* constant-ε deviation of the global output distribution from uniform requires on the order of 2^{128} samples—the birthday wall again, now wearing the hat of an *information-theoretic lower bound* on testing rather than an algorithmic cost. No battery, however clever, that draws fewer than $\sim 2^{128}$ outputs can certify or refute uniformity of the full distribution; the question is not hard, it is *unanswerable* at feasible sample sizes. So whatever SP 800-22 and BigCrush are doing when they "pass SHA-256," they are emphatically *not* testing the 256-bit distribution.

What they *are* testing is the reconciling point, and it is exactly the sublinear-feasible part. Every test in those batteries is a low-dimensional or low-complexity statistic—the frequency of single bits, of short blocks, of runs; linear complexity; a fixed template count—each of which lives on a domain small enough (2^k for modest k, or a real-valued summary) that $\sqrt{\cdot}$ samples suffice and a laptop has them. Passing means: on every *marginal* anyone has thought to compute and can afford to test, SHA-256 matches uniform. The 2^{128} ceiling says the joint, high-dimensional distribution is untestable; the batteries probe its cheap shadows, and the shadows are clean. That precise divorce—feasible low-dimensional marginals all uniform, infeasible global distribution forever unknown—is the honest content of "passes every test," and it is a sharper statement than the phrase usually carries.

Two further refinements are worth flagging because each squeezes signal from limited samples rather than brute count. First, *closeness* (two-sample) testing: deciding whether SHA-256's output distribution *equals* a reference distribution, rather than testing it against the explicit uniform, has its own sublinear theory (Batu et al., 2013), and it is the right frame for the comparative program of section 4—one does not test SHA-256 against "uniform" in the abstract but against a measured random-map *control*, in the low-dimensional Flajolet coordinates where $\sqrt{\cdot}$ samples reach. Second, the *p-value-of-p-values* trick that SP 800-22 actually uses (Rukhin et al., 2010): run one weak test on many independent streams, then test whether the resulting

p-values are themselves uniform on $[0, 1]$. A bias too faint for any single run to flag becomes visible in the second-order distribution of the p-values—a genuine way to amplify a subtle distributional signal without paying for prohibitively many samples per test. Against SHA-256 these meta-tests, too, report uniform. None of this proves anything; but it locates, to the sample, the boundary between what the empirical program can decide and what it provably cannot.

Project 7: Collision testing at the birthday threshold

Implement the collision-count uniformity tester on the n-bit output of truncated SHA-256 for n up to ~48, and measure its *sample complexity* empirically: how many outputs are needed to reject uniformity for reduced-round or deliberately-biased variants, and how does the number scale with n? Confirm the $\Theta(\sqrt{2^n})$ law (Canonne, 2020) and watch it become the same $2^{n/2}$ wall the collision-hunting project of section 4.3 runs into—one barrier, met from the testing side and the attacking side at once. Then add the p-value-of-p-values layer and see how much fainter a bias it can catch at fixed budget.

Prerequisites: probability; programming *Window:* 4–6 weeks

6.6 Interlude: what a finished theory looks like

Two mature bodies of mathematics sit directly adjacent to our object, and both show what "understanding a pseudorandom process" means when understanding is actually achieved.

Pseudorandom numbers and finite fields. Pseudorandom number generators were born with the stored-program computer: Monte Carlo simulation demanded rivers of reproducible random digits, physical sources were slow and unrepeatable, and by mid-century von Neumann's middle-square method and Lehmer's linear congruential generator $x_{n+1} = ax_n + c \bmod m$ had founded the subject (von Neumann also supplied its founding joke, about the "state of sin" of anyone producing random digits by arithmetic). The subject's demands are precise and *non-adversarial*: a simulation needs equidistribution of the statistics it consumes, a long period, and above all speed—a physics run may consume 10^{12} variates, so a generator must cost a few word operations per output. The discipline was codified by Knuth (1997, Ch. 3), and its mature mathematical home is the arithmetic of finite fields, as systematized by Lidl and Niederreiter (Lidl and Niederreiter, 1997; Niederreiter, 1992). And here the theorems are *theorems*. A congruential generator attains full period exactly when three elementary congruence conditions on (a, c, m) hold (the Hull–Dobell criterion; Knuth, 1997, §3.2.1). A linear recurrence over $\mathbb{F}_2$ with characteristic polynomial f has period equal to the order of f, maximal $2^k - 1$ precisely when f is primitive (Lidl and Niederreiter, 1997, Ch. 8)—which is how Matsumoto and Nishimura could *prove* that their Mersenne Twister has period $2^{19937} - 1$ and equidistribution in 623 dimensions (Matsumoto and Nishimura, 1998). Marsaglia *proved* that successive d-tuples from any congruential generator fall on at most $(d!\, m)^{1/d}$ hyperplanes (Marsaglia, 1968)—for $d = 10$ and $m = 2^{32}$, a few dozen planes carrying every point the generator will ever produce. For nonlinear congruential generators, equidistribution is controlled through exponential-sum estimates, so that Weil's Riemann hypothesis for curves over finite fields quietly underwrites the uniformity of practical generators (Niederreiter, 1992). Classical pseudorandomness, in short, is a satellite of the arithmetic of finite fields.

Now the double-edged contrast. These theorems exist *because* the generators are algebraically transparent—one clean structure, fully exposed—and the same transparency is what makes them cryptographically worthless: Marsaglia's planes and the Twister's $\mathbb{F}_2$-linearity are simultaneously the content of the theorems and the substance of the distinguishers (section 6.4). Efficiency tells the story from the cost side. A Twister step is a handful of word operations; SHA-256 spends on the order of a few thousand word operations per 256-bit output—one to two orders of magnitude more work per bit in software—and the entire surcharge purchases exactly one property, resistance to an intelligent adversary, which no simulation needs. Where the classical theory lives inside a single algebraic structure, SHA-256 deliberately straddles two incompatible ones (section 3.2) precisely so that no analogue of the Lidl–Niederreiter machinery can grip it. That is the trade: the theorems stop where the adversary begins.

Random permutations and the symmetric group. The second finished theory concerns a different pseudorandom object: a *shuffle* is a pseudorandom number generator valued in the symmetric group, and "how many riffle shuffles randomize a deck?" is the question "how many rounds suffice?" asked of cards. Here the theory reached a summit. Modeling repeated riffle shuffles as a random walk on S_{52}, Bayer and Diaconis (1992) computed the total-variation distance to uniformity essentially exactly, exhibiting the famous answer (seven shuffles) and, more importantly, the *cutoff phenomenon*. Their theorem is worth quoting in the exact form that makes the abruptness visible. Riffle-shuffling a deck of n cards $k = \frac{3}{2}\log_2 n + \theta$ times, the total-variation distance to the uniform distribution obeys

$$\left\| Q^{*k} - U \right\|_{\mathrm{TV}} \;=\; 1 - 2\,\Phi\!\left(\tfrac{-2^{-\theta}}{4\sqrt{3}}\right) + o(1), \qquad n \to \infty,$$

where Φ is the standard normal cumulative distribution function

$$\Phi(z) \;=\; \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{z} e^{-t^2/2}\,\mathrm{d}t$$

(Bayer and Diaconis, 1992, Thm. 1), so that the approach to uniformity is governed by the Gaussian tail $\int e^{-t^2/2}\,\mathrm{d}t$ evaluated at the exponentially small argument $-2^{-\theta}/(4\sqrt{3})$. The scaling parameter θ measures shuffles past the threshold $\frac{3}{2}\log_2 n$; because the right-hand side collapses from 1 to 0 as θ runs through an $O(1)$ window while the threshold itself grows like $\log_2 n$, the transition is a true cutoff. For $n = 52$ this puts the knee at $k \approx 8.55$ and yields distances dropping from 1.000 at $k = 5$ to 0.334 at $k = 7$ and 0.167 at $k = 8$: the origin of "seven shuffles." The technical engine behind such exact statements is the representation theory of finite groups: for the walk generated by random transpositions, Diaconis and Shahshahani (1981) proved cutoff at $\frac{1}{2}n\log n$ by Fourier analysis on S_n, with the sharp exponential rate $\|Q^{*k} - U\|_{\mathrm{TV}} \le \frac{1}{2}\,e^{-2c}$ once $k = \frac{1}{2}n(\log n + c)$—the irreducible representations, indexed by Young diagrams, diagonalize the walk, and a single upper-bound lemma converts character sums into sharp convergence inequalities. The method is expounded in Diaconis's monograph (Diaconis, 1988), and it is the standing model of what a *hard convergence theorem* for an iterated mixing process looks like: not "eventually close to uniform," but *exactly when*, with matching bounds.

Random walks on the hypercube, and the cutoff phenomenon. Between the card-shuffling summit and our object sits the solved mixing problem closest to SHA-256's own geometry: the

random walk on the hypercube $\mathbb{F}_2^n$—the very space of section 3.2, walked one coordinate at a time. At each step, pick one of the n bit positions uniformly and replace that bit by a fresh coin flip (this is the Ehrenfest urn of statistical mechanics, wearing bits). How many steps until the position is uniform on all 2^n points? The answer is a second exact cutoff theorem: the total-variation distance stays pinned near 1 until $\frac{1}{4}\, n \ln n$ steps, then collapses to 0 inside a window of width $O(n)$—asymptotically instantaneous (fig. 11; the phenomenon, its history, and this example are surveyed in Diaconis, 1996). The threshold is pure coupon collecting: a coordinate never yet touched is a coordinate whose bit is still the initial one, and $\frac{1}{4} n \ln n$ is when the last stragglers have been visited; the logarithm is the price of the *slowest* coordinate, not the average one—a moral worth remembering whenever "rounds needed to mix" is discussed.

Better still, the machinery is transparent enough to expose a coincidence this paper finds genuinely striking. On the abelian group $\mathbb{F}_2^n$ the Fourier analysis that diagonalized card shuffles becomes elementary: the characters are the Walsh functions $x \mapsto (-1)^{\langle u, x \rangle}$, every translation-invariant walk has them as eigenvectors, and *the eigenvalues are exactly the Walsh–Hadamard coefficients of the one-step distribution*. Mixing is slow precisely in the directions u where that coefficient is large. Now reread the empirical ledger of section 6.4: "flat Walsh–Hadamard spectrum, no good linear approximation" is the property *linear cryptanalysis* probes. The quantity the cryptanalyst hunts and the quantity that governs mixing are the same number: a linear distinguisher is a slow eigenvector, caught in the act. Mixing analysis and linear cryptanalysis are, in this abelian setting, one computation performed by two communities that rarely cite each other—and the projects of section 6.7 amount to measuring SHA-256's spectral gap experimentally, round by round. For scale: at $n = 256$ the theorem's threshold is $\frac{1}{4}\, n \ln n \approx 355$ single-coordinate updates, while one SHA-256 round nominally touches all 256 state bits at once—but in correlated fashion, which is exactly why no theorem applies and the measured curve is the interesting object.

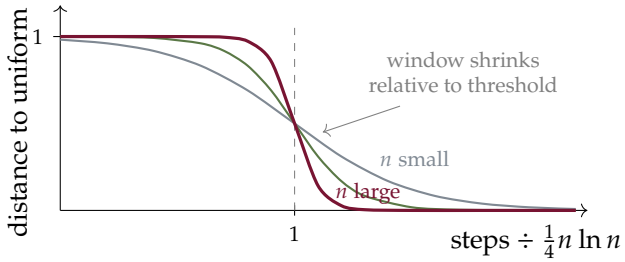


Figure 11: The cutoff phenomenon, schematically: distance to uniformity for the hypercube walk stays near 1, then collapses within a window that is asymptotically negligible compared to the $\frac{1}{4} n \ln n$ threshold. "How many rounds are enough?" has, for this toy relative of SHA-256, an answer sharp to within lower-order terms—and the toy-compression-walk project of this section asks whether SHA-256-family walks show the same knee.

The bridge to our object is not an analogy but a change of alphabet. Every keyed block cipher is a family of permutations, and SHA-256's own compression function is one in disguise (its core is a block cipher, run in Davies–Meyer mode, section 3.3). More directly: the chaining recursion $h_i = f(h_{i-1}, m_i)$ of section 3.1, fed *random* message blocks, is literally a random walk on the state space $\mathcal{S}$, driven by 2^{512} generators. Diaconis's questions transfer verbatim—how many blocks until the distribution of h_i is uniform? is there cutoff? what is the spectral

gap?—and, to our knowledge, none has been answered, or seriously asked, for any member of the SHA family at any scale. The obstruction is instructive: the representation-theoretic engine wants a group acting on itself, and the walk lives merely on a set—unless one exploits the two group structures the set actually carries (section 3.2). Here is a place where the paper's finite geometry and a fully developed classical theory face each other across a gap that small experiments can begin to survey.

Project 8: Cutoff for toy compression walks

Shrink the compression function (section 3.3) to an 8- or 16-bit state. Random message blocks then make chaining a random walk on 2^8 or 2^{16} points: build its transition matrix, compute the spectral gap, plot total-variation distance to uniformity against the number of blocks, and look for a cutoff profile. Vary the rounds-per-block and watch the gap and the cutoff window move. Every quantity here is exactly computable at toy scale, and the questions are verbatim those of the shuffling literature.

Prerequisites: probability; linear algebra; programming *Window:* 4–6 weeks

The interlude's moral for section 6.7: on one flank of SHA-256 stands a complete algebraic theory of linear generators, on the other a complete probabilistic theory of mixing walks on groups; the function itself was engineered into the mathematical no-man's-land between them—touching complex analysis through section 4, finite fields and 2-adic analysis through section 3.2, representation theory through the walk just described, and complexity theory through section 6.1—and nearly every one of those contacts is an unfollowed lead rather than a developed theory.

6.7 The gap, stated plainly

Now place the object among its relatives, the pseudorandom number generators, along two informal axes: how fast it is, and how much theory supports its randomness claims (fig. 12).

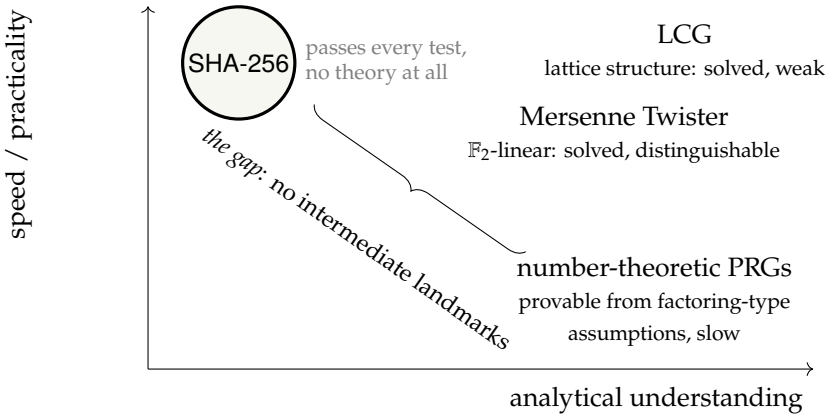


Figure 12: A cartoon of PRNG space. Classical generators are fast and fully understood—which is exactly how one knows they are weak. Number-theoretic generators carry proofs (conditional on hardness assumptions) and are slow. SHA-256 sits alone: fastest, empirically strongest, and analytically blank.

The classical generators are *transparent*: linear congruential generators are so well understood that their defects (Marsaglia's lattice planes) are theorems; twisters are $\mathbb{F}_2$-linear and hence completely analyzable, and completely distinguishable. At the other extreme, number-theoretic generators (of Blum–Micali type) come with genuine reductions—distinguishing their output is provably as hard as factoring-type problems—purchased at a price of thousands of arithmetic operations per output bit. SHA-256 belongs to neither culture: it has *no* reduction to any studied hard problem, *no* algebraic model that survives even a handful of rounds, and *no* distinguisher after twenty-five years. It is simultaneously the best-tested and least theorized object in the entire space.

Why should the gap be so large? Part of the answer is a legitimate excuse: an unconditional proof that SHA-256 (or anything) is a one-way or pseudorandom function would imply P ≠ NP and more; in Impagliazzo's celebrated taxonomy of possible computational worlds, whether we live in a world with one-way functions at all ("Minicrypt"/"Cryptomania" versus "Algorithmica"/"Pessiland") is precisely the great open question (Impagliazzo, 1995). Nobody should demand a proof. But the excuse covers less than the gap. Between "unconditional proof" and "nothing" there is an enormous unexplored middle: reductions to *any* clean assumption; partial structure theorems for few rounds; quantitative mixing results in either of the two geometries of section 3.2; distributional results in Flajolet's coordinates for scaled-down families (section 4). For linear generators that middle territory is dense with theorems; for ARX functions it is nearly empty—not, as far as we can tell, because theorems are known to be impossible, but because the object fell into a disciplinary crack: too applied for one community, too structureless for another, and (a hypothesis this prospectus exists to test) never yet attacked with the analytic-combinatorics instrument at the appropriate model-organism scale.

That is the research program: not "prove SHA-256 secure"—nobody sane proposes that—but *populate the gap with measurements*: an atlas of scaled-down SHA-256-family maps (fewer rounds, narrower words, fewer words), located in Flajolet's coordinate system, with the two-geometry anatomy of section 3.2 guiding which scaled families to compare. Every panel of that atlas is a small, finishable, publishable piece of natural history; section 9 cuts it into undergraduate-sized pieces.

Project 9: Where do the tests start failing?

Reduced-round SHA-256 must fail statistical batteries—at one round it is nearly raw message. Find the phase transition: run SP 800-22 (Rukhin et al., 2010) and TestU01 (L'Ecuyer and Simard, 2007) against generators built from r-round SHA-256 for $r = 1, 2, \ldots$ and chart which tests fail at which r. Compare with the round counts where cryptanalysis stalls (section 6.4). The two "difficulty frontiers" have, to our knowledge, never been plotted on one axis.

Prerequisites: programming; statistics *Window:* 4–6 weeks

Project 10: Avalanche cartography

Flip one input bit; which of the 256 output bits flip? For the true random-function silhouette each flips independently with probability $\frac{1}{2}$ (Webster and Tavares, 1986). Measure the full 512×256 influence matrix round by round, render it as heatmaps, and quantify convergence to $\frac{1}{2}$ (in what norm? after how many rounds? uniformly over bit positions?).

Then close the loop to the ledger of section 6.4: confirm that full-round avalanche forces the counter stream's lag-1 autocorrelation to zero, and measure how many rounds are needed before that self-correlation vanishes. Relate the observed diffusion speed to the rotation constants of section 3.3—the designers' choices become visible as geography.

Prerequisites: programming; linear algebra *Window:* 4–6 weeks

6.8 The road not taken: hashes with theorems

One region of fig. 12 deserves its own visit before we leave the map: the lower right, where proofs live and speed dies. It is not empty of hash functions. *Provably secure* hash functions exist, several are beautiful mathematics, and the world declined every one of them; both halves of that sentence are instructive.

Exhibit one: VSH, "very smooth hash" (Contini et al., 2006). Its collision resistance is an honest theorem, conditional on a single clean number-theoretic assumption (finding congruences of a certain smooth-square kind modulo a hard-to-factor n), and it is fast by the standards of its genre—only 25 times or so slower than the SHA family, where earlier provable designs had been slower by thousands. But the same multiplicative structure that powers the proof is *visible in the outputs*: VSH is not remotely a random-oracle stand-in (short relations among hash values exist by construction), so of the whole catalogue of section 2.1 it can serve collision-bound uses only. The proof purchases one property by installing exactly the kind of algebraic transparency that section 6.6 showed to be fatal to all the others.

Exhibit two is closer to this paper's geometry. *Cayley hashes* walk a graph: fix a group G and two generators g_0, g_1; hash a bit string by reading it as directions, $m_1 m_2 \cdots m_k \mapsto g_{m_1} g_{m_2} \cdots g_{m_k}$. Tillich and Zémor proposed $G = \mathrm{SL}_2(\mathbb{F}_{2^n})$ in 1994 (Tillich and Zémor, 1994); Charles, Goren, and Lauter modernized the idea with Lubotzky–Phillips–Sarnak Ramanujan graphs and, in a second variant, supersingular isogeny graphs of elliptic curves (Charles et al., 2009). Two theorems come for free. A collision is a pair of distinct words with the same product—so collision resistance *is* a group-theoretic statement (no short relations among the generators are findable), inherited whole from a recognizable mathematical problem. And output quality is spectral: if the Cayley graph is an *expander*—nontrivial eigenvalues uniformly bounded away from the degree—then the distribution of hashes of length-k messages converges to uniform at an exponential rate governed by the spectral gap (Hoory et al., 2006). The mixing mathematics of section 6.6 returns, no longer as analysis but as a *design principle*: these are hash functions whose avalanche property is a theorem about eigenvalues.

Then the pothole. In 2008 the LPS instantiation was broken—collisions computed outright—by lifting the problem from the finite group to a characteristic-zero arithmetic where Euclidean-style algorithms find short relations; the authors of the break were Tillich and Zémor, the designers of 1994, and their own original SL_2 proposal later fell, for its published parameters, to a similar lift (Tillich and Zémor, 2008). Note carefully what died. The theorems were and remain true; the *assumptions* were false—the underlying group problems, young and algebraically rich, simply were not hard. (The isogeny-graph sibling's assumption has so far survived, and seeded a whole branch of post-quantum cryptography.) The moral is the sober one: a security proof does not abolish risk, it relocates risk from the object to the assumption, and an assumption chosen for its provability can be weaker than ignorance. Set beside section 2.3: MD5 fell *with* twenty years of massive use and no theorem; LPS fell *with* a theorem and almost no use. The

fossil record refuses to pick a side.

Why, then, does the world run on the theorem-free object? Speed is the visible reason—two orders of magnitude—but the load-bearing one is now easy to state with this section's vocabulary: every use of section 2.1 beyond bare collision resistance needs the random-oracle *pretension*, behaving like structurelessness itself, and a provable design is structured *by construction*; its proof and its disqualification are the same fact. The market equilibrium of fig. 12 is thus not an accident of laziness: given the choice between an object about which something is proved and something else is visibly false, and an object about which nothing is proved and nothing is visibly false, every deployed system chose the second, with Rogaway's human-ignorance calculus (section 6.1) as the honest license. The gap is not a hole in the market; it is the market.

Project 11: Cayley hashes by hand

Implement the Tillich–Zémor hash (Tillich and Zémor, 1994) at toy scale: $\mathrm{SL}_2(\mathbb{F}_{2^n})$ for $n = 8, \ldots, 16$, two generator matrices, hash = matrix product read off the message bits. (i) Measure output equidistribution against message length and match the exponential rate to the spectral gap of the Cayley graph, computable by eigenvalue methods at this scale (Hoory et al., 2006)—the mixing theory of section 6.6 as a bench experiment. (ii) Hunt collisions two ways, generic birthday search versus structure (palindromic words, elements of small order, subfield membership), and compare costs. (iii) As a stretch goal, read the lifting attack (Tillich and Zémor, 2008) and implement its Euclidean core for tiny parameters. Deliverable: a working Cayley hash, gap-versus-equidistribution plots, and a collision gallery—provable cryptography's rise and fall on one laptop.

Prerequisites: group theory; linear algebra; programming *Window:* 3–4 weeks

7 The adversary's calculus: differential cryptanalysis

Section 6.4 reported the adversarial state of the art as bare round counts. This section opens the instrument that produces those numbers. Differential cryptanalysis is the only mathematical technology that has ever killed a member of this design family—it felled MD5, SHA-0, and SHA-1 (section 2.3)—and its core idea is elementary enough to state in one line: *since the function cannot be understood, study its finite differences instead*. What follows is the calculus itself, its startling secret history, the story of its mechanization, and an honest account of where, on SHA-256, it is currently stuck. Readers who absorb this section will understand precisely what the "round counts" of section 6.4 measure.

7.1 A calculus of differences

For a map F on bit strings and a fixed *input difference* Δ, consider the discrete derivative

$$D_\Delta F(x) \;=\; F(x \oplus \Delta) \oplus F(x),$$

the exact finite-difference analogue of differentiation, with $\oplus$ playing subtraction in the $\mathbb{F}_2$-geometry of section 3.2. The adversary's question is distributional: for random x, what does $D_\Delta F(x)$ look like?

Two observations power everything. First, *differencing kills constants and linear structure*. If L is $\mathbb{F}_2$-linear—a rotation, a shift, one of the Σ/σ maps of section 3.3—then

$$D_\Delta L(x) \;=\; L(\Delta) \qquad \text{for every } x :$$

the derivative is a *constant*, independent of the unknown input, exactly as $\mathrm{d}/\mathrm{d}x$ annihilates constants and linearizes. All the values the attacker does not know—the other message words, the chaining state, the round constants K_t—drop out of the linear part of every round. This is why one differentiates: a difference propagates through the transparent parts of the circuit deterministically, leaving a residue of uncertainty only at the nonlinear gates.

Second, at those nonlinear gates—the modular additions $\boxplus$ and the quadratic functions $\mathrm{Ch}, \mathrm{Maj}$—a difference does not propagate deterministically but *branches into a probability distribution over output differences*. The fundamental case is a single addition (fig. 13). Give one addend a one-bit difference at position i and add the same unknown y to both members of the pair: the two sums differ at bit i certainly, but whether the difference stops there is decided by the *carries*, which the $\mathbb{F}_2$-lens cannot see. If the carries out of position i agree—this happens for exactly half of all (x, y)—the output difference is again the single bit e_i; otherwise a carry-difference runs leftward, one further bit at a time, each extra bit halving the probability. One position is exceptional: a difference in the top bit ($i = 31$) propagates with probability 1, because the disagreeing carry falls off the end of the word and is discarded by the reduction mod 2^{32}. Attackers therefore park differences in the most significant bit whenever they can—a free move in an economy where everything else costs. The complete theory of differential propagation through addition (arbitrary difference patterns, exact probabilities, all computable in linear time) was worked out by Lipmaa and Moriai (2002), and its headline is a pleasing cost principle: *the probability of the straight-through transition is* $2^{-(\textit{number of active bits, msb excluded})}$. To first order, difference weight is money, and the exchange rate is a factor of two per active bit. The bitwise functions obey the same economy: at each bit where its inputs differ, Ch or Maj passes or absorbs the difference depending on the values of the other, unknown operands—a coin flip per active bit, one more factor of $\frac{1}{2}$ each.

Notice what has happened in the language of section 3.2: the attacker has *chosen the $\mathbb{F}_2$-geometry* and agreed to pay, in probability, at every point where the design invokes the other one. All of the analytical cost is concentrated exactly at the carry chains and quadratic gates—the terms $\mathbb{F}_2$-linear algebra cannot see. There is a dual calculus using arithmetic differences $x' - x \bmod 2^{32}$, which tames the additions and instead pays at the xors and rotations; the craft of the great hash attacks (section 7.2) was, in large part, the discovery that one should track *both* lenses simultaneously, bit by bit, in the form of signed differences. The two-geometry tension that section 3.2 presented as the design's defense is thus also, precisely, the structure of the attack: differential cryptanalysis is what the two-lens problem looks like when an adversary refuses to give up.

7.2 Trails, tolls, and free rounds

A single round is only the first step. A *differential characteristic* (or *trail*) prescribes a difference for every intermediate quantity through all the rounds—a full flight plan for the perturbation—and, under a heuristic independence assumption, its probability is the *product* of the per-gate transition probabilities along the way. An attacker who wants a collision needs a trail beginning at a nonzero message difference and ending, after the feedforward, at state difference *zero*; if

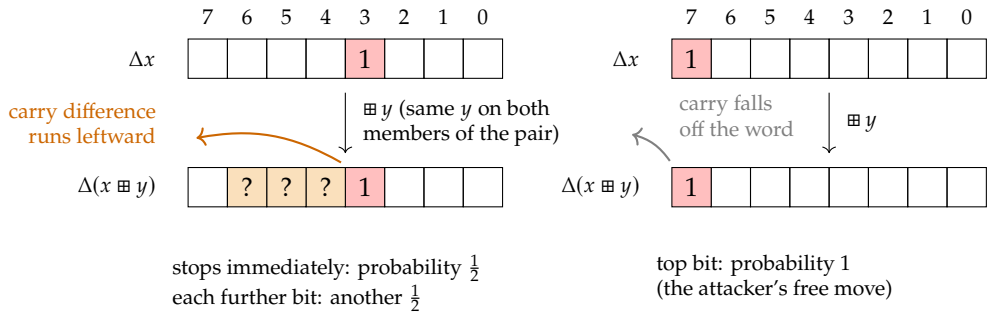


Figure 13: How a one-bit difference passes through modular addition (bit 7 = most significant, in this 8-bit cartoon of a 32-bit word). *Left:* an interior difference survives unchanged only if the carries agree—probability $\frac{1}{2}$—and every extra bit of carry-difference costs another factor $\frac{1}{2}$—the Lipmaa–Moriai cost formula of the text. *Right:* a difference in the top bit always passes intact, since the disagreeing carry is discarded mod 2^{32}. Differential cryptanalysis views the whole of SHA-256 through this lens: linear pieces are free wires; every active bit at a nonlinear gate levies a toll of $\frac{1}{2}$.

the trail has probability p, then about $1/p$ random conforming attempts are expected before one message pair follows the entire flight plan and the hashes agree. The attack economy is thus brutally simple: *total toll* = sum, over all rounds, of active bits at nonlinear gates; *cost* $\approx 2^{\text{toll}}$; the design's job is to force every possible flight plan to accumulate an unpayable toll.

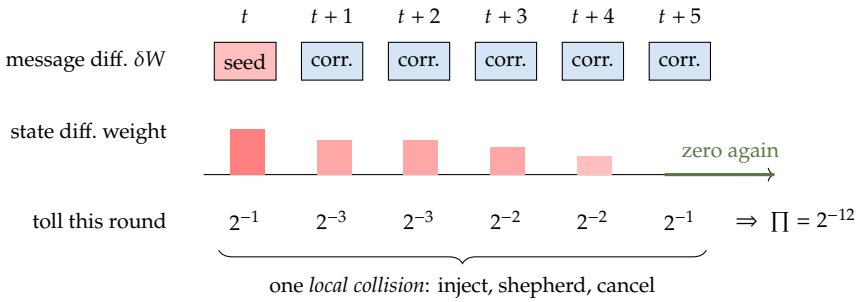


Figure 14: Cartoon of a *local collision*, the modular building block of every attack on this family since Chabaud and Joux's 1998 analysis of SHA-0 (cited in the text): a seed difference is injected through one message word, then five successive message-word corrections shepherd and finally cancel the disturbance, leaving the state clean. Each round levies its toll (fig. 13); the trail's probability is the product. A full attack chains many local collisions along a pattern compatible with the message schedule—and the toll of the best full trail is what decides whether the design lives or dies.

Three refinements turn this arithmetic into the attacks of section 2.3, and all three are conceptually simple. First, *local collisions* (fig. 14): rather than one monolithic trail, the attacker builds a short pattern—inject a difference through one message word, cancel it a few rounds later through others—and then chains copies of the pattern wherever the message schedule permits (Chabaud and Joux, 1998). Second, *message freedom*: for the first sixteen rounds the words W_t are the message itself (section 3.3), so the attacker does not *hope* the early tolls are

paid—she *solves* for message bits that pay them deterministically. Wang's celebrated breaks of MD5 and SHA-1 (Wang and Yu, 2005; Wang et al., 2005) rested on pushing this "message modification" astonishingly deep; the practical effect is that the toll meter starts running only around round 17, and the first quarter of the function contributes nothing to its own defense. Third, *automation*: what Wang's school did by virtuosic hand calculation—juggling xor- and arithmetic-differences as signed, bit-level conditions—was recast by De Cannière and Rechberger (2006) as a constraint-propagation search over "generalized conditions," mechanically discovering trails no human would find. The current records on SHA-256 are held by exactly this lineage of tools, now built on MILP, SAT, and SMT solvers (Mendel et al., 2013; Li et al., 2024), closing the loop with section 8.1: the adversary's calculus has itself become a branch of solver engineering.

7.3 A method twenty years secret

The published history begins in 1990, when Biham and Shamir announced differential cryptanalysis and used it to attack reduced variants of DES, the American encryption standard of 1977 (Biham and Shamir, 1991). A curiosity surfaced immediately: DES, designed in the early 1970s, was *strangely well prepared* for the new attack. Its nonlinear tables were, component for component, close to optimally resistant; the method that broke every academic variant of DES bounced off the real thing.

The explanation arrived in 1994, and it is one of the most remarkable documents in the literature. Coppersmith, of the original IBM design team, was cleared to reveal the design criteria and wrote them down plainly: the IBM team had discovered differential cryptanalysis themselves—internally, the "T-attack"—*in 1974*, and hardened DES against it before standardization; the NSA, consulted on the design, asked that the rationale be kept secret, since publishing the criteria "would reveal the technique of differential cryptanalysis, a powerful technique that can be used against many ciphers" (Coppersmith, 1994). The central tool of public cryptanalysis was thus twenty years old on the day it was discovered. For sixteen of those years, every open researcher who studied DES was probing defenses calibrated against a weapon whose existence they did not suspect.

This is not merely a good story; it is a load-bearing datum for the epistemology of section 2.4, because the pattern *repeats within our own object's lineage*. Recall the footnote of section 2: in 1994–95 the NSA withdrew SHA-0 and changed a single rotation, citing an undisclosed weakness; the public rediscovery of what that tweak defends against—a differential collision attack—took until 1998 (Chabaud and Joux, 1998). Twice measured, the gap between the classified and the public understanding of this exact attack has been twenty years and four years. The honest inference for a naturalist: the map of section 6.4 is the *public* frontier, a lower bound on the true state of knowledge, drawn by a community that has historically been behind by years to decades—and SHA-256's design parameters were set by the party that was ahead. The comfort and the discomfort of that fact are left to the reader to apportion.

7.4 The siege of SHA-256, and where it stands

Why did the calculus shatter MD5 and SHA-1 while SHA-256, made of the same primitives, holds? The anatomy of section 3.3 contains the visible answers. SHA-1's message schedule is entirely $\mathbb{F}_2$-linear, so schedule differences propagate by exact linear algebra and *sparse*

schedules—low total weight, hence low toll—can be found by searching a linear code for low-weight words; this is precisely the handle the collision attacks gripped (section 3.7). SHA-256 interleaves modular additions into the schedule itself and taxes every round twice over: each of Σ_0, Σ_1 fans any single active bit into three, both tracks of the two-track recurrence pass through a quadratic gate every round, and the σ-mixing makes a sparse schedule difference bloom within a few steps. The result, empirically: low-toll trails simply stop existing a dozen-odd rounds past the free zone, and two decades of search—human, then mechanical—have moved the frontier only from about 19 rounds (2006) to 31 rounds (collisions) today. The precise current records: practical colliding pairs for 28 rounds and a $2^{65.5}$ collision attack on 31 rounds of the 64, both from the automated-search school (Mendel et al., 2013); practical *semi-free-start* collisions (the attacker may choose the chaining value, removing the Merkle–Damgård anchor of theorem 3.1) for 38 rounds in 2013 and 39 rounds in 2024, the latter found with a SAT/SMT trail search (Li et al., 2024; Alamgir et al., 2024); and, on the inversion side, preimages to 45 rounds at a cost indistinguishable from brute force (Khovratovich et al., 2012). Li, Liu, and Wang open their record-setting 2024 paper with the flat sentence that "there is almost no progress in collision attacks on SHA-2 after ASIACRYPT 2015"—a decade of solver revolutions purchasing one round.

The shape of the stall is worth internalizing, because it is the quantitative content of the phrase "security margin." Rounds 1–16 are free (message freedom); rounds 17 to roughly 40 are the contested zone, where automated searches still find trails but each round multiplies the toll; beyond that, *no usable trail has ever been exhibited*, not because a theorem forbids one but because every known search method drowns. The design's 64 rounds thus stand at about twice the depth the best siege engines have ever reached with unlimited choice of starting position. Whether that factor of two is a fortress wall or a picket fence is exactly the question section 2.3 showed nobody can answer from first principles: no theory predicts the frontier's future velocity, and the SHA-1 precedent—from academic 2^{69} to industrial collision in twelve years—is the reason nobody treats the margin as a moat. (One further credential: the compression function's core, extracted and run as a block cipher under the name SHACAL-2, survived the open European NESSIE evaluation and was selected for its 2003 portfolio—the same engine vetted from a second angle, as a cipher rather than a hash.)

For this paper's program the moral is double. The adversary's calculus is the most refined body of exact knowledge about *reduced* members of the family—every trail is a theorem-with-certificate about a specific truncation, machine-checkable pair in hand—and at the same time it is purely *local* knowledge: a trail certifies one orbit of one difference, and says nothing global about the map. The statistical coordinates of section 4 and the trail calculus of this section are complementary instruments, and to our knowledge no one has systematically pointed them at the same scaled-down family to see whether the round where trails die is the round where the Flajolet silhouette locks in (section 9).

Project 12: Differential trails in miniature

Build the toll calculus for a toy SHA-256 on 8-bit words. First verify fig. 13 empirically: tabulate the exact distribution of $\Delta(x \boxplus y)$ over all pairs for every one-bit Δx, and check the Lipmaa–Moriai cost formula (Lipmaa and Moriai, 2002). Then implement a branch-and-bound search for the lowest-toll trail through r rounds of the toy compression function and plot $\log_2$(best trail probability) against r. Two landmarks make the plot speak: the

birthday line ($-n/2$ for an n-bit toy state), below which a trail is useless to an attacker, and the round at which your search itself starts to drown. You will have reproduced, at fruit-fly scale, both the designer's round-count decision and the attacker's wall—the two sides of section 7.4—and the same code instruments the comparative questions of section 9.

Prerequisites: probability; programming; patience with bit manipulation *Window:* 4–6 weeks

7.5 Coda: can a machine learn the difference?

One newer thread deserves flagging, because it closes a loop with the proof-barrier of section 8.4 in a way that is either profound or a curiosity, and nobody yet knows which. In 2019 Gohr trained a neural network to distinguish the outputs of a round-reduced ARX cipher (Speck, a near-relative of SHA-256's primitives) from random, and it *beat* the best hand-built differential distinguishers—and, on inspection, had learned features *beyond* the difference distribution table, correlations the human theory of section 7.1 does not track (Gohr, 2019). Read against Razborov–Rudich (section 8.4), this is quietly startling. A trained distinguisher *is* a natural property in their precise sense—an efficiently computable, large predicate that separates the function from random—so a machine that reliably learns such properties on SHA-256-family maps would be walking, empirically, along the exact edge the natural-proofs barrier says cannot be crossed at full scale. Whether the network finds real, nameable structure (extractable back into human mathematics, in the spirit of section 3.7) or merely a more efficient encoding of the same statistical residue is, at present, an open and very testable question—and a natural apex to the undergraduate program, where a student can point a modern learning tool at a scaled-down family and see, concretely, what a machine can and cannot pull out of manufactured randomness.

Project 13: A neural distinguisher at toy scale

Reproduce Gohr's experiment (Gohr, 2019) on a SHA-256-family toy: train a small network to distinguish r-round outputs (or output differences) from random, and chart its advantage against r alongside the trail-toll curve of the miniature-trails project above and the statistical-battery frontier of section 6.7. Then ask the hard question: *interpret* the trained model—does its decision reduce to a differential-distribution feature the human theory already knows (section 7.1), or to something new? A negative result is a clean measurement; a positive one is the first thread of a natural property in a real hash family, and worth far more than a course grade.

Prerequisites: programming; a machine-learning course *Window:* 6–8 weeks (flagship)

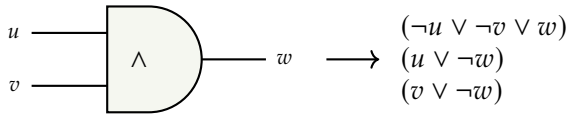
8 The view from computability

The gap of section 6.7 is a statement about missing *positive* knowledge. It is worth closing the overview by asking the complementary, sharper question: what is *provably* true about the difficulty of breaking SHA-256? Here the news is a genuine mix of hard theorems and hard barriers, and the boundary between them is itself part of the object's significance.

8.1 Breaking SHA-256 is an NP search problem

Fix a target digest y and ask for a preimage: some x with $\mathsf{SHA\text{-}256}(x) = y$. Because SHA-256 is computable by a small circuit (section 2), a candidate x can be *checked* in linear time, so preimage-finding is an NP search problem in the exact sense of Cook and Levin (Cook, 1971): the "inversion" relation is polynomial-time decidable, and the only question is the cost of *search*. Encode one evaluation of the compression function as a Boolean circuit—about 2^{16} gates—and the equation $\mathsf{SHA\text{-}256}(x) = y$ becomes a concrete instance of the satisfiability problem SAT, the canonical NP-complete language.

The translation is worth seeing once, because it is almost embarrassingly direct (fig. 15). Give every wire of the circuit its own Boolean variable; each gate then constrains its three wires by a handful of clauses, satisfied exactly by the gate's truth table; pin the 256 output variables to the bits of y. The conjunction of all clauses is satisfiable *if and only if* a preimage exists, and any satisfying assignment, read off the input wires, *is* a preimage. The whole of SHA-256's round structure—Σ's, carries, message schedule—flattens into a few hundred thousand such clauses, syntactically indistinguishable from any other SAT instance. A modern conflict-driven ("CDCL") solver then does something almost mathematical with it: it explores partial assignments, and each dead end is analyzed into a *learned clause*—a small lemma, added to the formula, recording why that region of the search space is empty. A solver run is a proof-by-cases of astronomical width, conducted by a machine that invents its own case analysis.



one fresh variable per wire, a few clauses per gate;
$\sim 2^{16}$ gates $\Rightarrow$ a few $\times 10^5$ clauses; pin 256 output literals to y and solve.

Figure 15: The reduction that makes "invert SHA-256" a SAT instance: each gate becomes clauses that are satisfied exactly by its truth table (shown: $w = u \wedge v$). Nothing about the construction is clever, and that is the point—membership in NP is a statement about *checkability*, and it holds for SHA-256 as mechanically as for any puzzle whose solutions can be verified.

This is not a metaphor: the reduction is run in practice, and by now the solvers are genuine instruments of record. As early as 2006, Mironov and Zhang reproduced Wang-style MD5 collisions with an off-the-shelf SAT solver, no differential expertise wired in—the solver rediscovering, clause by learned clause, constraints it had never been told (Mironov and Zhang, 2006). Today the traffic runs both directions: the current record semi-free-start collision on 39-round SHA-256 was found with a SAT/SMT search for the differential trails of section 7.2 (Li et al., 2024), and a programmatic SAT solver—one that injects cryptanalytic knowledge as it runs—produced the first practical 38-round semi-free-start collision independently of the classical toolchain (Alamgir et al., 2024). The empirical shape of all this effort is a cliff: instances for few rounds collapse in milliseconds, then somewhere past round 20 the running times turn vertical, and no amount of solver engineering has moved the wall more than a round or two per decade. The cliff's very existence is one more coordinate in the natural history of the function—and, unlike most such coordinates, it is cheap to measure.

Project 14: The SAT cliff

Build (or generate with existing tools) CNF encodings of r-round SHA-256 preimage instances (fig. 15) and feed them to two or three modern solvers. Chart solving time against r until timeout; then vary what is pinned (full digest, partial digest, collision instead of preimage) and watch the cliff move. Plot, on the same axis, where the statistical batteries start failing (section 6.7's project) and where the differential trails of section 7.4 die: three instruments' frontiers, never to our knowledge drawn on one chart. The deliverable is that chart—a measured portrait of "hardness turning on" round by round.

Prerequisites: programming; propositional logic *Window:* 4–6 weeks

But note precisely what NP-completeness does and does not say. It says the *worst case* of SAT is hard unless P = NP; it says almost nothing about the specific, highly structured instances that hashing produces. Cryptography needs *average-case* and *one-way* hardness—hard instances that are easy to *generate* with a solution—and this is a strictly stronger and more delicate demand than worst-case NP-hardness. No reduction shows that inverting SHA-256 is NP-hard, and this is not merely an unfilled cell in the table: there is a theorem-shaped reason to expect no such reduction. Akavia, Goldreich, Goldwasser, and Moshkovitz showed that a black-box reduction (of the standard, non-adaptive kind) from an NP-hard problem to *inverting a one-way function* would itself trigger complexity-theoretic collapses nobody believes (Akavia et al., 2006). The missing bridge between P ≠ NP and SHA-256's security is not awaiting a clever graduate student; by the best current evidence, the standard bridge-building technology provably cannot span that river.

8.2 Five worlds, one function

The distinction just drawn—worst-case hardness versus generatable hardness—organizes the deepest open question in the area, and Impagliazzo gave it an unforgettable form: five possible worlds, exactly one of which is real (Impagliazzo, 1995). Figure 16 draws the ladder and scores our function in each world. The point of the exercise for this paper: *whether an object like SHA-256 can exist at all* is not a detail of cryptography but the exact content of the rung between Pessiland and Minicrypt—the existence of one-way functions—and it is wide open. Everything this paper treats as mysterious about one specific function is, one level up, the mystery on which the classification of computational reality turns.

Two theorems furnish that observation with content, and a third—the one this section builds toward—turns it into an exact equivalence.

First: *Minicrypt has a canonical witness.* Levin constructed an explicit, fully specified "universal one-way function"—a concrete program one can write down today—which is one-way *if any one-way function exists at all* (the tale, and the construction, are told in Levin, 2003). Existence of the whole kingdom is thus equivalent to the hardness of one specific function, in print since the 1980s. Why, then, does anyone bother with SHA-256? Because Levin's function is a diagonalization—roughly, it runs all short programs and combines their outputs—and its guarantee is bought at constants and exponents that make it a philosophical instrument, not an engineering material. Once any one-way function exists, moreover, the whole private-key toolbox follows by explicit constructions: Håstad, Impagliazzo, Levin, and Luby showed pseudorandom generators can be built from *any* one-way function (Håstad et al.,

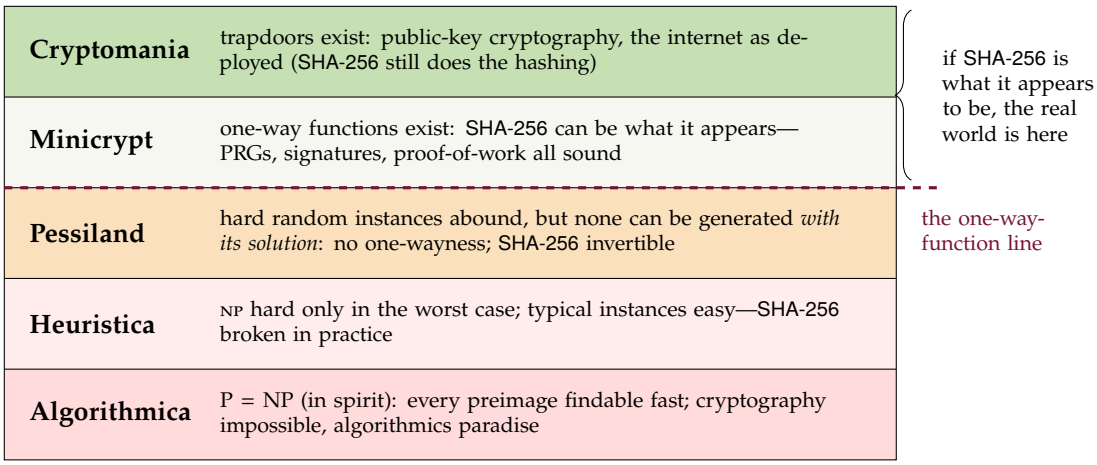


Figure 16: Impagliazzo's five worlds: mutually exclusive possibilities for how computation works, exactly one of them actual, none of them ruled out. SHA-256's conjectured properties assert that we live above the one-way-function line (at least Minicrypt)—a claim strictly *weaker* than what public-key cryptography bets on, and strictly *stronger* than P ≠ NP. Note where the quantum threat falls: Shor-type algorithms attack the classical inhabitants of Cryptomania, while against the generic hashing of Minicrypt the provable quantum gain is only the square root of section 8.4.

1999), with signatures and message authentication downstream. Read against section 6.7: theory already possesses, with proofs, the entire architecture the world needs—resting on an unproven ground floor and running too slowly to use. SHA-256 is the engineering shadow of Levin's function: the object you deploy when you want the universal guarantee at gigabytes per second, and are willing to trade the theorem away for it.

Second, the same result read as a boundary: the Akavia–Goldreich–Goldwasser–Moshkovitz theorem above says the ground floor cannot be poured from worst-case NP-hardness by standard reductions. The two faces together frame the epistemic situation exactly: everything above the one-way-function line is solved mathematics; the line itself is untethered to the rest of complexity theory.

8.3 An exact address for the one-way line

The third result is recent (2020) and, to our taste, the most beautiful in the whole subject, because it identifies the one-way-function line of fig. 16 with the hardness of a problem this paper has been circling from the start: *detecting structure in apparently random strings*. Making that precise takes four definitions, each elementary; we give them carefully, because the payoff is a genuine "if and only if" and it is worth being able to read it exactly.

Kolmogorov complexity. Fix once and for all a universal Turing machine U (a fixed interpreter: it runs a program Π on an input and prints the result). The *Kolmogorov complexity* of a string $x \in \{0,1\}^*$ is the length of the shortest program that outputs it,

$$K(x) \;=\; \min\{\, |\Pi| \;:\; U(\Pi) = x \,\}.$$

This is the honest measure of "information content": a patterned string like $(01)^n$ has a tiny generator ("print `01` n times"), so $K \approx \log_2 n$, while a structureless string has no description shorter than itself, $K(x) \approx |x|$. The trouble, for our purposes, is that K is *uncomputable*—pinning it down solves the halting problem—so it cannot be the hard problem cryptography stands on. Something must give the program a deadline.

Time-bounded Kolmogorov complexity. Fix a polynomial time bound $t(\cdot)$. The *t-time-bounded* complexity restricts the minimization to programs that print x *within* $t(|x|)$ *steps*:

$$K^t(x) \;=\; \min\big\{\, |\Pi| \;:\; U(\Pi) = x \text{ in at most } t(|x|) \text{ steps} \,\big\}.$$

Now the quantity *is* computable—enumerate all programs of length $\le |x|$, run each for $t(|x|)$ steps, keep the shortest that prints x—but that brute force costs about $2^{|x|}$. The entire question is whether one can do better than brute force, and specifically whether one can do better *on average*, over a random string x.

Why almost every string is incompressible. A counting fact fixes the landscape (fig. 17). There are only $2^{n-d} - 1 < 2^{n-d}$ programs shorter than $n - d$ bits, so at most a 2^{-d} fraction of the 2^n strings of length n can have $K^t(x) < n - d$. Compressible strings—those with real structure a fast program can exploit—are *exponentially rare*; the mass of K^t piles up just below the diagonal $K^t(x) = n$. In particular the *trivial estimate* "$K^t(x) = |x|$"—equivalently, the one-line algorithm that ignores x and outputs its length—is correct to within d bits on all but a 2^{-d} fraction of inputs. Estimating K^t better than this trivial baseline is precisely the act of *detecting the rare structure*, and that is the task whose difficulty the theorem measures.

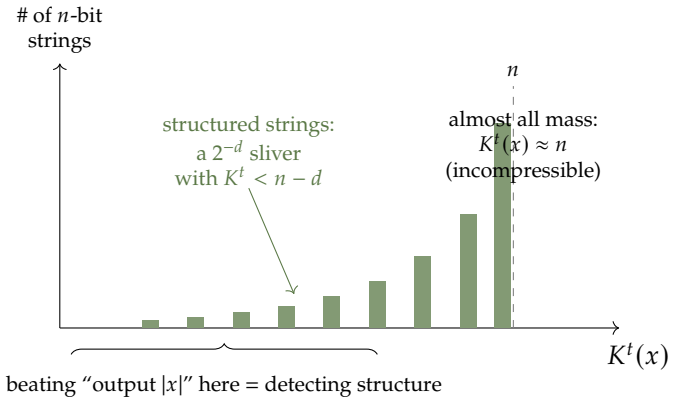


Figure 17: The distribution of time-bounded Kolmogorov complexity over random n-bit strings. By counting, at most a 2^{-d} fraction can be compressed below $n - d$; the histogram is crushed against the incompressible edge $K^t = n$. The one-line "output $|x|$" algorithm is therefore right for nearly everyone—and Liu–Pass say that noticeably out-performing it, on average, is exactly as hard as inverting every one-way function.

Mild average-case hardness, and one-wayness. Two more definitions, now the load-bearing ones. Say K^t is *mildly hard-on-average* if some polynomial p makes it uncomputable on a

noticeable slice: for every probabilistic polynomial-time algorithm A and all large n,

$$\Pr_{x \leftarrow \{0,1\}^n}\left[\, A(x) \neq K^t(x) \,\right] \;\geq\; \frac{1}{p(n)}.$$

The word to dwell on is *mild*: A is permitted to be right on a $1 - 1/p(n)$ super-majority of strings; hardness on just an inverse-polynomial sliver suffices. And a *one-way function*—the one-wayness informally invoked back in section 8.1—is a polynomial-time computable f that no probabilistic polynomial-time A can invert except with vanishing probability: for every such A, every polynomial p, and all large n,

$$\Pr_{x \leftarrow \{0,1\}^n}\left[\, f(A(f(x))) = f(x) \,\right] \;<\; \frac{1}{p(n)}.$$

Existence of any such f is the Minicrypt hypothesis—the rung of fig. 16 on which SHA-256's conjectured life depends.

Theorem 8.1 (Liu–Pass 2020)**.** *For every polynomial $t(n) \geq (1+\varepsilon)n$ with $\varepsilon > 0$ constant, the following are equivalent:*

1. *One-way functions exist (equivalently: secure private-key encryption, pseudorandom generators, pseudorandom functions, digital signatures, and commitments all exist).*
2. *K^t is mildly hard-on-average.*

Moreover the equivalence is robust to approximation: if one-way functions exist then no efficient algorithm can even $(d \log n)$-approximate K^t on meaningfully more than the trivial fraction, for any constant d (Liu and Pass, 2020).

The approximation clause is the sharpest way to feel the result. The do-nothing algorithm "output $|x|$" already $(d \log n)$-approximates K^t with probability $\geq 1 - n^{-d}$ (the counting fact again); the theorem says that beating even *that* baseline, by any inverse-polynomial margin, is as hard as breaking *every* one-way function at once. A hair of non-trivial structure-detection, made reliable, would collapse all of private-key cryptography.

Now read theorem 8.1 in this paper's language. "$K^t(x)$ is small" means *x has detectable structure*—a short program generates it fast. Estimating K^t on random inputs is therefore exactly *hunting for hidden structure in apparently random strings, on average*—the naturalist's question of section 2.4, promoted from one function to all strings. The theorem says this hunt is (mildly) hard *if and only if* objects like SHA-256 can exist at all. The connection is not a loose analogy; it is the same activity. A pseudorandom string—say a stretch of the counter stream SHA-256(1), SHA-256(2), ... of section 6.4—has genuinely *low* K^t (the short program "print SHA-256(1..m)" generates it quickly), yet is engineered to look maximally incompressible; a distinguisher that noticed the gap would be estimating K^t better than trivial on exactly these strings, which is to say breaking the generator. So the empirical program of section 6.7—point an instrument at a pseudorandom object and measure whether any structure pokes through—is not merely *analogous* to the problem on which the existence of cryptography rests. Up to the formalization, it *is* that problem, shrunk to a single function and a laptop. We know no cleaner statement of why the small experiments of section 9 sit on the main line of the subject rather than beside it.

8.4 Two-sided barriers: what is provably out of reach

The deepest results in this corner are *impossibility* theorems, and they cut in two directions at once.

On the side of *attacks*, the quantum model yields rare unconditional lower bounds. Grover search inverts any black-box function on N points in $\Theta(\sqrt{N})$ queries, and this is provably optimal—no quantum algorithm does better *relative to the oracle* (Bennett et al., 1997); for collisions, the Brassard–Høyer–Tapp birthday-style algorithm costs $\Theta(N^{1/3})$, and Aaronson and Shi proved a matching $\Omega(N^{1/3})$ quantum lower bound (Aaronson and Shi, 2004). These theorems bound what *any* algorithm can do against a function that behaves as a random oracle, so they translate into concrete, believed-tight security levels for SHA-256—128 quantum bits of preimage resistance, and the reason a 256-bit digest is considered "quantum-safe" for collision resistance. The catch, familiar by now, is the model: the bounds hold in the black-box world and become claims about SHA-256 only under the heuristic that no shortcut exploits its actual circuit.

It is worth pausing to give exponents like 2^{128} an empirical scale, because the planet happens to be running the relevant experiment. The proof-of-work industry of section 2.1 evaluates SHA-256 about 10^{21} times per second—roughly 2^{70}—and its cumulative total to date is on the order of 2^{96} evaluations: comfortably the largest computation of any kind in human history, every cycle of it hammering on this one function. Two readings. As a *statistical* instrument: block arrivals track the uniform model with textbook fidelity, so this is the most expensive test of the random-function conjecture ever conducted, passed to date. As a *yardstick*: the deepest partial inversion all that work has ever exhibited is a digest beginning with roughly a hundred zero bits—each mined block is a certified partial preimage, currently costing about 2^{79} evaluations apiece—which still leaves the full 256-bit inversion about 2^{156} times beyond the species' total output, and even the 2^{128} birthday bound a factor of four billion past everything computed so far. When section 4.3 says the global geometry of the functional graph is "operationally unreachable," this is the industrial base against which "unreachable" is calibrated.

On the side of *proofs*, the barrier theorems explain the gap of section 6.7 as something deeper than a lack of effort. The *natural proofs* barrier of Razborov and Rudich (Razborov and Rudich, 1997) shows that any proof of a strong circuit lower bound which is itself "constructive and large"—the form essentially all known lower-bound techniques take—would, ironically, yield an algorithm breaking the very pseudorandom functions such a lower bound is meant to certify. In slogan form: *the existence of objects like SHA-256 is precisely what prevents us from proving that objects like SHA-256 are hard* by current methods. This is why nobody expects an unconditional proof of SHA-256's security in the foreseeable future, and why the honest theorems in the subject are either conditional (section 6), relativized (the quantum bounds above), or about components rather than the whole (theorem 3.4).

8.5 What the proof assistants did prove

After a section of impossibilities, it is worth recording where machine-checked mathematics has *succeeded* with SHA-256—because it has, completely, one floor down. In 2015 Appel produced a formal, machine-checked proof, in the Coq proof assistant, that OpenSSL's C implementation of SHA-256 computes exactly the function specified in FIPS 180-4: every pointer offset, every rotation, every corner of C semantics, verified down to a mechanized model of the compiler's

view of the program (Appel, 2015). The HACL* project carried the method to an entire cryptographic library, written in the F* proof language and compiled to C; its verified SHA-256 ships inside Firefox, executing in ordinary users' browsers (Zinzindohoué et al., 2017). These are not toy exercises; they retire a whole species of real historical failure, the *implementation* bug—and they are proofs about SHA-256 in the strictest sense of the word.

The epistemological inversion deserves to be savored. *Everything provable about SHA-256 has now been proved by machines: namely, that the program is the function.* What no one can prove is any property of the function itself. Formal verification thus sharpens the boundary of section 6.7 into a visible line: below it, mechanized certainty—code equals specification, checked symbol by symbol; above it, Rogaway's human ignorance (section 6.1), trust underwritten by two decades of failed attack. The axiom has quietly moved. It is no longer "this function is one-way"; it is "this function is the one we wrote down"—and only the second statement is checkable. A mathematician could not ask for a cleaner exhibit of where, exactly, in one industrial object, the proved ends and the conjectured begins.

8.6 Where this leaves the program

The computability view sharpens, rather than dampens, the case for the empirical program of this paper. Unconditional hardness is provably out of reach by known methods; worst-case NP-hardness is the wrong target, and provably unavailable by standard reductions (section 8.2); the clean theorems that do exist are relativized bounds assuming random-oracle behavior, or certify the code rather than the function (section 8.5). What is *not* ruled out—by any barrier theorem—is the discovery of exact, non-statistical structure in specific reduced systems, of the kind section 3.7 catalogued next door and theorem 3.4 exhibited inside the function itself. The barriers forbid a certain style of *general* proof; they say nothing against a Josephus-style collapse of a *particular* finite object. That is exactly the crack the undergraduate program of section 9 proposes to work in: not to prove SHA-256 hard—provably hopeless—but to map, instance by instance and coordinate by coordinate, where the structure ends and the noise begins.

9 Undergraduate short projects

The preceding sections flagged thirteen concrete projects at the point where each becomes natural; here we collect them and add the program-level view. The unifying activity is *measurement*: locating scaled-down members of the SHA-256 design family in coordinate systems (Flajolet's above all) where "looks random" becomes a number with error bars. None of the projects requires cryptographic background beyond this paper; all require programming and a taste for experiment; each produces figures and data that are a contribution regardless of what they show. That last property is worth underlining for research with undergraduates in short windows: these are experiments that *cannot fail*, only surprise.

project (defined in)	prerequisites	window	deliverable
Single ops under the 2-adic lens (section 3.2)	elementary number theory	3–4 wks	verified ergodicity criteria, orbit visualizations
The linear algebra of diffusion (section 3.5)	linear algebra	3–4 wks	invertibility & group structure of the Σ/σ maps
Exact inverses of Σ_0, Σ_1 (section 3.5)	polynomials over $\mathbb{F}_2$	2–3 wks	closed-form inverse maps, verified two ways
Functional-graph atlas of truncated SHA-256 (section 4.3)	probability, programming	6–8 wks	atlas of measured vs. predicted statistics
Collision hunting, van Oorschot–Wiener style (section 4.3)	probability, programming	4–6 wks	timing distributions vs. theory; parallel implementation
The dynamics of iteration (section 5)	probability, programming	6–8 wks	image-shrinkage law, fixed points, Hellman trade-off
Cutoff for toy compression walks (section 6.6)	probability, linear algebra	4–6 wks	spectral gaps and cutoff profiles vs. rounds
Cayley hashes by hand (section 6.8)	group theory, linear algebra	3–4 wks	spectral gap vs. equidistribution; collision gallery
Differential trails in miniature (section 7.4)	probability, programming	4–6 wks	exact toll tables; best-trail probability vs. rounds
The SAT cliff (section 8.1)	programming, logic	4–6 wks	solver-time cliff chart; three frontiers on one axis
A neural distinguisher at toy scale (section 7.5)	programming, machine learning	6–8 wks	learned-advantage vs. rounds; interpretation of the model
Where do the tests start failing? (section 6.7)	statistics, programming	4–6 wks	round-by-round battery failure chart
Avalanche cartography (section 6.7)	linear algebra, programming	4–6 wks	influence-matrix heatmaps by round

Three remarks on the program as a whole.

Model organisms. The full-scale function is beyond experiment (section 4.3) and beyond proof (section 6.7); the program therefore studies the *family* obtained by shrinking each design parameter—word size 32 → 8 or 16, state words 8 → 4, rounds 64 → r—the way biology studies fruit flies. The two-geometry anatomy of section 3.2 says which shrinkings are honest: one must preserve the ARX alternation, or the object degenerates into something a single lens solves. A systematic scaled family, with its statistics measured, would be a reusable community resource; no standard one exists.

The comparative method. Every measurement gains meaning from controls on both sides: genuinely random maps (the Flajolet silhouette) on one side, and visibly structured maps (bijections, $\mathbb{F}_2$-linear maps, one-round toys) on the other. The interesting science is in the *trajectory*—how, as rounds accumulate, a SHA-256-family map walks from the structured region onto the random silhouette, in which coordinates it arrives first, and whether the walk is

gradual or a phase transition. SHA-1 and SHA-256 can be given the identical treatment; given section 2.3, any measurable divergence between their trajectories would be a finding of genuine interest.

Odds of surprise. Realistically: most measurements will confirm the random silhouette, and their value is the atlas itself plus trained students. But the history of section 1 cuts both ways—structure, when it exists in innocent-looking discrete processes, has repeatedly been found by exactly this kind of patient small-case empiricism, and essentially nobody has pointed the analytic-combinatorics instrument at this family at model-organism scale. The expected value of looking is small-positive; the tail is long.

On tooling. Everything above runs comfortably in Python or Julia on a laptop for $n \leq 30$; the collision-hunting and atlas projects scale gracefully to whatever cluster time is available. We maintain reference implementations and are happy to share scaffolding so that student time goes to science rather than plumbing.

A Reference implementation: inverting Σ_0 and Σ_1

The listing below (`tools/invert_sigma.py` in the distribution accompanying this document) is the computational companion to section 3.5. It is included in full for three reasons.

First, it discharges a proof obligation. Theorem 3.4 certifies that Σ_0 and Σ_1 are bijections but exhibits nothing; eq. (2) exhibits the inverses but asks to be believed. The script closes the loop: it constructs each map's 32×32 matrix over $\mathbb{F}_2$, inverts it by Gaussian elimination, reads the inverse off as an xor of rotations (the inverse of a circulant map is circulant, so the image of the basis word 1 determines everything), and then verifies—functionally, against the standard's own definition (National Institute of Standards and Technology, 2015) of ROTR—that the seventeen-term maps of eq. (2) invert Σ_0 and Σ_1 on thousands of words, along with the order facts $\Sigma_0^8 = \mathrm{ROTR}^8$, $\Sigma_0^{32} = \Sigma_1^{16} = \mathrm{id}$, and $\Sigma_i^{-1} = \Sigma_i^{\mathrm{ord}-1}$ from section 3.5. A skeptical reader can rerun the whole certificate in under a second.

Second, the verification is *convention-proof*, and seeing why is a small lesson in the algebra of section 3.5. Any claim phrased in the polynomial ring R silently commits to a bit-numbering convention (is bit i the coefficient of x^i or of x^{31-i}? is rotation multiplication by x^r or x^{-r}?), and such commitments are exactly where off-by-one and mirror-image errors breed. But the composition law $\mathrm{ROTR}^a \circ \mathrm{ROTR}^b = \mathrm{ROTR}^{a+b \bmod 32}$ holds under every convention, so the statement "the rotation multiset $S_0 + \{2, 13, 22\}$ mod 32 covers 0 an odd number of times and every other offset an even number of times" – which is what eq. (2) asserts – is convention-free, and the script checks it against the operational definition rather than any ring identification. This is the paper's epistemological theme (section 2.4) in miniature: replace an argument one must trust with an artifact anyone can audit.

Third, it is deliberately written as scaffolding. The two project boxes of section 3.5 ask for the same computation done other ways—extended Euclid in $\mathbb{F}_2[x]$, repeated Frobenius squaring—and for the σ_0, σ_1 question that eq. (2) deliberately leaves open; the functions here (`matrix_of`, `invert_matrix`) apply verbatim to the shift-containing maps, and changing the single constant `W` repeats every experiment at any word length, where Rivest's gcd condition (Rivest, 2011) comes alive (try `W = 33`).

```python
"""Exact inverses of SHA-256's diffusion maps Sigma_0 and Sigma_1.

Companion script to "SHA-256 as a Mathematical Object" (appendix).
Words are 32-bit integers; bit i is the coefficient of x^i, so that
ROTR^r is an F_2-linear map of the word space F_2^32.

The script (1) builds each map's 32x32 matrix over F_2, (2) inverts it
by Gaussian elimination, (3) reads the inverse off as an xor of
rotations -- the inverse of a circulant is circulant -- and (4) verifies
everything functionally on random words, including the order facts
Sigma_0^8 = ROTR^8 and Sigma_1^16 = id proved in the text.
"""
import random

W = 32
MASK = (1 << W) - 1


def rotr(x, r):
    r %= W
    return ((x >> r) | (x << (W - r))) & MASK


def Sigma0(x):
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)


def Sigma1(x):
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)


def matrix_of(f):
    """Column i (as a bitmask of rows) is f applied to the basis word 2^i."""
    return [f(1 << i) for i in range(W)]


def invert_matrix(cols):
    """Invert an F_2 matrix given by column bitmasks; None if singular."""
    rows = []
    for r in range(W):  # rows of [M | I] as bitmasks over columns
        m = sum(((cols[c] >> r) & 1) << c for c in range(W))
        rows.append([m, 1 << r])
    piv = 0
    for c in range(W):
        sel = next((r for r in range(piv, W) if (rows[r][0] >> c) & 1), None)
        if sel is None:
            return None
        rows[piv], rows[sel] = rows[sel], rows[piv]
        for r in range(W):
            if r != piv and (rows[r][0] >> c) & 1:
                rows[r][0] ^= rows[piv][0]
                rows[r][1] ^= rows[piv][1]
        piv += 1
    return [sum(((rows[r][1] >> c) & 1) << r for r in range(W))
            for c in range(W)]


```

```python
def inverse_rotation_set(f):
    """Rotation offsets S with f^{-1} = XOR of ROTR^r over r in S."""
    inv = invert_matrix(matrix_of(f))
    if inv is None:
        raise ValueError("map is not invertible")
    y = inv[0]  # image of the basis word 1; ROTR^r(1) sets bit (32-r) % 32
    return sorted((32 - b) % 32 for b in range(W) if (y >> b) & 1)


def xor_of_rotations(S):
    def f(x):
        acc = 0
        for r in S:
            acc ^= rotr(x, r)
        return acc
    return f


def iterate(f, x, n):
    for _ in range(n):
        x = f(x)
    return x


if __name__ == "__main__":
    random.seed(20260716)
    words = [0, 1, MASK] + [random.getrandbits(W) for _ in range(5000)]
    for name, f, order in [("Sigma_0", Sigma0, 32), ("Sigma_1", Sigma1, 16)]:
        S = inverse_rotation_set(f)
        finv = xor_of_rotations(S)
        assert all(finv(f(x)) == x and f(finv(x)) == x for x in words)
        assert all(iterate(f, x, order) == x for x in words)          # f^order = id
        assert not all(iterate(f, x, order // 2) == x for x in words)  # exact order
        assert all(iterate(f, f(x), order - 1) == x for x in words)   # f^-1 = f^(order-1)
        print(f"{name}^-1 = XOR of ROTR^r, r in {S}  ({len(S)} terms, "
              f"order of {name} is {order})")
    assert all(iterate(Sigma0, x, 8) == rotr(x, 8) for x in words)     # Sigma_0^8 = ROTR^8
    print("all identities verified on", len(words), "words")
```

References

The references are grouped by theme rather than alphabetically *en masse*, to make the subject's connections visible. Green marks (►) link to archived copies in the `refs/` folder distributed with this document (see `refs/README.md` for provenance); works without a mark carry a note saying where to find them.

Patterns, exact solutions, and the algebra of iteration

Where seemingly procedural discrete dynamics collapsed to closed forms—the genre of sections 1, 3.5 and 3.7.

Vladimir Anashin. Ergodic transformations in the space of p-adic integers. In *p-adic Mathematical Physics: 2nd International Conference*, volume 826 of *AIP Conference Proceedings*, pages 3–24. American Institute of Physics, 2006. doi: 10.1063/1.2193107. arXiv preprint math/0602083 archived as ► `refs/anashin-2006-ergodic-transformations-padic.pdf`. *Cited in §3.2, §3.2, and §3.7.*

Charles L. Bouton. Nim, a game with a complete mathematical theory. *Annals of Mathematics*, 3(1/4):35–39, 1901. doi: 10.2307/1967631. Public domain; freely readable via JSTOR and Wikisource (not archived). *Cited in §1.*

Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, Reading, MA, second edition, 1994. ISBN 978-0-201-55802-9. No free copy exists; obtain via library. The Josephus problem opens §1.1. *Cited in §1.*

Alexander Klimov and Adi Shamir. A new class of invertible mappings. In *Cryptographic Hardware and Embedded Systems — CHES 2002*, volume 2523 of *Lecture Notes in Computer Science*, pages 470–483. Springer, 2003. doi: 10.1007/3-540-36400-5_34. PDF archived as ► `refs/klimov-shamir-2002-invertible-mappings.pdf`. *Cited in §3.2, §3.2, and §3.7.*

Jonathan Lubin. Nonarchimedean dynamical systems. *Compositio Mathematica*, 94(3):321–346, 1994. Numdam open-archive digitization archived as ► `refs/lubin-1994-nonarchimedean-dynamical-systems.pdf`. *Cited in §3.2.*

Olivier Martin, Andrew M. Odlyzko, and Stephen Wolfram. Algebraic properties of cellular automata. *Communications in Mathematical Physics*, 93(2):219–258, 1984. doi: 10.1007/BF01223745. Available locally as ► `refs/martin-odlyzko-wolfram-1984-algebraic-ca.pdf`. *Cited in §3.7.*

John Milnor. *Dynamics in One Complex Variable*, volume 160 of *Annals of Mathematics Studies*. Princeton University Press, Princeton, NJ, third edition, 2006. ISBN 978-0-691-12488-9. The author's 1990–91 lecture-notes version is free (arXiv:math/9201272); archived as ► `refs/milnor-1990-dynamics-one-complex-variable.pdf`. *Cited in §3.7.*

Ronald L. Rivest. The invertibility of the XOR of rotations of a binary word. *International Journal of Computer Mathematics*, 88(2):281–284, 2011. doi: 10.1080/00207161003596708. Author's prepublication version archived as ► `refs/rivest-2011-xor-rotations-invertibility.pdf`. *Cited in §3.5 and §A.*

Ernst Schröder. Ueber iterirte Functionen. *Mathematische Annalen*, 3(2):296–322, 1871. doi: 10.1007/BF01443992. Public domain; digitized scans freely available via the Göttingen Digitalisierungszentrum and EuDML (not archived). *Cited in §3.7.*

Klaus Sutner. Linear cellular automata and the garden-of-Eden. *The Mathematical Intelligencer*, 11(2):49–53, 1989. doi: 10.1007/BF03023823. No free copy found; obtain via library. *Cited in §3.7.*

Hashing before cryptography, and the birth of one-wayness

Scatter storage in early computing, and the adversary's gradual arrival (section 2.3).

Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi: 10.1109/TIT.1976.1055638. PDF archived as ▶ `refs/diffie-hellman-1976-new-directions.pdf`. *Cited in §2.3.*

Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, second edition, 1998. ISBN 978-0-201-89685-5. No free copy exists; obtain via library. §6.4 treats hashing, with its history. *Cited in §2.2.*

Ralph C. Merkle. *Secrecy, Authentication, and Public Key Systems*. PhD thesis, Stanford University, 1979. PDF archived as ▶ `refs/merkle-1979-thesis.pdf`. *Cited in §2.3.*

Robert Morris and Ken Thompson. Password security: A case history. *Communications of the ACM*, 22(11):594–597, 1979. doi: 10.1145/359168.359172. Open access in the ACM Digital Library backfile (automated download blocked; not archived). *Cited in §2.3.*

W. Wesley Peterson. Addressing for random-access storage. *IBM Journal of Research and Development*, 1(2):130–146, 1957. doi: 10.1147/rd.12.0130. No free copy found; obtain via library. *Cited in §2.2.*

George B. Purdy. A high security log-in procedure. *Communications of the ACM*, 17(8):442–445, 1974. doi: 10.1145/361082.361089. Open access in the ACM Digital Library backfile (automated download blocked; not archived). *Cited in §2.3.*

The SHA family: standards, design theory, breaks, hardware

The object itself: specifications, the Merkle–Damgård theorem, the fossil record of attacks, and silicon.

Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. ECRYPT Hash Workshop, 2007. URL `https://keccak.team/files/SpongeFunctions.pdf`. PDF archived as ▶ `refs/bertoni-daemen-peeters-vanassche-2007-sponge.pdf`. *Cited in §3.1 and §6.3.*

Eli Biham, Rafi Chen, Antoine Joux, Patrick Carribault, Christophe Lemuet, and William Jalby. Collisions of SHA-0 and reduced SHA-1. In *Advances in Cryptology — EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 36–57. Springer, 2005. doi: 10.1007/11426639_3. PDF archived as ▶ `refs/biham-chen-joux-2005-sha0-collisions.pdf`. *Cited in §1.*

John Black, Phillip Rogaway, and Thomas Shrimpton. Black-box analysis of the block-cipher-based hash-function constructions from PGV. In *Advances in Cryptology — CRYPTO 2002*, volume 2442 of *Lecture Notes in Computer Science*, pages 320–335. Springer, 2002. doi: 10.1007/3-540-45708-9_21. Authors' full version archived as ▶ `refs/black-rogaway-shrimpton-2002-pgv-analysis.pdf`. *Cited in §3.4.*

Florent Chabaud and Antoine Joux. Differential collisions in SHA-0. In *Advances in Cryptology — CRYPTO '98*, volume 1462 of *Lecture Notes in Computer Science*, pages 56–71. Springer, 1998. doi: 10.1007/BFb0055720. PDF archived as ▶ refs/chabaud-joux-1998-sha0-differential.pdf. *Cited in §1, §3.6, §7.2, and §7.3.*

Luigi Dadda, Marco Macchetti, and Jeff Owen. The design of a high speed ASIC unit for the hash function SHA-256 (384, 512). In *Proceedings of the Design, Automation and Test in Europe Conference (DATE 2004), Designers' Forum*, pages 70–75. IEEE, 2004. URL https://past.date-conference.com/proceedings-archive/2004/DATE04/DF_FILES/04D_5.PDF. PDF archived as ▶ refs/dadda-macchetti-owen-2004-sha256-asic.pdf. *Cited in §2.*

Ivan Damgård. A design principle for hash functions. In *Advances in Cryptology — CRYPTO '89*, volume 435 of *Lecture Notes in Computer Science*, pages 416–427. Springer, 1990. doi: 10.1007/0-387-34805-0_39. PDF archived as ▶ refs/damgard-1989-design-principle.pdf. *Cited in §3.1.*

Antoine Joux. Multicollisions in iterated hash functions. application to cascaded constructions. In *Advances in Cryptology — CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 306–316. Springer, 2004. doi: 10.1007/978-3-540-28628-8_19. PDF archived as ▶ refs/joux-2004-multicollisions.pdf. *Cited in §6.3 and §10.*

John Kelsey and Tadayoshi Kohno. Herding hash functions and the Nostradamus attack. In *Advances in Cryptology — EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 183–200. Springer, 2006. doi: 10.1007/11761679_12. Authors' version archived as ▶ refs/kelsey-kohno-2006-herding.pdf. *Cited in §6.3.*

John Kelsey and Bruce Schneier. Second preimages on n-bit hash functions for much less than 2^n work. In *Advances in Cryptology — EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 474–490. Springer, 2005. doi: 10.1007/11426639_28. Authors' version archived as ▶ refs/kelsey-schneier-2005-second-preimages.pdf. *Cited in §6.3.*

Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology — CRYPTO '89*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238. Springer, 1990a. doi: 10.1007/0-387-34805-0_21. PDF archived as ▶ refs/merkle-1989-certified-digital-signature.pdf. *Cited in §2.1.*

Ralph C. Merkle. One way hash functions and DES. In *Advances in Cryptology — CRYPTO '89*, volume 435 of *Lecture Notes in Computer Science*, pages 428–446. Springer, 1990b. doi: 10.1007/0-387-34805-0_40. PDF archived as ▶ refs/merkle-1989-one-way-hash-des.pdf. *Cited in §3.1.*

National Institute of Standards and Technology. Secure hash standard. Federal Information Processing Standards Publication 180-1, 1995. Supersedes FIPS 180 (1993). PDF archived as ▶ refs/nist-1995-fips-180-1.pdf. *Cited in §1.*

National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, 2015. PDF archived as ▶ refs/nist-2015-fips-180-4.pdf. *Cited in §1, §2, §3.3, §3.6, and §A.*

Bart Preneel, René Govaerts, and Joos Vandewalle. Hash functions based on block ciphers: A synthetic approach. In *Advances in Cryptology — CRYPTO '93*, volume 773 of *Lecture Notes in Computer Science*, pages 368–378. Springer, 1994. doi: 10.1007/3-540-48329-2_31. PDF archived as ▶ `refs/preneel-govaerts-vandewalle-1993-blockcipher-hash.pdf`. *Cited in §3.4.*

Ronald L. Rivest. The MD5 message-digest algorithm. RFC 1321, Internet Engineering Task Force, 1992. URL `https://www.rfc-editor.org/rfc/rfc1321`. PDF (rendered from the official text) archived as ▶ `refs/rivest-1992-rfc1321-md5.pdf`. *Cited in §2.3.*

Marc Stevens, Elie Bursztein, Pierre Karpman, Ange Albertini, and Yarik Markov. The first collision for full SHA-1. In *Advances in Cryptology — CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 570–596. Springer, 2017. doi: 10.1007/978-3-319-63688-7_19. PDF archived as ▶ `refs/stevens-2017-shattered-sha1.pdf`. *Cited in §2.3.*

Michael Bedford Taylor. The evolution of bitcoin hardware. *IEEE Computer*, 50(9):58–66, 2017. doi: 10.1109/MC.2017.3571056. Available locally as ▶ `refs/taylor-2017-bitcoin-hardware.pdf`. *Cited in §2.*

Xiaoyun Wang and Hongbo Yu. How to break MD5 and other hash functions. In *Advances in Cryptology — EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 19–35. Springer, 2005. doi: 10.1007/11426639_2. PDF archived as ▶ `refs/wang-yu-2005-break-md5.pdf`. *Cited in §2.3 and §7.2.*

Xiaoyun Wang, Yiqun Lisa Yin, and Hongbo Yu. Finding collisions in the full SHA-1. In *Advances in Cryptology — CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 17–36. Springer, 2005. doi: 10.1007/11535218_2. PDF archived as ▶ `refs/wang-yin-yu-2005-sha1-collisions.pdf`. *Cited in §2.3 and §7.2.*

A. F. Webster and Stafford E. Tavares. On the design of S-boxes. In *Advances in Cryptology — CRYPTO '85*, volume 218 of *Lecture Notes in Computer Science*, pages 523–534. Springer, 1986. doi: 10.1007/3-540-39799-X_41. Introduces the strict avalanche criterion. Available locally as ▶ `refs/webster-tavares-1985-avalanche.pdf`. *Cited in §6.4 and §6.7.*

Wikipedia contributors. SHA-2. Wikipedia, The Free Encyclopedia, 2026. URL `https://en.wikipedia.org/wiki/SHA-2`. Accessed July 16, 2026; snapshot archived as ▶ `refs/wikipedia-sha2-snapshot.pdf`. *Cited in §3.3.*

Uses: fingerprints, trees, proofs, money

What the world does with a function it cannot analyze (section 2.1).

Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In *Advances in Cryptology — CRYPTO '96*, volume 1109 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 1996. doi: 10.1007/3-540-68697-5_1. Authors' full version archived as ▶ `refs/bellare-canetti-krawczyk-1996-hmac.pdf`. *Cited in §2.1.*

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive,

Paper 2018/046, 2018. URL https://eprint.iacr.org/2018/046. PDF archived as ▶ refs/bensasson-bentov-horesh-riabzev-2018-stark.pdf. *Cited in §2.1.*

Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. doi: 10.1137/0218012. Scanned SIAM version (no text layer) archived as ▶ refs/goldwasser-micali-rackoff-1989-knowledge-complexity.pdf. *Cited in §2.1.*

Leslie Lamport. Password authentication with insecure communication. *Communications of the ACM*, 24(11):770–772, 1981. doi: 10.1145/358790.358797. PDF archived as ▶ refs/lamport-1981-password-chains.pdf. *Cited in §5.4.*

Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, Boca Raton, FL, 1996. ISBN 978-0-8493-8523-0. URL https://cacr.uwaterloo.ca/hac/. Chapter PDFs freely distributed by the authors with publisher permission; Chapter 9 (hash functions) archived as ▶ refs/menezes-vanoorschot-vanstone-1996-hac-chap9.pdf. *Cited in §2.1.*

Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988. doi: 10.1007/3-540-48184-2_32. PDF archived as ▶ refs/merkle-1988-tree-signature.pdf. *Cited in §2.1.*

Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. White paper, 2008. URL https://bitcoin.org/bitcoin.pdf. PDF archived as ▶ refs/nakamoto-2008-bitcoin.pdf. *Cited in §2.1.*

Meltem Sönmez Turan, Elaine Barker, William Burr, and Lily Chen. Recommendation for password-based key derivation, part 1: Storage applications. NIST Special Publication 800-132, 2010. PDF archived as ▶ refs/nist-2010-sp800-132-pbkdf.pdf. *Cited in §5.1 and §5.4.*

Random mappings and analytic combinatorics

Flajolet's telescope (section 4): the exact statistical silhouette of a structureless map.

Philippe Flajolet and Andrew M. Odlyzko. Random mapping statistics. In *Advances in Cryptology — EUROCRYPT '89*, volume 434 of *Lecture Notes in Computer Science*, pages 329–354. Springer, 1990. doi: 10.1007/3-540-46885-4_34. PDF archived as ▶ refs/flajolet-odlyzko-1990-random-mapping-statistics.pdf. *Cited in §4, §4.1, §5.1, and §5.4.*

Philippe Flajolet and Robert Sedgewick. *Analytic Combinatorics*. Cambridge University Press, Cambridge, 2009. ISBN 978-0-521-89806-5. URL https://ac.cs.princeton.edu/home/. Full text freely available from the authors; archived as ▶ refs/flajolet-sedgewick-2009-analytic-combinatorics.pdf. *Cited in §4.2.*

Bernard Harris. Probability distributions related to random mappings. *The Annals of Mathematical Statistics*, 31(4):1045–1062, 1960. doi: 10.1214/aoms/1177705677. Open access; PDF archived as ▶ refs/harris-1960-random-mappings.pdf. *Cited in §4.*

Martin E. Hellman. A cryptanalytic time–memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980. doi: 10.1109/TIT.1980.1056220. PDF archived as ▶ `refs/hellman-1980-time-memory-tradeoff.pdf`. *Cited in §5.4 and §5.4.*

Philippe Oechslin. Making a faster cryptanalytic time–memory trade-off. In *Advances in Cryptology — CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 617–630. Springer, 2003. doi: 10.1007/978-3-540-45146-4_36. PDF archived as ▶ `refs/oechslin-2003-rainbow-tables.pdf`. *Cited in §5.4.*

John M. Pollard. Monte Carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978. doi: 10.1090/S0025-5718-1978-0491431-9. Open archive; PDF archived as ▶ `refs/pollard-1978-monte-carlo-index.pdf`. *Cited in §4.3.*

Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12(1):1–28, 1999. doi: 10.1007/PL00003816. Authors' full version archived as ▶ `refs/vanoorschot-wiener-1999-parallel-collision-search.pdf`. *Cited in §4.3.*

Pseudorandom numbers, finite fields, and mixing

The finished classical theories (section 6.6): linear generators over finite fields, and random walks on groups.

Elaine Barker and John Kelsey. Recommendation for random number generation using deterministic random bit generators. NIST Special Publication 800-90A, Revision 1, 2015. PDF archived as ▶ `refs/nist-2015-sp800-90a.pdf`. *Cited in §6.4.*

Tuğkan Batu, Lance Fortnow, Ronitt Rubinfeld, Warren D. Smith, and Patrick White. Testing closeness of discrete distributions. *Journal of the ACM*, 60(1):4:1–4:25, 2013. doi: 10.1145/2432622.2432626. Authors' version (arXiv:1009.5397) archived as ▶ `refs/batu-2013-closeness.pdf`. *Cited in §6.5.*

Dave Bayer and Persi Diaconis. Trailing the dovetail shuffle to its lair. *The Annals of Applied Probability*, 2(2):294–313, 1992. doi: 10.1214/aoap/1177005705. Open access at Project Euclid; scanned version archived as ▶ `refs/bayer-diaconis-1992-dovetail-shuffle.pdf`. *Cited in §6.6.*

Clément L. Canonne. A survey on distribution testing: Your data is big. but is it blue? *Theory of Computing, Graduate Surveys*, 9:1–100, 2020. doi: 10.4086/toc.gs.2020.009. ECCC report 2015/063; PDF archived as ▶ `refs/canonne-2020-distribution-testing-survey.pdf`. *Cited in §6.4 and §6.5.*

Persi Diaconis. *Group Representations in Probability and Statistics*, volume 11 of *IMS Lecture Notes–Monograph Series*. Institute of Mathematical Statistics, Hayward, CA, 1988. URL `https://projecteuclid.org/ebooks/institute-of-mathematical-statistics-lecture-notes-monograph-series/Group-representations-in-probability-and-statistics/toc/10.1214/lnms/1215467407`. Open access at Project Euclid (chapter downloads at the URL); not archived. *Cited in §6.6.*

Persi Diaconis. The cutoff phenomenon in finite Markov chains. *Proceedings of the National Academy of Sciences*, 93(4):1659–1664, 1996. doi: 10.1073/pnas.93.4.1659. PDF archived as ▶ `refs/diaconis-1996-cutoff-pnas.pdf`. *Cited in §6.6.*

Persi Diaconis and Mehrdad Shahshahani. Generating a random permutation with random transpositions. *Zeitschrift für Wahrscheinlichkeitstheorie und verwandte Gebiete*, 57(2):159–179, 1981. doi: 10.1007/BF00535487. No free copy found; obtain via library. *Cited in §6.6.*

Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, Reading, MA, third edition, 1997. ISBN 978-0-201-89684-8. No free copy exists; obtain via library. Chapter 3 is the classical treatment of random-number testing. *Cited in §6.4 and §6.6.*

Pierre L'Ecuyer and Richard Simard. TestU01: A C library for empirical testing of random number generators. *ACM Transactions on Mathematical Software*, 33(4):22:1–22:40, 2007. doi: 10.1145/1268776.1268777. Available locally as ▶ `refs/lecuyer-simard-2007-testu01.pdf`. *Cited in §6.4 and §6.7.*

Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, Cambridge, second edition, 1997. ISBN 978-0-521-39231-0. No free copy exists; obtain via library. Linear recurring sequences are Chapter 8. *Cited in §6.6.*

George Marsaglia. Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61(1):25–28, 1968. doi: 10.1073/pnas.61.1.25. Open access at PNAS (automated download blocked; not archived). *Cited in §6.4 and §6.6.*

Makoto Matsumoto and Takuji Nishimura. Mersenne twister: A 623-dimensionally equidistributed uniform pseudorandom number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1):3–30, 1998. doi: 10.1145/272991.272995. Available locally as ▶ `refs/matsumoto-nishimura-1998-mersenne-twister.pdf`. *Cited in §6.4 and §6.6.*

Harald Niederreiter. *Random Number Generation and Quasi-Monte Carlo Methods*, volume 63 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. SIAM, Philadelphia, 1992. doi: 10.1137/1.9781611970081. No free copy found; obtain via library. *Cited in §6.6.*

Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, Alan Heckert, James Dray, and San Vo. A statistical test suite for random and pseudorandom number generators for cryptographic applications. NIST Special Publication 800-22, Revision 1a, 2010. PDF archived as ▶ `refs/nist-2010-sp800-22r1a.pdf`. *Cited in §6.4, §6.5, and §6.7.*

Foundations: definitions, idealized models, limits of proof

What "pseudorandom" and "secure" officially mean, and why unconditional theorems are out of reach (section 6).

Mihir Bellare and Tadayoshi Kohno. Hash function balance and its impact on birthday attacks. In *Advances in Cryptology — EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 401–418. Springer, 2004. doi: 10.1007/978-3-540-24676-3_24. Authors' full version archived as ▶ `refs/bellare-kohno-2004-hash-balance.pdf`. *Cited in §2.*

Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *Proceedings of the 1st ACM Conference on Computer and Communications Security (CCS '93)*, pages 62–73. ACM, 1993. doi: 10.1145/168588.168596. Authors' full version archived as ▶ `refs/bellare-rogaway-1993-random-oracles.pdf`. *Cited in §6.2.*

Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited. *Journal of the ACM*, 51(4):557–594, 2004. doi: 10.1145/1008731.1008734. Authors' version archived as ▶ `refs/canetti-goldreich-halevi-2004-random-oracle-revisited.pdf`. *Cited in §6.2.*

Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-damgård revisited: How to construct a hash function. In *Advances in Cryptology — CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 430–448. Springer, 2005. doi: 10.1007/11535218_26. Authors' full version archived as ▶ `refs/coron-dodis-malinaud-puniya-2005-md-revisited.pdf`. *Cited in §6.2.*

Oded Goldreich. *Foundations of Cryptography, Volume 1: Basic Tools*. Cambridge University Press, Cambridge, 2001. ISBN 978-0-521-79172-3. URL `https://www.wisdom.weizmann.ac.il/~oded/frag.html`. Distinguishers and pseudorandomness are Chapter 3; the author's freely posted working draft of that chapter is archived as ▶ `refs/goldreich-1995-foundations-draft-ch3.pdf`. *Cited in §6.1.*

Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986. doi: 10.1145/6490.6503. Scanned JACM version archived as ▶ `refs/goldreich-goldwasser-micali-1986-random-functions.pdf`. *Cited in §6.1.*

Russell Impagliazzo. A personal view of average-case complexity. In *Proceedings of the 10th Annual Structure in Complexity Theory Conference*, pages 134–147. IEEE, 1995. doi: 10.1109/SCT.1995.514853. Available locally as ▶ `refs/impagliazzo-1995-five-worlds.pdf`. *Cited in §6.7 and §8.2.*

Ueli Maurer, Renato Renner, and Clemens Holenstein. Indifferentiability, impossibility results on reductions, and applications to the random oracle methodology. In *Theory of Cryptography (TCC 2004)*, volume 2951 of *Lecture Notes in Computer Science*, pages 21–39. Springer, 2004. doi: 10.1007/978-3-540-24638-1_2. Available locally as ▶ `refs/maurer-renner-holenstein-2004-indifferentiability.pdf`. *Cited in §6.2.*

Phillip Rogaway. Formalizing human ignorance: Collision-resistant hashing without the keys. In *Progress in Cryptology — VIETCRYPT 2006*, volume 4341 of *Lecture Notes in Computer Science*, pages 211–228. Springer, 2006. doi: 10.1007/11958239_14. Available locally as ▶ `refs/rogaway-2006-human-ignorance.pdf`. *Cited in §2.4 and §6.1.*

Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption (FSE 2004)*, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004. doi: 10.1007/978-3-540-25937-4_24. Authors' full version archived as ▶ `refs/rogaway-shrimpton-2004-hash-basics.pdf`. *Cited in §2.1.*

Andrew C. Yao. Theory and applications of trapdoor functions. In *23rd Annual Symposium on Foundations of Computer Science (FOCS 1982)*, pages 80–91. IEEE, 1982. doi: 10.1109/SFCS.1982.45. No free copy found; obtain via library. *Cited in §6.1.*

Hashes with theorems: the provable road

Hash functions whose collision resistance is a theorem—and what that trade costs (section 6.8).

Denis X. Charles, Eyal Z. Goren, and Kristin E. Lauter. Cryptographic hash functions from expander graphs. *Journal of Cryptology*, 22(1):93–113, 2009. doi: 10.1007/s00145-007-9002-x. PDF archived as ▶ `refs/charles-goren-lauter-2009-expander-hash.pdf`. *Cited in §6.8.*

Scott Contini, Arjen K. Lenstra, and Ron Steinfeld. VSH, an efficient and provable collision-resistant hash function. In *Advances in Cryptology — EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 165–182. Springer, 2006. doi: 10.1007/11761679_11. PDF archived as ▶ `refs/contini-lenstra-steinfeld-2006-vsh.pdf`. *Cited in §6.8.*

Shlomo Hoory, Nathan Linial, and Avi Wigderson. Expander graphs and their applications. *Bulletin of the American Mathematical Society*, 43(4):439–561, 2006. doi: 10.1090/S0273-0979-06-01126-8. PDF archived as ▶ `refs/hoory-linial-wigderson-2006-expanders.pdf`. *Cited in §6.8.*

Jean-Pierre Tillich and Gilles Zémor. Hashing with SL_2. In *Advances in Cryptology — CRYPTO '94*, volume 839 of *Lecture Notes in Computer Science*, pages 40–49. Springer, 1994. doi: 10.1007/3-540-48658-5_5. PDF archived as ▶ `refs/tillich-zemor-1994-sl2-hash.pdf`. *Cited in §6.8.*

Jean-Pierre Tillich and Gilles Zémor. Collisions for the LPS expander graph hash function. In *Advances in Cryptology — EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 254–269. Springer, 2008. doi: 10.1007/978-3-540-78967-3_15. PDF archived as ▶ `refs/tillich-zemor-2008-lps-collisions.pdf`. *Cited in §6.8.*

The adversary's calculus: differential and dedicated cryptanalysis

The method that felled MD5 and SHA-1, and the state of its siege of SHA-256 (section 7).

Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology*, 4(1):3–72, 1991. doi: 10.1007/BF00630563. Journal version paywalled; the CRYPTO '90 extended abstract is archived as ▶ `refs/biham-shamir-1990-differential-cryptanalysis.pdf`. *Cited in §3.7 and §7.3.*

Don Coppersmith. The Data Encryption Standard (DES) and its strength against attacks. *IBM Journal of Research and Development*, 38(3):243–250, 1994. doi: 10.1147/rd.383.0243. PDF archived as ▶ `refs/coppersmith-1994-des-strength.pdf`. *Cited in §7.3.*

Christophe De Cannière and Christian Rechberger. Finding SHA-1 characteristics: General results and applications. In *Advances in Cryptology — ASIACRYPT 2006*, volume 4284 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2006. doi: 10.1007/11935230_1. PDF archived as ▶ `refs/decanniere-rechberger-2006-sha1-characteristics.pdf`. *Cited in §7.2.*

Aron Gohr. Improving attacks on round-reduced Speck32/64 using deep learning. In *Advances in Cryptology — CRYPTO 2019*, volume 11693 of *Lecture Notes in Computer Science*, pages 150–179. Springer, 2019. doi: 10.1007/978-3-030-26951-7_6. Authors' version archived as ▶ `refs/gohr-2019-deep-learning-cryptanalysis.pdf`. *Cited in §7.5.*

Dmitry Khovratovich, Christian Rechberger, and Alexandra Savelieva. Bicliques for preimages: Attacks on Skein-512 and the SHA-2 family. In *Fast Software Encryption (FSE 2012)*, volume 7549 of *Lecture Notes in Computer Science*, pages 244–263. Springer, 2012. doi: 10.1007/978-3-642-34047-5_15. PDF archived as ▶ `refs/khovratovich-rechberger-savelieva-2012-bicliques.pdf`. *Cited in §6.4 and §7.4.*

Yingxin Li, Fukang Liu, and Gaoli Wang. New records in collision attacks on SHA-2. In *Advances in Cryptology — EUROCRYPT 2024*, volume 14651 of *Lecture Notes in Computer Science*, pages 158–186. Springer, 2024. doi: 10.1007/978-3-031-58716-0_6. Authors' full version archived as ▶ `refs/li-liu-wang-2024-sha2-collision-records.pdf`. *Cited in §6.4, §7.2, §7.4, and §8.1.*

Helger Lipmaa and Shiho Moriai. Efficient algorithms for computing differential properties of addition. In *Fast Software Encryption (FSE 2001)*, volume 2355 of *Lecture Notes in Computer Science*, pages 336–350. Springer, 2002. doi: 10.1007/3-540-45473-X_28. PDF archived as ▶ `refs/lipmaa-moriai-2001-addition-differential.pdf`. *Cited in §7.1 and §7.4.*

Florian Mendel, Tomislav Nad, and Martin Schläffer. Improving local collisions: New attacks on reduced SHA-256. In *Advances in Cryptology — EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 262–278. Springer, 2013. doi: 10.1007/978-3-642-38348-9_16. Available locally as ▶ `refs/mendel-nad-schlaffer-2013-sha256-local-collisions.pdf`. *Cited in §6.4, §7.2, and §7.4.*

Computability, complexity, and barriers

Where breaking SHA-256 sits in the complexity landscape (section 8): NP-hardness, SAT solvers, quantum limits, and proof barriers.

Scott Aaronson and Yaoyun Shi. Quantum lower bounds for the collision and the element distinctness problems. *Journal of the ACM*, 51(4):595–605, 2004. doi: 10.1145/1008731.1008735. Author's version archived as ▶ `refs/aaronson-shi-2004-collision-lower-bound.pdf`. *Cited in §8.4.*

Adi Akavia, Oded Goldreich, Shafi Goldwasser, and Dana Moshkovitz. On basing one-way functions on NP-hardness. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing (STOC 2006)*, pages 701–710. ACM, 2006. doi: 10.1145/1132516.1132614. PDF archived as ▶ `refs/akavia-goldreich-goldwasser-moshkovitz-2006-owf-np.pdf`. *Cited in §8.1.*

Nahiyan Alamgir, Saeed Nejati, and Curtis Bright. SHA-256 collision attack with programmatic SAT. *arXiv preprint arXiv:2406.20072*, 2024. Available locally as ▶ `refs/alamgir-nejati-bright-2024-sha256-sat.pdf`. *Cited in §7.4 and §8.1.*

Andrew W. Appel. Verification of a cryptographic primitive: SHA-256. *ACM Transactions on Programming Languages and Systems*, 37(2):7:1–7:31, 2015. doi: 10.1145/2701415. PDF archived as ▶ `refs/appel-2015-verified-sha256.pdf`. *Cited in §8.5.*

Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 26 (5):1510–1523, 1997. doi: 10.1137/S0097539796300933. Available locally as ▶ `refs/bennett-bernstein-brassard-vazirani-1997-quantum.pdf`. *Cited in §8.4.*

Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC 1971)*, pages 151–158. ACM, 1971. doi: 10.1145/800157.805047. No free copy found; obtain via library. *Cited in §8.1.*

Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4): 1364–1396, 1999. doi: 10.1137/S0097539793244708. Authors' version archived as ▶ `refs/hastad-impagliazzo-levin-luby-1999-prg-owf.pdf`. *Cited in §8.2.*

Leonid A. Levin. The tale of one-way functions. *Problems of Information Transmission*, 39(1):92–103, 2003. doi: 10.1023/A:1023634616182. PDF archived as ▶ `refs/levin-2003-tale-one-way.pdf`. *Cited in §8.2.*

Yanyi Liu and Rafael Pass. On one-way functions and Kolmogorov complexity. In *Proceedings of the 61st IEEE Symposium on Foundations of Computer Science (FOCS 2020)*, pages 1243–1254. IEEE, 2020. doi: 10.1109/FOCS46700.2020.00118. PDF archived as ▶ `refs/liu-pass-2020-kolmogorov-owf.pdf`. *Cited in §8.1.*

Ilya Mironov and Lintao Zhang. Applications of SAT solvers to cryptanalysis of hash functions. In *Theory and Applications of Satisfiability Testing (SAT 2006)*, volume 4121 of *Lecture Notes in Computer Science*, pages 102–115. Springer, 2006. doi: 10.1007/11814948_13. PDF archived as ▶ `refs/mironov-zhang-2006-sat-hash.pdf`. *Cited in §8.1.*

Alexander A. Razborov and Steven Rudich. Natural proofs. *Journal of Computer and System Sciences*, 55(1):24–35, 1997. doi: 10.1006/jcss.1997.1494. No free copy retrievable by automation; freely readable via the author's page and JCSS open backfile. *Cited in §8.4.*

Jean Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. HACL*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS 2017)*, pages 1789–1806. ACM, 2017. doi: 10.1145/3133956.3134043. PDF archived as ▶ `refs/zinzindohoue-bhargavan-protzenko-beurdouche-2017-hacl.pdf`. *Cited in §8.5.*